MACSE502 Programming for Data Science

Topic notes with practice questions, Modules 1 to 6

Short notes on every topic and subtopic in the syllabus, each followed by sample questions with worked answers: 273 questions in 60 sections, 84 of them tagged **tricky**. The tricky topics (PCA, K-means, broadcasting, pandas traps, data leakage, Scala evaluation and linearization, Spark laziness, shuffles and closures) get extra depth and hand-worked numbers. Every output printed here comes from a program that was run.

What is inside

Module	Title	Sections	Questions	Tricky
1	Python data structures, OOP, APIs and NLP	16	74	25
2	Python libraries for data science	15	82	27
3	Scala data structures and object-oriented programming	13	58	18
4	Scala libraries and Spark basics	9	40	10
5	Programming language selection criteria	3	10	2
6	Contemporary issues	4	9	2

How to use these notes

1. Read the short notes at the top of a section (they are a summary of the syllabus topic, not a textbook).
2. Cover the answer under each question, try it on paper, then compare. Questions tagged **tricky** are the ones that usually cost marks; redo them after a day.
3. Run or read the demo programs for the sections you find hard. Every program is in *Question_Bank/code* and its real output in *Question_Bank/outputs*.
4. Use the finder below to jump to a tricky topic, the cheat sheet at the back for last-minute revision, and the companion *Question Bank* PDF for full exam-style questions with marks splits.

Tags: **[2 marks]** etc. show the size of answer expected (my estimate, not an official weight); **[predict the output]** asks you to work out what a program prints; **[tricky]** marks a common trap.

Tricky-topic finder

Topic	Where	The trap in one line
PCA by hand, scaling, choosing k, signs, limits	2.11	variance depends on units; centre, scale, eigen-decompose; signs are arbitrary
K-means: one iteration, local minima, choosing k, scaling	2.12	result depends on start; scale features; inertia always falls with k
Data leakage, pipelines, accuracy paradox, overfitting	2.10	fit on training data only; accuracy hides a missed minority class
Gradient step by hand, zero_grad, int64 labels, early stopping	2.13, 2.14	PyTorch accumulates gradients; validation set is not the test set
NumPy broadcasting, views vs copies, dtype wrap-around	2.8	right-align shapes; slices share memory; uint8 wraps
pandas: loc vs iloc, chained assignment, alignment, merge explosion	2.1, 2.2, 2.6	end label included; write through one `.loc`; many-to-many multiplies rows
Python aliasing, mutable defaults, `t[0] += ...`, copies	1.2, 1.3, 1.6	names share objects; defaults are created once
Heap is not sorted; priority ties; `pop(0)` cost	1.8, 1.9	add a counter; use deque
`finally` with return, exception order, MRO	1.11, 1.12	finally can override return; specific handlers first
REST status codes, retry rules, timeouts	1.13	retry 429/5xx, not 404; always set a timeout
File modes, Windows `seek`, JSON round trip	1.14	`w` truncates; tuples become lists
Stemming vs lemmatization, stopwords and negation	1.15	lemmatizer needs the POS tag; `not` is a stopword
`val` / `def` / `lazy val`, call-by-name	3.1	def runs at every use; by-name re-evaluates
foldLeft vs foldRight, `::` associativity, Array equality, `'5'.toInt`	3.3, 3.4, 3.5	direction matters; arrays compare by reference; Char gives a code
Initialisation order, case class with an Array, companion access	3.7, 3.8, 3.9	overridden `val` is null in the super-constructor
Trait linearization and mixin order	3.11	`B with C` gives D, C, B, A; the order of `with` changes results
Try vs NonFatal vs Either, `.get` on Failure	4.3	fatal errors are not caught by Try; `.get` re-throws
Lazy evaluation, cache, shuffle, reduceByKey vs groupByKey	4.6	each action recomputes lineage unless cached
Closures, accumulators, broadcast	4.7	driver variables are not updated from tasks
Catalyst plans, DataFrame vs Dataset	4.8	filters merge, literals fold, typos fail at different times
Decision-matrix arithmetic and weaknesses	5.3	show the weighted sum line by line

If you are preparing for CAT 2 (your predicted syllabus file)

Predicted item	Read
Module 2: data visualisation (Matplotlib, Seaborn, Bokeh, Plotly, Folium)	2.15
Module 3: functions, strings, collections, combinators, classes, case classes, traits, access modifiers, REPL	3.1 to 3.13
Module 4: Breeze, Saddle, exception handling, Spark architecture, big data, components	4.1 to 4.9
Module 5: selection criteria	5.1 to 5.3

Check this against what your faculty actually announced for CAT 2; the file only holds a prediction.

Contents

Module 1: Python data structures, OOP, APIs and NLP **5**

1.1 Conditions and branching . . . 5
1.2 Lists . . . 6
1.3 Tuples . . . 7
1.4 Dictionaries . . . 7
1.5 Sets . . . 8
1.6 Mutability, aliasing and copying (cross-cutting traps) . . . 9
1.7 Stack (user-defined structure) . . . 9
1.8 Queue (user-defined structure) . . . 10
1.9 Priority queue and heapq . . . 11
1.10 String handling . . . 12
1.11 Exception handling . . . 13
1.12 Object-oriented concepts . . . 13
1.13 APIs, REST and HTTP requests . . . 14
1.14 File handling (text, CSV, JSON) . . . 16
1.15 NLTK (natural language processing) . . . 17
1.16 ChatterBot . . . 18

Module 2: Python libraries for data science **20**

2.1 Pandas: Series and DataFrame . . . 20
2.2 Selecting and filtering data . . . 21
2.3 Handling missing values . . . 22
2.4 Built-in functions and data operations . . . 22
2.5 Working with text data . . . 23
2.6 Merging, joining, concatenating and grouping . . . 23
2.7 Data mining: Beautiful Soup and Scrapy . . . 25
2.8 NumPy . . . 26
2.9 SciPy . . . 28
2.10 Machine-learning workflow with scikit-learn . . . 29
2.11 PCA in depth (tricky topic) . . . 30
2.12 K-means clustering in depth (tricky topic) . . . 34
2.13 Deep learning frameworks: Keras, TensorFlow, PyTorch . . . 35
2.14 XGBoost . . . 37
2.15 Data visualisation: Matplotlib, Seaborn, Bokeh, Plotly, Folium . . . 38

Module 3: Scala data structures and object-oriented programming **41**

3.1 Functions: expanded format, short forms, higher-order functions . . . 41
3.2 Variables, strings and user input . . . 42
3.3 Numbers, variable types, operators and booleans . . . 43
3.4 Arrays, lists, tuples, sets, hash sets and maps . . . 44
3.5 Functional combinators . . . 45
3.6 Option and pattern matching . . . 46
3.7 Classes, objects, constructors, this and method overloading . . . 47
3.8 Singleton, anonymous and companion objects . . . 47
3.9 Case classes . . . 48
3.10 Inheritance, field overriding, final and abstract classes . . . 49
3.11 Traits, mixins and linearization (tricky topic) . . . 50
3.12 Access modifiers . . . 50
3.13 Scala arrays and the REPL . . . 51

Module 4: Scala libraries and Spark basics **52**

4.1 Breeze (numerical computing) . . . 52
4.2 Saddle (data frames) and Spire (exact numerics) . . . 53
4.3 Exception handling in Scala . . . 54
4.4 Big data and Apache Spark: why Spark . . . 55
4.5 Spark architecture . . . 55
4.6 RDDs, transformations, actions and lazy evaluation (tricky topic) . . . 56
4.7 Shared variables and closures (tricky topic) . . . 58
4.8 Spark SQL, DataFrames, Datasets and Catalyst . . . 58
4.9 Spark components: Core, SQL, Streaming, MLlib, GraphX . . . 60

Module 5: Programming language selection criteria **61**

5.1 The language-selection criteria . . . 61
5.2 Python, Scala, Java and R compared . . . 62
5.3 Weighted decision matrix (worked example) . . . 62

Module 6: Contemporary issues **64**

6.1 Privacy, security and ethics in data work . . . 64
6.2 Bias, fairness and responsible AI . . . 64
6.3 Generative AI and AI coding assistants . . . 65
6.4 Model and data drift, monitoring, reproducibility . . . 65

Cheat sheet and last-minute revision **66**

Index of tricky questions **68**

Appendix: how the demonstrations were produced **72**

Module 1: Python data structures, OOP, APIs and NLP

Syllabus (8 hours): conditions and branching; list, tuple, dictionary, set; stack, queue, priority queue; string handling methods; exception handling; object-oriented concepts; APIs and data collection, simple and REST APIs, HTTP requests; file handling; NLTK; ChatterBot.

Where each syllabus topic is covered

Syllabus topic	Section
Condition and branching statements	1.1
Built-in data structures: list, tuple, dictionary, set	1.2 to 1.5 (copying traps: 1.6)
User-defined data structures: stack, queue, priority queue	1.7, 1.8, 1.9
String handling methods	1.10
Exception handling	1.11
Object-oriented concepts	1.12
APIs and data collection, simple API and REST APIs, HTTP requests	1.13
File handling, read/write	1.14
Frameworks and libraries: NLTK, ChatterBot	1.15, 1.16

1.1 Conditions and branching

- `if / elif / else` run the first branch whose condition is true; indentation (4 spaces) defines the block. Order the tests from most specific to least specific.
- Comparison: `== != < > <= >=`, chained `1 < x < 10`. Logical: `and or not`. Membership: `in`, `not in`. Identity: `is`.
- **Truthiness**: `0, 0.0, '', [], {}, set(), None` are false; everything else is true.
- `and` / `or` short-circuit and **return one of their operands**, not necessarily `True` or `False`.
- Conditional expression: `a if cond else b`. The `match` statement (Python 3.10+) does structural pattern matching.

Verified output

```
--- truthiness and short-circuit (and / or return an OPERAND, not True/False) ---
falsy values: [False, False, False, False, False, False, False] | truthy: [True, True, True, True, True]
0 or 'x' -> 'x' | '' and 5 -> '' | 3 and 4 -> 4 | None or [] -> []
short-circuit: right-hand side skipped, calls made = [False, True]
chained comparison 1 < x < 10: True | not x == 7: False | x != 7: False
ternary: odd | 'in': True 'z' not in 'abc': True
None test: a is None -> True | [] == None -> False | 0.1 + 0.2 == 0.3 -> False
math.isclose(0.1 + 0.2, 0.3) -> True

--- if / elif order matters ---
wrong order for 90, 75, 55, 30: ['C', 'C', 'C', 'Fail']
right order for 90, 75, 55, 30: ['A', 'B', 'C', 'Fail']
match statement (Python 3.10+): ['origin', 'on x-axis at 3', 'on y-axis at 4', 'dict with name A', 'elsewhere']
```

1.1-Q1 What does `print(0 or 'x', '' and 5, 3 and 4)` print? [predict the output]

`x 4` (the middle item is the empty string, so there are two spaces). `0 or 'x'` returns the first truthy operand `'x'`; `'' and 5` stops at the first falsy operand and returns `''`; `3 and 4` has no falsy operand so it returns the last one, `4`.

1.1-Q2 A grading function uses `if m >= 50: 'C' elif m >= 70: 'B' elif m >= 85: 'A'`. What is wrong? [tricky]

The first test already catches every mark of 50 or more, so `B` and `A` can never be returned. In the output above the wrong order gives `C, C, C, Fail` for 90, 75, 55, 30. Test the highest bound first: `>= 85`, then `>= 70`, then `>= 50`.

1.1-Q3 Why is `if x == None` discouraged, and when is `is` correct? [2 marks]

`==` can be overridden by a class; `is` tests identity, and there is exactly one `None`, so write `x is None`. Use `is` only for `None`, `True`, `False`. For numbers and strings use `==` (identity of equal values is an implementation detail).

1.1-Q4 Why does `0.1 + 0.2 == 0.3` give `False`, and how do you compare floats? [tricky]

Binary floating point cannot store 0.1 or 0.2 exactly, so the sum is 0.30000000000000004. Compare with a tolerance: `math.isclose(a, b)` (or `abs(a - b) < 1e-9`).

1.1-Q5 Rewrite `if n % 2 == 0: label = 'even' else: label = 'odd'` as one line. [1 mark]

```
label = 'even' if n % 2 == 0 else 'odd'    # condition goes in the middle
```

1.2 Lists

- Ordered, mutable, allow duplicates, any mix of types. Implemented as a dynamic array of references.
- Indexing from 0, negative indexes from the end, slicing `a[start:stop:step]` (stop excluded) returns a new list.

Operation	Cost	Note
`a[i]`, `a[i] = x`, `len(a)`	O(1)	
`a.append(x)`, `a.pop()`	O(1) amortised	work at the right end
`a.insert(i, x)`, `a.pop(0)`, `del a[0]`	O(n)	every later element shifts
`x in a`, `a.index(x)`, `a.count(x)`, `a.remove(x)`	O(n)	linear scan
`a.sort()`, `sorted(a)`	O(n log n)	stable; `sort` returns None
slice `a[i:j]`	O(j - i)	creates a new list

Verified output

```
--- list methods ---
append adds ONE element: [1, 2, 3, [4, 5]]
extend adds each element: [1, 2, 3, 4, 5] | L + [6] makes a new list: [1, 2, 3, 4, 5, 6] | L: [1, 2, 3, 4, 5]
insert(1, 99): [1, 99, 2, 3, 4, 5]
remove(99) deletes the first match: [1, 2, 3, 4, 5]
pop() -> 5 | pop(0) -> 1 | list now: [2, 3, 4]
remove of a missing item -> list.remove(x): x not in list
index(5): 0 (first match) | count(5): 2 | reverse in place: [1, 5, 3, 5] | len: 4 | min/max/sum: 1 5 14
del M[0]: [5, 3, 5]
sorted(key=len): ['a', 'bb', 'ccc'] | reverse=True: [3, 2, 1] | sort is stable: [(0, 'z'), (1, 'b'), (1, 'a')]
enumerate: [(1, 'a'), (2, 'b')] | zip: [(1, 'a'), (2, 'b')] | unzip: [(1, 2), ('a', 'b')]

--- comprehensions ---
squares of evens: [0, 4, 16, 36, 64] | nested: [(0, 0), (0, 1), (1, 0), (1, 1)]
if-else goes BEFORE for: ['even', 'odd', 'even', 'odd'] | filter goes AFTER: [4, 5]
dict comp: {'ab': 2, 'cde': 3} | set comp: {0, 1, 2} | generator sum: 10
```

Verified output

```
--- list: aliasing, copies, sort ---
a = [3, 1, 2, 9] | c = [3, 1, 2] | a is b: True
sorted(a) -> [1, 2, 3, 9] | a unchanged: [3, 1, 2, 9]
a.sort() returns None | a is now [1, 2, 3, 9]
s[2:8:2] = [2, 4, 6] | s[::-1] = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] | s[-3:] = [7, 8, 9]
```

1.2-Q1 Difference between `append` and `extend`? Between `a + b` and `a += b`? [2 marks]

`append(x)` adds one element (even a list: `[1,2,3,[4,5]]`); `extend(it)` adds each element. `a + b` builds a new list; `a += b` extends `a` in place, so other names bound to `a` see the change.

1.2-Q2 After `a = [3, 1, 2]; b = a; c = a[:]; b.append(9)`, what are `a` and `c`? [tricky]

`a` is `[3, 1, 2, 9]` and `c` is `[3, 1, 2]`. `b = a` copies the reference (same object); `a[:]` makes a new list. Check with `a is b` (True).

1.2-Q3 What does `grid = [[0] * 2] * 2; grid[0][0] = 1` give? How do you build a real 2 x 2 grid? [tricky]

`[[1, 0], [1, 0]]`: multiplication repeats the *same* inner list twice, so both rows are one object. Build rows independently with a comprehension: `[[0] * 2 for _ in range(2)]`.

1.2-Q4 What is the output of `bad(1), bad(2)` for `def bad(x, acc=[]): acc.append(x); return acc`? [tricky]

`[1, 2] [1, 2]`. The default list is created once, when the function is defined, and shared by all calls. Use `acc=None` and create the list inside (output above shows `good` returns `[1] [2]`).

1.2-Q5 What does `print(a.sort())` print and why? [tricky]

`None`. `list.sort()` sorts in place and returns `None`; `sorted(a)` returns a new sorted list. Writing `a = a.sort()` destroys your list.

1.2-Q6 Write a comprehension for the squares of the even numbers below 10, and one that labels each of 0..3 as even or odd. [2 marks]

```
[n * n for n in range(10) if n % 2 == 0]             # [0, 4, 16, 36, 64]   (filter AFTER the loop)
['even' if n % 2 == 0 else 'odd' for n in range(4)]   # if-else goes BEFORE the loop
```

1.2-Q7 Why is repeated `pop(0)` on a long list a performance bug? [2 marks]

Each call shifts all remaining elements (O(n)). In the run above, 20,000 removals from the front of a 100,000-item list took about 3,300 ms with `list.pop(0)` and 1.3 ms with `deque.popleft()` (one machine, your numbers will differ).

1.3 Tuples

- Ordered, **immutable**, allow duplicates. Created with commas: `(1, 2)`; a one-element tuple needs a trailing comma: `(5,)`.
- Hashable when every element is hashable, so tuples can be dictionary keys and set members.
- Typical uses: fixed records `(name, cgpa)`, returning several values, unpacking `a, b = b, a`, `first, *rest = t`, `collections.namedtuple` for named fields.

Verified output

```
--- tuple ---
append inside a tuple's list works: ([1, 3], 2)
t[0] += [4] raises TypeError: 'tuple' object does not support item assignment | but t is now ([1, 3, 4], 2)
one-item tuple: (5,) tuple int
swap: 2 1 | unpack: 10 [20, 30]
namedtuple: P(name='Asha', cgpa=8.6) Asha 8.6
```

1.3-Q1 Differentiate list and tuple. [3 marks]

	List	Tuple
Mutable	yes	no
Syntax	`[1, 2]`	`(1, 2)` or `1, 2`
Hashable	no	yes if items are
Use	collection that changes	fixed record, dict key, return value

1.3-Q2 `t = ([1], 2)`. Does `t[0].append(3)` work? What does `t[0] += [4]` do? [tricky]

`t[0].append(3)` works: the tuple still holds the same list, and the list is mutable, giving `([1, 3], 2)`. `t[0] += [4]` raises `TypeError` because the assignment half tries to rebind `t[0]`, **but the list was already extended**, so `t` becomes `([1, 3, 4], 2)`. Immutability applies to the tuple's references, not to the objects they point to.

1.3-Q3 What is the type of `(5)` and of `(5,)`? [1 mark]

`int` and `tuple`. The comma makes the tuple, not the brackets.

1.3-Q4 Swap two variables without a temporary, and unpack `10, 20, 30` into a head and a tail. [2 marks]

```
x, y = y, x
first, *rest = (10, 20, 30)     # first = 10, rest = [20, 30]
```

1.4 Dictionaries

- Key-value mapping; keys unique and **hashable**; insertion order is preserved (Python 3.7+). Lookup, insert, delete are O(1) on average (hash table).
- `d[k]` raises `KeyError` if missing; `d.get(k, default)` does not; `d.setdefault(k, v)` inserts if missing; `d.items()`, `.keys()`, `.values()` are live views.
- `collections.defaultdict(list)` creates missing entries automatically; `collections.Counter` counts items.
- Merge: `d1 | d2` (3.9+) or `{**d1, **d2}`; later keys win. Comprehension: `{k: f(k) for k in xs}`.

Verified output

```
--- dict ---
get missing: None 0
setdefault: {'a': 1, 'b': [1]}
defaultdict: {'t': ['to', 'to'], 'b': ['be', 'be'], 'o': ['or'], 'n': ['not']}
Counter: [('to', 2), ('be', 2)]
modify while iterating -> dictionary changed size during iteration
list as key -> unhashable type: 'list'
tuple key OK: {(1, 2): 'x'} | merge: {'a': 2, 'b': 3}
```

1.4-Q1 Why can a tuple but not a list be a dictionary key? [tricky]

Keys are located by their hash, which must not change. Lists are mutable and unhashable (`TypeError: unhashable type: 'list'`, shown above); a tuple of hashable items is hashable.

1.4-Q2 What happens if you add keys to a dictionary while looping over it? [tricky]

`RuntimeError: dictionary changed size during iteration`. Loop over a copy (`list(d)`) or build a new dict.

1.4-Q3 Count the words in a sentence with and without `Counter`. [2 marks]

```
counts = {}
for w in text.split():
    counts[w] = counts.get(w, 0) + 1

from collections import Counter
Counter(text.split()).most_common(2)
```

1.4-Q4 Difference between `d[k]`, `d.get(k)` and `d.setdefault(k, v)`. [2 marks]

`d[k]` raises `KeyError` when missing; `d.get(k)` returns `None` (or a given default) and does not change `d`; `setdefault` returns the value and *inserts* `v` when the key is missing.

1.4-Q5 Why is dictionary lookup O(1) on average but not always? [2 marks]

A hash maps the key to a slot. Collisions (two keys, same slot) are resolved by probing, so the cost is constant on average and degrades only when many keys collide.

1.5 Sets

- Unordered collection of unique, hashable items; membership test O(1) on average. `{}` is an empty **dict**; use `set()` for an empty set.
- Algebra: union `|`, intersection `&`, difference `-`, symmetric difference `^`, subset `<=`. `frozenset` is the immutable version (usable as a dict key).
- Use for removing duplicates, fast membership, and comparing groups (eligible-for-all, eligible-for-none questions).

Verified output

```
--- set ---
type({}) = dict | type(set()) = set
A|B {1, 2, 3, 4, 5} A&B {3, 4} A-B {1, 2} A^B {1, 2, 5} A<=B False
set of lists -> unhashable type: 'list'
frozenset usable as key: {frozenset({1, 2}): 'ok'}
duplicates removed: {1, 2, 3}

--- membership speed (100000 items, 200 lookups of the worst case) ---
list 134.02 ms, set 0.0135 ms -> set is about 9,927x faster here
```

1.5-Q1 What are `type({})` and `type(set())`? [tricky]

`dict` and `set`. There is no literal for an empty set.

1.5-Q2 Why can a set contain tuples but not lists? [2 marks]

Set members must be hashable; lists are mutable and unhashable (`TypeError`, shown above).

1.5-Q3 Given A = {1,2,3,4} and B = {3,4,5}, evaluate A|B, A&B, A-B, A^B. [2 marks]

`{1,2,3,4,5}`, `{3,4}`, `{1,2}`, `{1,2,5}` (see the output).

1.5-Q4 Why is a set preferred over a list for 'is this roll number registered?' [2 marks]

Hashing gives O(1) average membership. In the run above a worst-case lookup in a 100,000-item list took about 0.67 ms per call against roughly 0.00007 ms for the set (about 10,000 times faster on this machine).

1.6 Mutability, aliasing and copying (cross-cutting traps)

Immutable	Mutable
`int, float, bool, str, tuple, frozenset, bytes, None`	`list, dict, set, bytearray`, most user-defined objects

- Python passes **references to objects** ("pass by object reference"). A function can mutate a mutable argument; rebinding the parameter name changes only the local name.
- `b = a` shares the object. `a.copy()` / `a[:]` / `list(a)` make a **shallow** copy (new outer container, same inner objects). `copy.deepcopy(a)` copies everything.

Verified output

```
--- how arguments are passed: by object reference ---
after grow, rebind, bump: [1] 5
== compares values, is compares identity: True False True | id equal for aliases: True
rule: use == for values; use `is` only for None, True and False
```

Verified output

```
--- shallow vs deep copy, list multiplication ---
shallow: [[1, 2, 99], [3]] | deep: [[1, 2], [3]]
[[0]*2]*2 then grid[0][0]=1 -> [[1, 0], [1, 0]] (rows are the same object)
comprehension version        -> [[1, 0], [0, 0]]
```

1.6-Q1 Predict `data` and `num` after `grow(data)`, `rebind(data)`, `bump(num)` for the three functions in the output. [predict the output]

`[1] 5`. `grow` mutates the caller's list; `rebind` assigns a new list to the local name only; `bump` makes a new int because ints are immutable.

1.6-Q2 `nested = [[1, 2], [3]]; sh = nested.copy(); nested[0].append(99)`. What is `sh`? [tricky]

`[[1, 2, 99], [3]]`. A shallow copy duplicates only the outer list; the inner lists are shared. `copy.deepcopy(nested)` would still be `[[1, 2], [3]]`.

1.6-Q3 When do you use `==` and when `is`? [1 mark]

`==` compares values, `is` compares identity (same object). Use `is` for `None` only.

1.7 Stack (user-defined structure)

- LIFO: last in, first out. Operations: `push`, `pop`, `peek`, `is_empty`, `size`, all O(1) when the top is the *end* of a list.
- Applications: undo/redo, function-call stack and recursion, bracket matching, expression evaluation and conversion, back button, depth-first search.

Stack, Queue and PriorityQueue classes (from n1_queues.py)

```
class Stack:
    def __init__(self): self._data = []
    def push(self, x): self._data.append(x)
    def pop(self):
        if not self._data:
            raise IndexError("pop from empty stack")
        return self._data.pop()
    def peek(self):
        if not self._data:
            raise IndexError("peek at empty stack")
        return self._data[-1]
    def is_empty(self): return not self._data
    def __len__(self): return len(self._data)

class Queue:
    def __init__(self): self._data = deque()
    def enqueue(self, x): self._data.append(x)
    def dequeue(self):
        if not self._data:
            raise IndexError("dequeue from empty queue")
        return self._data.popleft()
    def is_empty(self): return not self._data
    def __len__(self): return len(self._data)

class PriorityQueue:
    """Smaller number = higher priority; the counter keeps first-come-first-served among equals."""
    def __init__(self): self._h, self._c = [], itertools.count()
    def push(self, item, priority): heapq.heappush(self._h, (priority, next(self._c), item))
    def pop(self):
        if not self._h:
            raise IndexError("pop from empty priority queue")
        return heapq.heappop(self._h)[2]
```

Verified output

```
--- stack (LIFO) ---
peek: 3 | pop: 3 | now: [1, 2]
balanced: [True, False, False, True]

--- user-defined classes (the usual exam question) ---
Stack: pop -> 3 | peek -> 2 | len -> 2
Queue: dequeue -> a | len -> 2
PriorityQueue pops: ['fix outage', 'fix leak', 'reset password', 'print report']
empty stack -> pop from empty stack
```

1.7-Q1 Write a stack class with `push`, `pop`, `peek`, `is_empty`; what should `pop` do on an empty stack? [5 marks]

See the class above. Raise a clear exception (`IndexError('pop from empty stack')`) instead of returning a silent value; the caller can catch it.

1.7-Q2 Check whether the brackets in a string are balanced using a stack. [5 marks]

Push every opening bracket; for each closing bracket pop and compare with the matching opener; the string is balanced if no mismatch occurs and the stack is empty at the end. Output above: `{[()]}` True, `([)]` False, `((` False, empty string True.

1.7-Q3 How are two stacks used for undo and redo? [3 marks]

Every new edit goes on the undo stack and clears the redo stack; undo pops the undo stack and pushes the same action on the redo stack; redo does the reverse (see the question bank, M1-C2).

1.8 Queue (user-defined structure)

- FIFO: first in, first out. Operations `enqueue` (append at the rear) and `dequeue` (remove at the front).
- Use `collections.deque`: `append` and `popleft` are O(1). `deque(maxlen=n)` keeps only the newest n items; `rotate(k)` rotates.
- `queue.Queue` (FIFO), `LifoQueue` (stack) and `PriorityQueue` are the **thread-safe** versions for producer/consumer programs.
- Applications: CPU/job scheduling, printer spooling, breadth-first search, buffering, call-centre waiting lines.

Verified output

```
--- queue (FIFO) with deque ---
popleft: 1 | now: [2, 3, 4]
rotate(1): [4, 2, 3]
deque(maxlen=3) keeps the newest: [3, 4, 5]
20000 removals from the front: list.pop(0) 3332.1 ms, deque.popleft() 1.32 ms

--- library queues ---
Queue: [3, 1, 2] | LifoQueue: [2, 1, 3] | PriorityQueue: [1, 2, 3] (thread-safe versions)
```

1.8-Q1 Why should a queue not be built with `list.pop(0)`? [2 marks]

It is O(n) per removal because every item shifts; `deque.popleft()` is O(1). Measured above: about 3,300 ms versus 1.3 ms for 20,000 removals.

1.8-Q2 What is a circular queue and why is it used? [3 marks]

A fixed-size array treated as a ring, with `front` and `rear` indexes that wrap around using modulo. It reuses freed slots, avoiding the wasted space of a plain array queue. `deque(maxlen=n)` behaves similarly for a buffer of the latest n items.

1.8-Q3 Difference between `collections.deque` and `queue.Queue`. [2 marks]

`deque` is a fast single-thread container. `queue.Queue` adds locking so several threads can `put` and `get` safely, and can block or time out.

1.8-Q4 Compare a stack and a queue. [3 marks]

	Stack	Queue
Order	LIFO	FIFO
Insert / remove	same end (top)	rear / front
Python	list `append`/`pop`	`deque` `append`/`popleft`
Uses	undo, recursion, brackets	scheduling, BFS, buffering

1.9 Priority queue and heapq

- Each item has a priority; the item with the best priority leaves first. Python: `heapq` keeps a list arranged as a binary **min-heap**.
- `heappush`, `heappop` O(log n); `h[0]` (peek) O(1); `heapify(list)` O(n); `nlargest(k, xs)`, `nsmallest`.
- Max-heap: push negated priorities. Ties: `(priority, counter, item)` keeps arrival order and avoids comparing the items themselves.
- A heap list is **not sorted**; only `h[0]` is guaranteed smallest (parent <= children).

Verified output

```
--- priority queue with heapq ---
heap list is NOT sorted: [1, 3, 2, 5, 9, 8] | data[0] is the minimum: 1
pop order: [0, 1, 2, 3, 5, 8, 9]
nlargest(2): [9, 8] | nsmallest(2): [1, 2]
max-heap by negating: 8

--- tie-breaking and non-comparable items ---
equal priority falls back to the next field: (1, 'task A')
same priority + objects without ordering -> '<' not supported between instances of 'Job' and 'Job'
with a counter, order is priority then arrival: ['urgent 1', 'urgent 2', 'routine 1', 'routine 2']

cost: heappush/heappop O(log n), peek O(1), heapify O(n)
```

1.9-Q1 `data = [5, 3, 8, 1, 9, 2]; heapq.heapify(data)`. Is `data` now sorted? [tricky]

No. It is `[1, 3, 2, 5, 9, 8]`: a valid heap (every parent <= its children) with the minimum at index 0. Repeated `heappop` returns sorted order.

1.9-Q2 Two tasks have the same priority and the second field is a custom object. What goes wrong with `heappush((1, obj))`? [tricky]

When priorities tie, Python compares the next tuple field; objects without `__lt__` raise `TypeError: '<' not supported between instances of 'Job' and 'Job'`. Insert a unique counter as the second field: `(priority, next(counter), obj)`, which also gives first-come-first-served among equals.

1.9-Q3 How do you get highest-priority-first (a max-heap) from `heapq`? [2 marks]

Push the negated priority, e.g. `(-cgpa, roll)`, and negate again after popping.

1.9-Q4 Where is a priority queue used? [2 marks]

CPU scheduling, Dijkstra and A* shortest paths, event simulation, hospital triage, top-k queries, merging sorted streams, the urgent-before-routine complaint chatbot.

1.10 String handling

- Immutable sequences of Unicode characters; every 'change' creates a new string. Slicing and indexing like lists: `s[::-1]` reverses.
- Cleaning: `strip lstrip rstrip lower upper title casefold replace`. Splitting: `split rsplit partition splitlines`; joining: `sep.join(items)`.
- Searching: `find` (returns -1) vs `index` (raises `ValueError`); `startswith/endswith(tuple)`; `count`. Tests: `isdigit isalpha isalnum isspace`.
- Formatting: f-strings `f'{x:.2f}'`, `format`, `%`. Padding: `zfill`, `center`, `ljust`, width specifiers `{x:>8}`.

Verified output

```
--- strings ---
immutable: 'str' object does not support item assignment
'Data Science' 'Data Science  ' 'hi' <- strip('x') removes a SET of characters, not a prefix
cabc <- strips any a/b from both ends
ecneicS | Science | gramming | 16 -1
index raises: substring not found
['Programming', 'for', 'Data', 'Science'] | ['Programming', 'for Data Science'] | ['Programming for Data',
    'Science'] | ('Programming', ' for ', 'Data Science')
a-b-c | ['a', 'b', '', 'c'] | ['a', 'b'] | ['', 'a', '', 'b', '']
ProgrAmming for DAta Science | 3 | True | 005 | **ab**
   Rohan|8.46|     8.5|00042|1,234,567|25.6%
Rohan scored 8.5 | Rohan scored 8.5
True | 'a' < 'B': False | ord/chr: 65 b
isdigit/isalpha/isalnum: True False True True
== vs is: True False (use == for values)
sorted words by length: ['a', 'fox', 'quick', 'brown']
```

1.10-Q1 What does `'xxhixx'.strip('x')` return? What does `'abcabc'.strip('ba')` return? [tricky]

`'hi'` and `'cabc'`. The argument is a *set of characters* to remove from both ends, not a prefix string.

1.10-Q2 Difference between `find` and `index`; `split()` and `split(' ')`. [tricky]

`find` returns -1 when absent, `index` raises `ValueError`. `' a b '.split()` splits on any run of whitespace and ignores the ends: `['a','b']`; `' a b '.split(' ')` keeps empty strings: `['', 'a', '', 'b', '']`.

1.10-Q3 Why does `s[0] = 'x'` fail, and how do you change a character? [2 marks]

Strings are immutable (`TypeError`). Build a new string: `s = 'x' + s[1:]` or `s.replace(...)`.

1.10-Q4 Format `8.4567` to two decimals right-aligned in width 8, and a percentage. [2 marks]

```
f'{gpa:8.2f}'      # '    8.46'
f'{0.256:.1%}'     # '25.6%'
f'{1234567:,}'     # '1,234,567'
```

1.10-Q5 Validate a roll number of the form 2 digits + 3 letters + 4 digits using string methods. [3 marks]

```
ok = len(r) == 9 and r[:2].isdigit() and r[2:5].isalpha() and r[5:].isdigit()
branch = r[2:5] if ok else None
```

1.11 Exception handling

- `try` (risky code) / `except Type as e` (handle) / `else` (no exception happened) / `finally` (always runs). Raise with `raise`; chain with `raise New from old`.
- Handlers are checked top to bottom: put **specific** exceptions before general ones. A bare `except:` also catches `KeyboardInterrupt`; prefer `except Exception`.
- Hierarchy: `BaseException` > `Exception` > `ArithmeticError` (`ZeroDivisionError`), `LookupError` (`IndexError`, `KeyError`), `ValueError`, `TypeError`, `OSError` (`FileNotFoundError`).
- Custom exception: `class AppError(Exception): ...`. Context managers (`with`) release resources even on errors.

Verified output

```
--- exceptions: order, else, finally ---
   finally for '12'
   finally for 'a'
   finally for None
ok 12 | ValueError | TypeError/KeyError
   finally runs before the value is returned
from try
finally can override return: from finally

--- hierarchy: specific before general ---
ZeroDivisionError is an ArithmeticError: True
IndexError is a LookupError (IndexError and KeyError share this parent)
KeyError is a LookupError (IndexError and KeyError share this parent)
other: ValueError

--- custom exception, chaining, assert, with ---
custom: balance 100 < 250 | attrs: 100 250
chained: config invalid | cause: ValueError("invalid literal for int() with base 10: 'x'")
AssertionError: math broke
file closed after with: True
KeyboardInterrupt is NOT an Exception: True
```

1.11-Q1 What is printed by `parse(None)` for the function in the output and in what order do the messages appear? [predict the output]

`TypeError/KeyError`, preceded by the `finally` message. The `finally` block runs for every call, even when `except` returns.

1.11-Q2 What does `g()` return: `try: return 'from try' finally: return 'from finally'`? [tricky]

`'from finally'`. A `return` in `finally` overrides the earlier return (and can even swallow an exception); avoid it.

1.11-Q3 Explain the roles of `else` and `finally`. [3 marks]

`else` runs only if the `try` block finished without raising; it keeps success code out of the guarded block. `finally` always runs (after return, break or an unhandled exception) and is for clean-up.

1.11-Q4 Why is the order `except Exception` then `except ValueError` a mistake? [2 marks]

`Exception` catches everything first, so the more specific handler is unreachable. Order handlers from most specific to most general.

1.11-Q5 How do you define and raise a custom exception with data, and chain an error? [3 marks]

Subclass `Exception`, pass a message to `super().__init__`, store extra attributes (`balance`, `amount` in the output). `raise RuntimeError('config invalid') from e` keeps the original error as `__cause__`.

1.12 Object-oriented concepts

- **Class** = blueprint; **object** = instance. `__init__(self, ...)` initialises an instance; `self` is the instance.
- **Encapsulation**: `name` public, `_name` protected by convention, `__name` name-mangled to `_Class__name` (discourages access, not real privacy); `@property` for controlled access.
- **Inheritance**: `class B(A)`, `super().__init__()`. Python allows multiple inheritance; the method resolution order (MRO) uses C3 linearization.
- **Polymorphism**: same method name, different behaviour (overriding, duck typing). **Abstraction**: `abc.ABC` with `@abstractmethod` cannot be instantiated.
- `@classmethod` gets the class (`cls`), `@staticmethod` gets nothing. Dunder methods: `__str__`, `__repr__`, `__eq__` (define `__hash__` too), `__lt__`, `__add__`, `__len__`.

Verified output

```
--- class attribute vs instance attribute ---
Bad  : ['x'] <- b2 sees b1's item
Good : []

--- encapsulation, name mangling, property ---
vars: ['_Account__balance', '_audit', 'owner']
acc._Account__balance = 100 | acc.balance = 100
acc.__balance -> 'Account' object has no attribute '__balance'
setter validation: negative balance

--- inheritance, super(), polymorphism, abstract class ---
abstract: Can't instantiate abstract class Shape with abstract method area
Circle area=3.14
Rect area=6.00
isinstance Student/Person: True True | issubclass: True

--- multiple inheritance and MRO (C3 linearization) ---
D().who() = D>B>C>A
MRO: ['D', 'B', 'C', 'A', 'object']
```

Verified output (continued)

```
--- classmethod, staticmethod, dunder methods ---
Temp(100.0) | 100.0 C | False | 3.0 C | True | sorted: [Temp(1.0), Temp(5.0)] | in set: 1
dataclass: Point(x=1, y=2) True

--- duck typing ---
['quack', 'beep'] (no common base class needed)
```

1.12-Q1 A class has `items = []` at class level and `add` appends to `self.items`. What goes wrong with two objects? [tricky]

`items` is a class attribute shared by every instance, so `b2.items` shows what `b1` added (output: `Bad : ['x']`). Create the list in `__init__` (`self.items = []`).

1.12-Q2 What is name mangling? Is `__balance` really private? [3 marks]

Inside `class Account`, `self.__balance` is stored as `_Account__balance`. Outside, `acc.__balance` raises `AttributeError`, but `acc._Account__balance` still works, so it prevents accidental clashes only.

1.12-Q3 For `class D(B, C)` where both extend `A`, in what order are `who()` methods called, and why? [tricky]

`D > B > C > A`. The MRO from C3 linearization is `[D, B, C, A, object]`, and each `super()` call moves one step along it. Compare Scala trait linearization, which orders mix-ins right to left.

1.12-Q4 What happens if you try `Shape()` where `Shape` has an `@abstractmethod`? [2 marks]

`TypeError: Can't instantiate abstract class Shape with abstract method area`. Subclasses must implement the method.

1.12-Q5 Difference between `@classmethod`, `@staticmethod` and an instance method. [3 marks]

	First parameter	Typical use
instance method	`self`	work on one object's data
`@classmethod`	`cls`	alternative constructors (`Temp.from_f`)
`@staticmethod`	none	utility related to the class (`Temp.is_valid`)

1.12-Q6 Why define `__hash__` when you define `__eq__`? [tricky]

Defining `__eq__` alone makes instances unhashable. Objects that compare equal must have equal hashes if they are to be used in sets or as dict keys (output: `in set: 1`).

1.13 APIs, REST and HTTP requests

- An **API** is a contract for programs to talk to each other. A **REST API** exposes *resources* at URLs and uses HTTP methods; it is stateless (each request carries everything needed), usually returns JSON.
- URL anatomy: `https://host/items/3?category=tech&page=2` = scheme, host, **path** (identifies the resource), **query string** (filters). Requests also have headers (`Authorization`, `Content-Type`) and an optional body.

- **Safe** methods do not change data (GET). **Idempotent** methods give the same result when repeated (GET, PUT, DELETE). POST is neither.

Method	Meaning	Typical success code
GET	read a resource or list	200
POST	create a resource	201 Created + `Location`
PUT	replace a resource	200 / 204
PATCH	change part of a resource	200
DELETE	remove	204 No Content

Class	Meaning	Examples
2xx	success	200 OK, 201 Created, 204 No Content
3xx	redirection	301, 302, 304
4xx	client error (fix the request)	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	server error (maybe retry)	500, 502, 503, 504

Verified output (real requests to a local server)

```
--- GET with query parameters ---
url sent  : http://127.0.0.1:PORT/items?category=tech&q=a+b%26c (params are URL-encoded)
status    : 200 OK | Content-Type: application/json
r.json()  : {'count': 1, 'items': [{'id': 1, 'name': 'Laptop', 'category': 'tech'}]}
r.text type: str | r.json() type: dict

--- status codes ---
GET    /items/1   -> 200 OK
GET    /items/99  -> 404 Not Found
GET    /nowhere   -> 404 Not Found
POST   /items     -> 201 Created Location=/items/3
PUT    /items/2   -> 200 OK
DELETE /items/1   -> 204 No Content
POST with a bad body        -> 400 {'error': 'invalid JSON'}

--- errors: raise_for_status, auth, rate limit, timeout, bad JSON ---
raise_for_status -> HTTPError | code: 404
no token -> 401 | with Bearer token -> 200
/limited try 1: 429 Retry-After=1
/limited try 2: 429 Retry-After=1
/limited try 3: 200
timeout=0.5 on a 1.5 s endpoint -> ReadTimeout
invalid JSON body -> ValueError subclass: JSONDecodeError
nothing listening ->  ConnectTimeout
Session reuses the connection and default headers: {'data': 'private'}
```

1.13-Q1 Differentiate GET and POST; PUT and POST. [3 marks]

GET reads and puts parameters in the URL; it is safe and idempotent. POST creates and sends a body; repeating it can create duplicates (the output shows `POST /items -> 201 Created Location=/items/3`). PUT replaces a known resource at a known URL and is idempotent: sending the same PUT twice leaves the same state.

1.13-Q2 Why does the printed URL contain `q=a+b%26c`? [tricky]

`requests` URL-encodes `params`: a space becomes `+` (or `%20`) and `&` becomes `%26` so that it cannot be mistaken for the separator between parameters.

1.13-Q3 A call returns 429 with `Retry-After: 1`. What should the client do? What about a 404, a 401, a 500? [3 marks]

429: wait the given time and retry. 404: the resource does not exist, do not retry. 401: authentication missing or wrong (add the `Authorization: Bearer ...` header). 500/503: server problem, retry a limited number of times with back-off. The output shows the local server rejecting two calls with 429, then accepting the third.

1.13-Q4 Which exceptions can `requests.get` raise, and what does `raise_for_status()` do? [3 marks]

`Timeout` (`ConnectTimeout`, `ReadTimeout`), `ConnectionError`, `HTTPError` (only after `raise_for_status()`), `JSONDecodeError`/`ValueError` from `.json()`. `raise_for_status()` turns a 4xx/5xx response into `HTTPError`; without it a 404 is just a response with `status_code == 404`. Always pass `timeout=`: without it a hung server blocks forever.

1.13-Q5 What is the difference between `r.text` and `r.json()`? [2 marks]

`r.text` is the body as a string; `r.json()` parses it into dict/list/numbers and raises `ValueError` if the body is not valid JSON (the `/bad-json` test above).

1.13-Q6 What are `requests.Session`, query parameters versus path parameters, and an API key? [3 marks]

A `Session` reuses the TCP connection and default headers across calls (faster, and carries the token). A path parameter identifies a resource (`/items/3`); query parameters filter or page (`?category=tech`). An API key or bearer token in a header identifies the caller; never hard-code it in source code (use an environment variable).

1.14 File handling (text, CSV, JSON)

- `open(path, mode, encoding='utf-8')`. Modes: `r` read (error if missing), `w` write (**truncates**), `a` append, `x` create (error if exists), `r+` read and write, `b` binary.
- Prefer `with open(...) as f:` so the file closes automatically, even after an error. Read big files by iterating (`for line in f`).
- `csv.DictReader/DictWriter` handle quoting (open with `newline=''`); `json.dump/load` use files, `json.dumps/loads` use strings.
- JSON limits: tuples become lists, dict keys become strings, single quotes are invalid.

Verified output

```
--- file modes ---
'w' truncates: size after reopening with 'w' = 0
'x' on an existing file -> FileExistsError
'r' on a missing file -> FileNotFoundError

--- reading styles ---
read(): 'line1\nline2\nline3\n'
readline(): 'line1\n' | readlines(): ['line2\n', 'line3\n']
iterate: ['line1', 'line2', 'line3'] (memory friendly for big files)
seek(6) then read(5):
line | tell(): 11
'r+' overwrote in place: LINE1

--- csv and json ---
reg,name,marks
VL1,Asha,86
VL2,"Kabir, S",58
csv returns strings: {'reg': 'VL2', 'name': 'Kabir, S', 'marks': '58'} | comma inside a field is quoted
    automatically
dumps: {"id": 1, "tags": ["a", "b"], "2": "int key", "ok": true, "none": null}
loads: {'id': 1, 'tags': ['a', 'b'], '2': 'int key', 'ok': True, 'none': None} <- tuple became list, int key became
    str, True->true->True
dump/load (file) vs dumps/loads (string): {'a': [1, 2]}
single quotes are not JSON: Expecting property name enclosed in double quotes
```

1.14-Q1 What does opening an existing file with mode `'w'` do? [tricky]

It truncates it to zero length immediately, before you write anything (output: size 0). Use `'a'` to keep the content.

1.14-Q2 Compare `read()`, `readline()`, `readlines()` and iterating over the file. [3 marks]

`read()` returns the whole text; `readline()` one line (with `\n`); `readlines()` a list of all lines; iterating yields one line at a time and is memory friendly for large files.

1.14-Q3 Why can `f.seek(6)` land in the wrong place on Windows in text mode? [tricky]

Windows writes `\n` as `\r\n`, so byte offsets differ from character counts; in the output `seek(6); read(5)` returned `'\nline'` instead of `'line2'`. In text mode only seek to 0 or to a value returned by `tell()`.

1.14-Q4 What changes when a Python object goes through `json.dumps` and `json.loads`? [tricky]

`{'id': 1, 'tags': ('a','b'), 2: 'int key', 'ok': True, 'none': None}` comes back as `{'id': 1, 'tags': ['a','b'], '2': 'int key', 'ok': True, 'none': None}`: tuple to list, int key to string; `True`/`None` map to `true`/`null` in the text.

1.14-Q5 Why does a CSV field `Kabir, S` not break the file? [2 marks]

`csv.writer` wraps fields that contain the delimiter in quotes (`"Kabir, S"`), and `csv.reader` removes them again. Splitting on commas yourself would break it.

1.15 NLTK (natural language processing)

- Pipeline: raw text > sentence/word tokenization > lower-casing > remove punctuation and stopwords > stem or lemmatize > features (counts, TF-IDF) > model.
- **Stemming** chops suffixes by rules (`PorterStemmer`, `LancasterStemmer`; output may not be a word). **Lemmatization** maps to a dictionary form (`WordNetLemmatizer`) and needs the part of speech to be accurate.
- POS tagging: `pos_tag(tokens)` gives Penn Treebank tags (NN noun, VB verb, JJ adjective, RB adverb). `FreqDist` counts tokens; `bigrams`/`ngrams` make n-grams.
- Vectorisation (scikit-learn): bag of words counts; TF-IDF = term frequency x inverse document frequency, so words found in few documents weigh more.
- Data packages needed once: `punkt_tab`, `stopwords`, `wordnet`, `averaged_perceptron_tagger_eng` (`nltk.download`).

Verified output

```
--- tokenization ---
sentences: ['Students are studying data science.', "Dr. Rao's class isn't easy, but the geese in the garden were
    running faster!"]
words    : ['Students', 'are', 'studying', 'data', 'science', '.', 'Dr.', 'Rao', "'s", 'class', 'is', "n't", 'easy',
     ',', 'but', 'the', 'geese', 'in', 'the', 'garden', 'were', 'running', 'faster', '!']
note: "isn't" -> 'is', "n't" and "Rao's" -> 'Rao', "'s"; punctuation becomes tokens

--- cleaning: lower-case, keep alphabetic, remove stopwords ---
English stopword list size: 198 | 'not' in list: True (removing it can flip sentiment)
['students', 'studying', 'data', 'science', 'rao', 'class', 'easy', 'geese', 'garden', 'running', 'faster']

--- stemming vs lemmatization ---
word      Porter    Lancaster  Lemma (default n)  Lemma (with POS)
studies   studi     study      study              -
studying  studi     study      studying           study
geese     gees      gees       goose              -
running   run       run        running            run
better    better    bet        better             good
faster    faster    fast       faster             fast
was       wa        was        wa                 be

--- POS tagging ---
[('The', 'DT'), ('students', 'NNS'), ('are', 'VBP'), ('studying', 'VBG'), ('data', 'NNS'), ('science', 'NN'),
    ('quickly', 'RB')]
POS-aware lemmas: ['the', 'student', 'be', 'study', 'data', 'science', 'quickly']
```

Verified output (continued)

```
--- frequency, bigrams ---
FreqDist: [('data', 4), ('needs', 3), ('science', 2)]
bigrams : [('data', 'science'), ('science', 'needs'), ('needs', 'data'), ('data', 'and')] ...

--- bag of words and TF-IDF ---
vocabulary: ['data', 'fun', 'games', 'is', 'needs', 'science']
BoW counts:
 [[1 1 0 1 0 1]
 [1 0 0 0 1 1]
 [0 3 1 0 0 0]]
TF-IDF (rounded):
 [[0.46 0.46 0.   0.6  0.   0.46]
 [0.52 0.   0.   0.   0.68 0.52]
 [0.   0.92 0.4  0.   0.   0.  ]]
idf per word: {'data': 1.29, 'fun': 1.29, 'games': 1.69, 'is': 1.69, 'needs': 1.69, 'science': 1.29}
words found in only one document (games, is, needs) get the highest idf; words found in two documents get less.
```

1.15-Q1 Stemming versus lemmatization: compare the outputs for `studies, studying, geese, running, better, was`. [tricky]

word	Porter stem	default lemma	lemma with POS
studies	studi	study	-
studying	studi	studying	study (v)
geese	gees	goose	-
running	run	running	run (v)
better	better	better	good (a)
was	wa	wa	be (v)

Stemming is faster but can produce non-words (`studi`, `wa`). Lemmatization returns real words but treats every word as a noun unless you pass the POS: `lemmatize('better', 'a')` gives `good`.

1.15-Q2 Why can removing stopwords hurt sentiment analysis? [tricky]

The NLTK English list contains negations such as `not` (checked above: True), so 'not good' becomes 'good'. Keep negations or use a custom list.

1.15-Q3 How does `word_tokenize` treat "isn't" and "Rao's"? [2 marks]

As `is`, `n't` and `Rao`, `'s` (Treebank tokenizer), and punctuation becomes separate tokens. Filter with `token.isalpha()` if you only want words.

1.15-Q4 Explain TF-IDF with the three-document example. [3 marks]

TF-IDF weights a word by how often it occurs in a document and how rare it is across documents. In the output the words found in only one document (`games`, `is`, `needs`) have the highest idf (1.69) and words in two documents have 1.29, so a common word contributes less to a document's vector.

1.15-Q5 List the steps to clean headlines for keyword counting. [3 marks]

Lower-case, `word_tokenize`, keep alphabetic tokens, drop stopwords and very short words, lemmatize (with POS if possible), count with `Counter`.

1.16 ChatterBot

- ChatterBot is a **retrieval** chatbot library: it finds the stored statement most similar to the input and returns a stored reply. It does not generate new sentences (version 1.2 adds experimental LLM adapters).
- **Training**: `ListTrainer` (list of alternating statements and replies), `ChatterBotCorpusTrainer` (ready-made corpora), `CsvFileTrainer`, `JsonFileTrainer`, `UbuntuCorpusTrainer`.
- **Logic adapters** choose the reply: `BestMatch` (default), `TimeLogicAdapter`, `MathematicalEvaluation`, `SpecificResponseAdapter`, `UnitConversion`. **Storage adapters**: `SQLStorageAdapter` (SQLite by default), MongoDB, Redis, Django.
- Similarity is computed by a comparison class; in the installed 1.2.15 the default is `LevenshteinDistance` (edit distance). `BestMatch` stops searching at `maximum_similarity_threshold` (default 0.95).
- Every response carries a `confidence` between 0 and 1. `read_only=True` stops the bot from learning from live conversations. 1.2 needs the spaCy model `en_core_web_sm`.

Verified output (ChatterBot 1.2.15)

```
--- retrieval, not generation: the reply is a stored answer to the closest stored question ---
'Hi'                        -> 'Hello! How can I help?'                                      confidence=1.00
'what are the mess timing'  -> 'Breakfast 7:30-9:00, lunch 12:30-2:00, dinner 7:30-9:00.'    confidence=0.96
'who is the warden'         -> 'The warden office is in Block A.'                            confidence=0.97
'tell me a joke'            -> 'Sorry, I do not know that.'                                  confidence=0.00
'asdf qwerty'               -> 'Sorry, I do not know that.'                                  confidence=0.00

--- the similarity measure used to find the closest known statement ---
'what are the mess timings' vs 'what are the mess timing': 0.98
'what are the mess timings' vs 'who is the warden': 0.48
storage: SQLite file holds statements; trainers: ListTrainer here, ChatterBotCorpusTrainer for ready-made corpora
```

1.16-Q1 How does ChatterBot decide what to reply? [5 marks]

It preprocesses the input, searches its stored statements for the closest match (Levenshtein similarity by default), picks a stored response to that statement (first one by default), and returns it with a confidence score. In the output 'what are the mess timing' (a typo) still matches the trained question with confidence 0.96.

1.16-Q2 Why did 'tell me a joke' get the default reply? [2 marks]

Nothing in the training data was similar enough, so the `BestMatch` adapter returned its `default_response` with confidence 0.00. For production, use the confidence value to fall back to a menu or a human.

1.16-Q3 What are the limitations of ChatterBot for a complaint system? [3 marks]

It only repeats what it was trained on, has no memory of intent, and can answer confidently with a wrong stored reply. Complaint type and urgency are better handled by explicit rules or a classifier (NLTK keywords), with ChatterBot only for FAQs and small talk, which is how the hostel chatbot in the question bank is built.

Module 2: Python libraries for data science

Syllabus (9 hours): Pandas: Series, DataFrame, missing values, built-in functions, data operations, filtering, extraction, text data, merging; data mining: Scrapy, Beautiful Soup; data processing and modelling: NumPy, SciPy, Pandas, Keras, scikit-learn, PyTorch, TensorFlow, XGBoost; data visualisation: Matplotlib, Seaborn, Bokeh, Plotly, Folium.

Where each syllabus topic is covered

Syllabus topic	Section
Pandas: Series, DataFrame	2.1
Handling missing values	2.3
Built-in functions, data operations	2.4
Filtering data in DataFrame, data extraction	2.2
Working with text data	2.5
Merging DataFrames	2.6
Data mining: Scrapy, Beautiful Soup	2.7
NumPy	2.8
SciPy	2.9
scikit-learn (workflow, metrics, leakage); PCA; K-means	2.10, 2.11, 2.12
Keras, PyTorch, TensorFlow	2.13
XGBoost	2.14
Matplotlib, Seaborn, Bokeh, Plotly, Folium	2.15

2.1 Pandas: Series and DataFrame

- **Series**: 1-D labelled array (values + index, one dtype). **DataFrame**: 2-D table of aligned Series (columns may have different dtypes). `df['a']` is a Series, `df[['a']]` is a DataFrame.
- Attributes: `shape`, `dtypes`, `columns`, `index`, `values`/`to_numpy()`, `info()`, `head()`. Creation: from dict, list of dicts, `read_csv`, NumPy array.
- **Arithmetic aligns on index labels**, not positions: labels present in only one operand give NaN.

Verified output

```
--- Series: labelled array, alignment on the index ---
s1 + s2 aligns by label (not position):
a     NaN
b    21.0
c    32.0
d     NaN
dtype: float64
s1.add(s2, fill_value=0): [10.0, 21.0, 32.0, 3.0]
dtype: int64 | s1['b']: 20 | s1.iloc[1]: 20 | s1[s1 > 15].index.tolist(): ['b', 'c']

--- DataFrame basics ---
    name  age     city  score
0   Asha   22  Chennai   88.5
1   Ravi   31    Delhi   72.0
2  Meena   27     Pune    NaN
3  Kabir   35    Delhi   65.5
4  Sneha   24  Chennai   91.0
shape (5, 4) | dtypes: {'name': 'object', 'age': 'int64', 'city': 'object', 'score': 'float64'}
describe (numeric):
         age  score
count   5.00   4.00
mean   27.80  79.25
std     5.26  12.45
min    22.00  65.50
25%    24.00  70.38
50%    27.00  80.25
75%    31.00  89.12
max    35.00  91.00
value_counts: {'Chennai': 2, 'Delhi': 2, 'Pune': 1} | nunique: 3
```

2.1-Q1 What is the result of `pd.Series([10,20,30], index=list('abc')) + pd.Series([1,2,3], index=list('bcd'))`? [tricky]

`a NaN, b 21, c 32, d NaN`. Addition aligns on labels; `a` and `d` exist in only one Series so they become NaN, and the dtype becomes float. `s1.add(s2, fill_value=0)` treats a missing label as 0.

2.1-Q2 Difference between a Series and a DataFrame; how do you get each from a DataFrame column? [3 marks]

A Series is one labelled column of values with one dtype; a DataFrame is a table of several. `df['age']` returns a Series; `df[['age']]` (list inside the brackets) returns a one-column DataFrame.

2.1-Q3 Why does an integer column turn into floats when a value is missing? [2 marks]

NumPy integers cannot hold NaN, so pandas upcasts to `float64`. The nullable dtype `Int64` keeps integers and shows `<NA>`.

2.2 Selecting and filtering data

Tool	Selects by	End of slice	Example
`df['col']`, `df[['a','b']]`	column name	-	`df[['name','age']]`
`df.loc[rows, cols]`	**labels** or boolean mask	**included**	`df.loc[1:2, 'name']`
`df.iloc[rows, cols]`	**integer positions**	**excluded**	`df.iloc[1:2, 0]`
boolean mask	condition on values	-	`df[(df.age > 25) & (df.city == 'Delhi')]`
`isin`, `between`, `query`	membership, range, text expression	-	`df.query('age > 30 and city == "Delhi"')`

- Combine masks with `&`, `|`, `~` and **parentheses** around each comparison (`and`/`or` raise `ValueError: truth value of a Series is ambiguous`).
- To change values use one `df.loc[mask, 'col'] = value`. Chained indexing `df[mask]['col'] = value` may write to a temporary copy.

Verified output

```
--- selecting: [], loc, iloc, boolean masks ---
df['age'] is a Series | df[['age']] is a DataFrame
loc[1:2] rows: ['Ravi', 'Meena'] | iloc[1:2] rows: ['Ravi']
age>25 and Delhi: ['Ravi', 'Kabir']
using 'and' instead of '&' -> The truth value of a Series is ambiguous. Use a.empty, a.boo ...
isin: ['Ravi', 'Meena', 'Kabir'] | between: ['Ravi', 'Meena', 'Sneha'] | query: ['Ravi', 'Kabir']

--- chained assignment trap ---
chained d2[mask]['score'] = 0 changed the data? False | warning: SettingWithCopyWarning
d2.loc[mask, 'score'] = 0 changed the data? True
```

2.2-Q1 For the default index, how many rows do `df.loc[1:3]` and `df.iloc[1:3]` return? [tricky]

3 and 2. `loc` includes the end label; `iloc` excludes the end position (the output shows `loc[1:2]` gives two names, `iloc[1:2]` one).

2.2-Q2 Why does `df[df.age > 25 and df.city == 'Delhi']` fail, and what is the fix? [tricky]

`and` needs a single truth value, but a Series has many (`ValueError: The truth value of a Series is ambiguous`). Use `&` with parentheses: `df[(df.age > 25) & (df.city == 'Delhi')]`.

2.2-Q3 `d2[d2.age > 30]['score'] = 0` leaves the data unchanged and warns. Why? How do you do it correctly? [tricky]

`d2[mask]` creates a new DataFrame (a copy); the second `['score'] = 0` modifies that temporary object, so the original is untouched (`SettingWithCopyWarning`). Use one call on the original: `d2.loc[d2.age > 30, 'score'] = 0` (output: changed? True).

2.2-Q4 Write the pandas code to select Name and Age, filter Age > 25, add a Country column and sort by Age descending. [3 marks]

Lab 1; run in the question bank, M2-C1

```
df[['Name', 'Age']]
df[df['Age'] > 25]
df['Country'] = 'India'
df.sort_values('Age', ascending=False)
```

2.3 Handling missing values

- Missing numbers are `NaN` (float) or `pd.NA`/`None`. `NaN == NaN` is False; test with `isna()`/`notna()`. Aggregations such as `mean`, `sum` **skip** NaN by default.
- Options: inspect `df.isna().sum()`; drop `dropna(subset=[...], thresh=k)`; fill `fillna(value | mean | median)`, `ffill()`, `bfill()`, `interpolate()`; flag with an indicator column when missingness is informative.
- Choose by column role: key fields missing -> drop the row; numeric -> mean or median (median if skewed); categorical -> mode or 'Unknown'; time series -> forward fill or interpolate.
- Do not fill using statistics from the test set (compute the mean on training data only).

Verified output

```
--- missing values ---
NaN == NaN: False | isna count: 1 | mean skips NaN: 79.25 | sum of [1, NaN]: 1.0
fillna(mean): [88.5, 72.0, 79.25, 65.5, 91.0]
ffill: [88.5, 72.0, 72.0, 65.5, 91.0] | interpolate: [88.5, 72.0, 68.75, 65.5, 91.0]
dropna(subset=['score']) keeps 4 of 5 rows | dropna(thresh=4) keeps 4
int column with NaN becomes float: float64 | nullable Int64: [1, 2, <NA>]
df.replace / where: [0, 31, 27, 35, 0]
```

2.3-Q1 Why does `np.nan == np.nan` return False and how do you test for missing values? [2 marks]

By IEEE-754 rules NaN is not equal to anything, including itself. Use `pd.isna(x)`, `df.isna()`, `np.isnan(arr)`.

2.3-Q2 Explain `dropna(subset=['score'])` and `dropna(thresh=4)`. [2 marks]

`subset` drops rows that are missing a value in the listed columns only. `thresh=4` keeps rows having at least 4 non-missing values. In the sample, both keep 4 of 5 rows.

2.3-Q3 Fill the missing Age with the mean, drop rows missing Name or City, convert Age to int, min-max scale it. In what order and why? [5 marks]

Mean fill first (`df['Age'].fillna(df['Age'].mean())`), then `dropna(subset=['Name','City'])`, then `astype(int)` (impossible while NaN remains), then `(a - a.min()) / (a.max() - a.min())`. State the order: the mean is computed before rows are dropped, which changes the result if you swap the steps (question bank, M2-C1).

2.3-Q4 Mean, median or forward fill: which for `[88.5, 72, NaN, 65.5, 91]`? [3 marks]

Mean fill gives 79.25; `ffill` repeats 72.0 and suits ordered data such as a time series; `interpolate` gives 68.75, midway between the neighbours. Prefer the median when outliers skew the mean.

2.4 Built-in functions and data operations

- Summaries: `describe()`, `value_counts()`, `nunique()`, `mean/median/std/min/max`, `corr()`. Ordering: `sort_values(by, ascending, na_position)`, `sort_index`.
- Changing: `assign`, `rename(columns=)`, `drop(columns=)`, `astype`, `pd.to_numeric(errors='coerce')`, `pd.to_datetime`, `pd.cut` (bins), `map` (dict/function per value), `apply` (function per row/column; `axis=1` is per row).
- **axis**: `axis=0` operates down the rows (one result per column); `axis=1` across the columns (one result per row).
- Prefer vectorised operations (`df['a'] * 2`) over `apply` or loops; they run in compiled code.

Verified output

```
--- add / drop / rename / sort / apply ---
 name  score grade
Sneha   91.0     A
 Asha   88.5     A
 Ravi   72.0     B
Kabir   65.5     C
Meena    NaN   NaN
rename: ['student', 'age', 'city', 'score'] | drop: ['name', 'age', 'score']
map: ['TN', 'DL', nan, 'DL', 'TN'] (unmapped -> NaN)
apply row-wise: ['Asha:22', 'Ravi:31'] | axis=0 means down the columns
astype: float64 | to_numeric(errors='coerce'): [1.0, nan, 3.0]
datetime: Monday | [1, 2]
```

2.4-Q1 What does `df['city'].map({'Chennai': 'TN', 'Delhi': 'DL'})` give for an unmapped value? [2 marks]

`NaN` (the output shows `nan` for Pune). Use `.fillna('Other')` or `df.replace`.

2.4-Q2 What does `apply(..., axis=1)` mean and when should you avoid `apply`? [3 marks]

It calls the function once per row (axis=0: once per column). It is a Python-level loop, so use vectorised column arithmetic, `np.where` or `.str` methods when possible.

2.4-Q3 `pd.to_numeric(pd.Series(['1', 'x', '3']), errors='coerce')`? [2 marks]

`[1.0, NaN, 3.0]`. Invalid entries become NaN instead of raising; then handle the NaN.

2.4-Q4 What does `pd.cut(score, [0, 70, 85, 100], labels=['C','B','A'])` do with a mark of exactly 70? [tricky]

Bins are right-closed by default, `(0, 70]`, so 70 falls in `C`. Use `right=False` for left-closed bins. Missing values stay NaN (the output shows Meena with NaN).

2.5 Working with text data

- The `.str` accessor applies string methods to a whole column: `str.strip/lower/title/replace/contains/startswith/split/extract/len`. Missing values pass through as NaN/None instead of raising.
- `contains` treats the pattern as a regular expression unless `regex=False`; pass `na=False` when using the result as a filter mask.
- `str.extract(r'(\d+)')` pulls capture groups into new columns; `str.split(expand=True)` returns a DataFrame.

Verified output

```
--- text data (.str) ---
strip/title: ['Asha Nair', 'Ravi.Kumar@Vit.Ac.In', None, 'Meena Rao']
contains (NaN stays NaN): [False, True, None, False]
na=False: [False, True, False, False]
extract domain: [nan, 'vit.ac.in', None, nan]
split + expand: (3, 2) | len: [12.0, 20.0, nan, 9.0]
replace regex: [' asha NAIR '] | lower+startswith: [False, True, None, False]
```

2.5-Q1 Why does filtering with `df[df.name.str.contains('a')]` sometimes raise an error? [tricky]

If the column has missing values, `contains` returns NaN for them and a mask with NaN cannot index (`ValueError`). Use `na=False` (output: `[False, True, False, False]`).

2.5-Q2 Clean `' asha NAIR '` to `'Asha Nair'` and validate an email ending in `.ac.in`. [2 marks]

```
s.str.strip().str.title()
s.str.endswith('.ac.in') & s.str.contains('@')
```

2.6 Merging, joining, concatenating and grouping

how=	Keeps	Row counts in the sample (3 customers, 4 orders)
inner	keys in both	3
left	all left rows	4
right	all right rows	4
outer	union of keys	5

- `pd.merge(a, b, on='key', how=...)` is a database join; `a.join(b)` joins on the index; `pd.concat([a, b], axis=0|1)` stacks rows or columns without matching on keys.
- Duplicate keys on both sides multiply rows (many-to-many). Use `validate='one_to_one'`/`'one_to_many'` to catch it, `indicator=True` to see where rows came from, `suffixes=` for clashing column names, `left_on/right_on` for different key names.
- **groupby = split-apply-combine**: split rows by key, apply `sum/mean/count` (agg), `transform` (same length as input, e.g. share of total), or `apply` (any function), combine into a result.
- `reset_index()` turns group keys back into columns; `pivot_table(index, columns, values, aggfunc)` is a groupby arranged as a table.

Verified output

```
--- groupby: agg vs transform vs apply ---
agg   (one row per group): {'N': 160, 'S': 140, 'W': 20}
transform (same length as input): [140, 140, 160, 160, 160, 20]
share of region total: [0.71, 0.29, 0.44, 0.19, 0.38, 1.0]
region   cat  total  n   avg
     N Books     60  1  60.0
     N  Tech    100  2  50.0
     S Books     40  1  40.0
     S  Tech    100  1 100.0
     W Books     20  1  20.0
size() counts rows incl. NaN; count() counts non-NaN: {'N': 3, 'S': 2, 'W': 1}
pivot_table:
cat     Books  Tech
region
N          60   100
S          40   100
W          20     0
groupby result index is the group key unless reset_index(): region

--- merge: how, keys, traps ---
inner  -> 3 rows
left   -> 4 rows
right  -> 4 rows
outer  -> 5 rows
 cid  name  oid     _merge
   1  Asha 10.0       both
   1  Asha 11.0       both
   2  Ravi 12.0       both
   3 Meena  NaN  left_only
   9   NaN 13.0 right_only
many-to-many join: 2 x 2 matching keys -> 4 rows (row explosion)
validate='one_to_one' catches it: Merge keys are not unique in either left or right datas ...
same column name in both -> suffixes: ['id', 'v_x', 'v_y'] | custom: ['id', 'v_l', 'v_r']
different key names: left_on/right_on -> ['cid', 'name', 'oid', 'customer', 'amt']
key dtype mismatch (text '1' vs number 1):
   -> You are trying to merge on object and int64 columns for key 'cid'. If  ...

--- concat and join ---
concat axis=0 keeps duplicate index labels: [0, 1, 0, 1] | ignore_index: [0, 1, 2, 3]
concat axis=1 shape: (2, 2) | join on index: ['name', 'oid', 'amt']
```

2.6-Q1 Explain split-apply-combine and show `agg` with named results. [5 marks]

```
sales.groupby(['region', 'cat']).agg(total=('amount', 'sum'), n=('amount', 'count'), avg=('amount',
    'mean')).reset_index()
```

Each region/category group is processed independently and the pieces are combined into one table (see the output). `groupby` is the pandas form of SQL `GROUP BY`.

2.6-Q2 Difference between `agg` and `transform`. [tricky]

`agg` reduces each group to one row (3 regions give 3 rows); `transform` returns a value for every original row (6 rows), e.g. each sale's share of its region total `amount / g.transform('sum')`.

2.6-Q3 Two tables each have two rows with `k = 1`. How many rows does `merge(a, b, on='k')` return? How do you guard against this? [tricky]

4 (2 x 2 row explosion). `validate='one_to_one'` raises `MergeError: Merge keys are not unique...`. Check key uniqueness with `df['k'].is_unique` before merging.

2.6-Q4 Inner, left, right, outer: what are the row counts for customers {1,2,3} and orders for customers {1,1,2,9}? [3 marks]

Inner 3 (orders for 1, 1, 2), left 4 (adds customer 3 with NaN order), right 4 (adds the order of customer 9 with NaN name), outer 5. `indicator=True` marks rows `both`, `left_only`, `right_only`.

2.6-Q5 Why does merging on a text column '1' and a number column 1 fail? [2 marks]

pandas raises `ValueError: You are trying to merge on object and int64 columns`. Cast one side (`astype(str)` or `astype(int)`) first.

2.6-Q6 `merge`, `join`, `concat`: differences. [3 marks]

	merge	join	concat
matches on	column values	index (default)	index/position; no key logic
axis	columns joined	columns joined	rows (0) or columns (1)
duplicate index	n/a	n/a	kept unless `ignore_index=True`

2.7 Data mining: Beautiful Soup and Scrapy

- **Beautiful Soup** parses HTML/XML you already downloaded (`requests.get(url).text`): `soup.find_all('div', class_='book')`, `select_one('span.price')`, `.get_text(strip=True)`, `tag['href']`. Synchronous, simple, ideal for one page.
- **Scrapy** is a crawling framework: spiders (`parse` callbacks), Scrapy Engine, Scheduler, Downloader, Item Pipelines, Feed exports, middlewares. Built on Twisted so many requests are in flight at once; settings such as `ROBOTSTXT_OBEY`, `DOWNLOAD_DELAY`, `AUTOTHROTTLE_ENABLED`, `CONCURRENT_REQUESTS`.
- Selectors: CSS (`response.css('h2::text').get()`) and XPath (`response.xpath('//h2/text()')`). `.get()` returns the first match, `.getall()` all.
- Pages built by JavaScript are not in the downloaded HTML; use the site's API, or a browser-automation tool such as Selenium or Playwright.
- Responsible scraping: read `robots.txt` and the terms of service, throttle requests, identify your crawler, avoid personal data.

Beautiful Soup version (tested)

```
"""M2-Q4a  Scrape a page with Beautiful Soup and store CSV + JSON (Lab experiment 5).
The HTML is embedded so the script runs offline; with a live site you would
replace `html` by requests.get(url, timeout=10).text.
"""
import csv
import json

from bs4 import BeautifulSoup

html = """
<html><body>
  <div class="book"><h2>Python Basics</h2><span class="price">Rs. 450</span><a href="/b/1">details</a></div>
  <div class="book"><h2>Data Science 101</h2><span class="price">Rs. 700</span><a href="/b/2">details</a></div>
  <div class="book"><h2>Scala in Depth</h2><span class="price">Rs. 820</span><a href="/b/3">details</a></div>
</body></html>
"""
with open("books.html", "w", encoding="utf-8") as f:       # reused by the Scrapy example
    f.write(html)

soup = BeautifulSoup(html, "html.parser")
rows = []
for div in soup.find_all("div", class_="book"):
    rows.append({
        "title": div.h2.get_text(strip=True),
        "price": int(div.select_one("span.price").get_text().replace("Rs.", "").strip()),
        "link": div.a["href"],
    })

with open("books_bs4.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["title", "price", "link"])
    w.writeheader()
    w.writerows(rows)
with open("books_bs4.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, indent=2)

print("Parsed", len(rows), "books")
print(open("books_bs4.csv", encoding="utf-8").read())
print("Average price:", sum(r["price"] for r in rows) / len(rows))
```

Output

```
Parsed 3 books
title,price,link
Python Basics,450,/b/1
Data Science 101,700,/b/2
Scala in Depth,820,/b/3

Average price: 656.6666666666666
```

Scrapy version (tested against the local file)

```
"""M2-Q4b  The same extraction as a Scrapy spider (runs against the local books.html
through a file:// URL so no website is contacted).
"""
import json
from pathlib import Path

import scrapy
from scrapy.crawler import CrawlerProcess

page = Path("books.html")
if not page.exists():
    raise SystemExit("run m2_q4_scrape_bs4.py first - it writes books.html")


class BookSpider(scrapy.Spider):
    name = "books"
    start_urls = [page.resolve().as_uri()]                # file:///.../books.html
    custom_settings = {"LOG_LEVEL": "ERROR", "ROBOTSTXT_OBEY": True,
                       "DOWNLOAD_DELAY": 1,               # politeness delay (ignored for file:///)
                       "AUTOTHROTTLE_ENABLED": True,
                       "FEEDS": {"books_scrapy.json": {"format": "json", "overwrite": True}}}

    def parse(self, response):
        for div in response.css("div.book"):
            yield {                                       # each yielded dict becomes an item
                "title": div.css("h2::text").get(),
                "price": int(div.css("span.price::text").get().replace("Rs.", "").strip()),
                "link": div.css("a::attr(href)").get(),
            }


process = CrawlerProcess()
process.crawl(BookSpider)
process.start()                                           # blocks until the crawl finishes

items = json.loads(Path("books_scrapy.json").read_text(encoding="utf-8"))
print("Scrapy yielded", len(items), "items")
for it in items:
    print(" ", it)
```

Output

```
Scrapy yielded 3 items
  {'title': 'Python Basics', 'price': 450, 'link': '/b/1'}
  {'title': 'Data Science 101', 'price': 700, 'link': '/b/2'}
  {'title': 'Scala in Depth', 'price': 820, 'link': '/b/3'}
```

2.7-Q1 Compare Beautiful Soup and Scrapy. [5 marks]

	Beautiful Soup	Scrapy
Type	parsing library	full crawling framework
Execution	synchronous (with requests)	asynchronous, event-driven (Twisted)
Output/storage	write your own CSV/JSON code	Item Pipelines and Feed exports
Politeness	manual `time.sleep`	`DOWNLOAD_DELAY`, AutoThrottle, robots.txt
Use	small, one-off extraction	large repeatable crawls

2.7-Q2 Why is Scrapy faster than a loop of `requests.get` + Beautiful Soup? [3 marks]

Most time is waiting on the network. Scrapy's Twisted event loop issues a new request while earlier ones wait, so total time approaches the slowest request rather than the sum; the blocking loop does one at a time.

2.7-Q3 A page shows data in the browser but `soup.find_all` finds nothing. Why? [tricky]

The data is probably rendered by JavaScript after the page loads, so it is not in the HTML that `requests` downloaded. Look for a JSON API call in the browser's network tab, or use a headless browser.

2.7-Q4 What does a Scrapy item pipeline do? [2 marks]

It receives every item the spider yields and can clean, validate, drop duplicates, and store it (database, file). Pipelines run in the order given by their numbers in `ITEM_PIPELINES`.

2.8 NumPy

- `ndarray`: fixed dtype, contiguous memory, vectorised operations in compiled code. Attributes `shape`, `ndim`, `dtype`, `size`. Creation: `array`, `arange`, `linspace`, `zeros`, `ones`, `eye`, `random.default_rng(seed)`.

- **Broadcasting**: align shapes on the right (pad with 1s on the left); each pair of dimensions must be equal or 1; size-1 dimensions stretch. Otherwise `ValueError`.
- **Views and copies**: basic slices are **views** (share memory); fancy (integer list) and boolean indexing return **copies**; `.copy()` forces a copy.
- `*` is element-wise; `@` / `np.dot` is the matrix product. `sum/mean/max(axis=...)`: axis=0 collapses rows (per column), axis=1 collapses columns (per row).
- dtype traps: integer overflow wraps (`uint8` 250 + 10 = 4), `/` always gives float, `//` floors toward minus infinity, `np.array([1, 2.5, 'a'])` becomes strings, `mean` of an array with NaN is NaN (`nanmean`).

Verified output

```
--- shape, axis ---
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
sum axis=0 (down rows): [12 15 18 21] | axis=1 (across columns): [ 6 22 38] | keepdims: (3, 1)
argmax: 11 | argmax(axis=0): [2 2 2 2] | reshape(-1): [0 1 2 3 4] ... | T shape: (4, 3)

--- view vs copy ---
slice is a view, original changed: 99 | shares memory with a: True
fancy index is a copy, original unchanged: 99 | shares memory: False
boolean mask returns a copy: True | values: [99  6  7  8]
explicit copy is independent: 5

--- operators: * vs @ vs dot, broadcasting ---
x * y (element-wise): [[10, 40], [90, 160]] | x @ y (matrix product): [[70, 100], [150, 220]] | np.dot same: [[70,
    100], [150, 220]]
(2,2) + (2,) : [[101, 202], [103, 204]] | (2,2) + (2,1): [[101, 102], [203, 204]]
== is element-wise: [[False, False], [False, False]] | np.array_equal: True | allclose(0.1+0.2, 0.3): True | exact:
    False

--- dtype traps ---
uint8 250 + 10 wraps around -> 4 | int array / int true-divides -> 3.5 | // floors -> -4
np.array([1, 2.5, 'a']) dtype: <U32 | np.array([1, 2.5]) dtype: float64
NaN: np.nan == np.nan False | np.isnan: True | mean with nan: nan | nanmean: 1.0

--- where, sort, unique, random ---
where: ['big', 'small', 'big', 'small'] | argsort: [1, 3, 0, 2] | unique+counts: [[3, 5, 8], [2, 1, 1]]
same seed -> same numbers: [8, 6, 5] [8, 6, 5] | linspace: [0.0, 0.25, 0.5, 0.75, 1.0] | arange step: [0.0, 0.25,
    0.5, 0.75]
```

Broadcasting examples (question bank M2-C3)

```
a =
[[10  6  7  9]
 [ 6  8  9  2]
 [ 0  3  3  9]]
shape (3, 4) | mean 6.0 | std 3.136

first two rows, last two columns:
[[7 9]
 [9 2]]

reshape(4, 3):
[[10  6  7]
 [ 9  6  8]
 [ 9  2  0]
 [ 3  3  9]]

element-wise product with another (4,3) array:
[[40  6 14]
 [36  6 32]
 [ 9  4  0]
 [ 6  6 18]]

--- broadcasting ---
(3,4) + (4,)  -> (3, 4)
(3,4) + (3,1) -> (3, 4)
(3,1) + (1,4) -> (3, 4)
(3,4) + (3,)  -> ValueError: operands could not be broadcast together with shapes (3,4) (3,)

column-wise standardisation using broadcasting:
[[-1.225 -1.225]
 [ 0.     0.   ]
 [ 1.225  1.225]]
```

2.8-Q1 State the broadcasting rules and decide whether (3,4)+(4,), (3,4)+(3,1), (3,4)+(3,) work. [tricky]

Right-align the shapes; each dimension pair must be equal or one of them 1. (3,4)+(4,): works, result (3,4). (3,4)+(3,1): works (3,4). (3,4)+(3,): aligned as (3,4) vs (1,3), 4 vs 3 differ and neither is 1, so `ValueError: operands could not be broadcast together with shapes (3,4) (3,)`.

2.8-Q2 `v = a[0, :2]; v[0] = 99`. Does `a` change? What about `c = a[[0, 1]]; c[0, 0] = -1`? [tricky]

Yes for the slice (a view: `np.shares_memory` True). No for the integer-list index (a copy). Boolean masks also return copies. Use `.copy()` when you want independence.

2.8-Q3 Difference between `x * y`, `x @ y` and `np.dot(x, y)` for 2 x 2 arrays. [2 marks]

`x * y` multiplies element by element ([[10,40],[90,160]] in the output); `x @ y` and `np.dot` give the matrix product ([[70,100],[150,220]]).

2.8-Q4 Why does `np.array([250], dtype=np.uint8) + 10` equal 4? [tricky]

`uint8` holds 0..255; 260 wraps around modulo 256 without an error. Choose a wider dtype when values may exceed the range.

2.8-Q5 What do `axis=0` and `axis=1` mean in `a.sum(axis=...)` for a 3 x 4 array? [2 marks]

axis=0 collapses the rows and returns 4 column sums; axis=1 collapses the columns and returns 3 row sums (`keepdims=True` keeps a (3,1) shape for later broadcasting).

2.8-Q6 Why is NumPy faster than a Python list for numeric work? [3 marks]

Contiguous typed memory, no per-element object overhead, and loops run in C (vectorisation). Measured earlier: 2.5 ms against 96 ms for doubling one million integers.

2.8-Q7 How do you make random results repeatable? [1 mark]

Use a seeded generator: `rng = np.random.default_rng(0)`. The same seed gives the same numbers (output: `[8, 6, 5]` twice).

2.9 SciPy

- SciPy builds on NumPy with scientific algorithms: `scipy.stats` (distributions, t-tests, correlations), `optimize` (minimise, curve fit, root finding), `integrate` (quad, ODEs), `linalg`, `sparse` (matrices that mostly hold zeros), `interpolate`, `signal`, `spatial`.
- Hypothesis test pattern: state H0, run the test (`ttest_ind`), compare the p-value with the significance level (0.05): p < 0.05 rejects H0.

Verified output

```
--- linear algebra (basis for PCA) ---
eigenvalues:[1.382 3.618]| eigenvectors (columns):
[[-0.8507  0.5257]
 [ 0.5257  0.8507]]
A v = lambda v holds: True | det: 5.0 | inv @ A = I: True
solve Ax=b: [0.8 1.4] | rank: 2
SVD singular values: [3.618 1.382] (for a symmetric positive matrix these equal the eigenvalues)

--- SciPy ---
ttest_ind: t=3.91, p=0.0029 -> significant at 5%
pearson r: 0.775 | norm.cdf(1.96): 0.975
optimize.minimize minimum at: [ 3. -1.] | integrate.quad(x^2, 0, 3): 9.0 | linalg.solve: [0.8 1.4]
```

2.9-Q1 What does a p-value of 0.0029 mean in the t-test output? [3 marks]

If the two groups had the same mean, a difference at least this large would appear in about 0.29% of samples. Since 0.0029 < 0.05 we reject the null hypothesis of equal means. It is not the probability that H0 is true.

2.9-Q2 When do you use `scipy.sparse`? [2 marks]

For matrices with mostly zeros (text vectors, graphs): CSR/CSC formats store only non-zero entries, saving memory and time. scikit-learn's `CountVectorizer` returns such a matrix.

2.9-Q3 NumPy versus SciPy. [2 marks]

NumPy supplies arrays and basic maths; SciPy adds higher-level scientific routines (optimisation, statistics, integration, sparse algebra) built on those arrays.

2.10 Machine-learning workflow with scikit-learn

- Estimator API: `fit(X, y)` learns from data, `predict(X)` makes predictions, `transform(X)` converts data (scalers, PCA). `fit_transform` = fit then transform. **Learn parameters on the training data only**.
- Workflow: split (`train_test_split`, `stratify=y` keeps class ratios) > preprocess > fit > evaluate on unseen data > tune with cross-validation (`cross_val_score`).
- Types: supervised *classification* (labels), supervised *regression* (numbers), unsupervised *clustering* and *dimensionality reduction*.
- **Pipeline** chains steps so every fold re-fits them correctly: `Pipeline([('scale', StandardScaler()), ('clf', LogisticRegression())])`.
- **Overfitting**: very high training score, much lower test score (memorising noise). **Underfitting**: both scores low. Remedies: more data, simpler model, regularisation, early stopping, cross-validation.

Verified output

```
=== CLASSIFICATION METRICS by hand ===
TP=2 FN=2 FP=1 TN=5
accuracy = (TP+TN)/all = 0.70 | precision = TP/(TP+FP) = 0.667 | recall = TP/(TP+FN) = 0.500 | F1 = 2PR/(P+R) =
    0.571
sklearn agrees: 0.7 0.667 0.5 0.571
confusion_matrix rows=true, cols=predicted:
 [[5 1]
 [2 2]]
accuracy paradox: predicting 'negative' for everyone on a 95/5 dataset gives accuracy 0.95 and recall 0.0

=== DATA LEAKAGE: choosing features on ALL the data, then cross-validating ===
noise data, selector fitted outside CV : accuracy 0.78   (looks like skill, but there is none)
same data, selector inside a Pipeline  : accuracy 0.59   (honest: about a coin flip)
Pipeline test accuracy: 0.959 | 5-fold CV (scaler refitted inside each fold): [0.982 0.982 0.974 0.974 0.991] mean
    0.981
stratify keeps class ratio: train positives 0.628 | test positives 0.626 | overall 0.627

=== OVERFITTING: training vs test accuracy ===
max_depth=1     train acc=0.75  test acc=0.67  leaves=2
max_depth=3     train acc=0.83  test acc=0.70  leaves=8
max_depth=None  train acc=1.00  test acc=0.66  leaves=53
a gap between train and test accuracy is the symptom; limit depth, add data, or regularise

=== PCA inside a pipeline (fit on training data only) ===
5-fold CV accuracy with 2 PCs: [0.867 0.967 0.833 0.933 0.967] | with all 4 features: [0.967 1.    0.933 0.9   1.
    ]
```

Metric	Formula	Question it answers
Accuracy	(TP + TN) / all	how often is the model right?
Precision	TP / (TP + FP)	of predicted positives, how many are real?
Recall (sensitivity)	TP / (TP + FN)	of real positives, how many did we find?
F1	2PR / (P + R)	balance of precision and recall
MSE / RMSE / R-squared	mean (y - p)^2 / its root / 1 - SSE/SST	regression error and explained variance

2.10-Q1 From TP=2, FN=2, FP=1, TN=5 compute accuracy, precision, recall and F1. [5 marks]

Accuracy = (2+5)/10 = 0.70; precision = 2/(2+1) = 0.667; recall = 2/(2+2) = 0.50; F1 = 2(0.667)(0.5)/(0.667+0.5) = 0.571. scikit-learn agrees (output). The confusion matrix has true classes as rows and predicted classes as columns: [[5, 1], [2, 2]].

2.10-Q2 A model predicts 'no fraud' for everyone on a data set with 5% fraud. What are its accuracy and recall? [tricky]

Accuracy 0.95 but recall 0.0: it finds none of the fraud. This is the accuracy paradox; use precision, recall, F1 or ROC-AUC on imbalanced data.

2.10-Q3 What is data leakage? Give an example the output demonstrates. [tricky]

Information from the test data (or the labels) influences training. With pure noise (no real signal), selecting the best 20 of 2,000 features on *all* the data and then cross-validating gives accuracy 0.78, while doing the selection inside a `Pipeline` gives the honest 0.59. Scaling or imputing before the split leaks in the same way, more mildly.

2.10-Q4 Why `scaler.fit_transform(X_train)` but `scaler.transform(X_test)`? [3 marks]

The test set must be treated as unseen: the mean and standard deviation come from the training data only. Calling `fit` on the test set would use its own statistics and leak information.

2.10-Q5 Explain overfitting using the decision-tree output. [3 marks]

`max_depth=None` gives training accuracy 1.00 with 53 leaves but test accuracy 0.66, below the depth-3 tree's 0.70. The gap between train and test accuracy is the symptom. Limit depth, prune, add data or use cross-validation to choose complexity.

2.10-Q6 What does `stratify=y` do in `train_test_split`? [2 marks]

It keeps the class proportions equal in train and test (positives 0.628 / 0.626 against 0.627 overall in the earlier run), which matters for imbalanced classes.

2.10-Q7 Why use a Pipeline with PCA or scaling when cross-validating? [3 marks]

Each fold must fit the scaler or PCA on its own training part. Fitting once on all the data lets the validation folds influence the transformation (leakage).

2.11 PCA in depth (tricky topic)

Goal. Principal Component Analysis rotates the data onto new orthogonal axes (principal components, PCs) ordered by how much variance they capture, so that a few PCs summarise most of the information. It is unsupervised (ignores the target), linear, and removes correlation between features.

Algorithm

Step	What you do	Formula
1 Centre	subtract each column's mean (standardise too if units differ)	Xc = X - mean
2 Covariance	variances on the diagonal, covariances elsewhere	C = XcT Xc / (n - 1)
3 Eigen-decompose	eigenvalues = variance along each PC, eigenvectors = PC directions (unit length, orthogonal)	C v = lambda v; det(C - lambda I) = 0
4 Sort and choose k	largest eigenvalues first; keep k PCs	ratio_i = lambda_i / sum(lambda)
5 Project	scores = coordinates of the data on the kept PCs	Z = Xc Wk (Wk = first k eigenvectors)
6 Reconstruct (optional)	map back; the error equals the discarded variance	Xhat = Z WkT + mean

Remember. For a 2 x 2 matrix: lambda^2 - trace(C) lambda + det(C) = 0. Then the eigenvector of lambda comes from the first row of (C - lambda I)v = 0: v is proportional to (c12, lambda - c11); divide by its length.

Worked example: five points (2,1), (3,5), (4,3), (5,6), (6,5)

Verified output (every number below is from this run)

```
STEP 1  mean and centering
mean = [4. 4.]
centered data:
 [[-2. -3.]
 [-1.  1.]
 [ 0. -1.]
 [ 1.  2.]
 [ 2.  1.]]
column sums of centered data: [0. 0.]

STEP 2  covariance matrix  C = Xc^T Xc / (n-1)
[[2.5  2.25]
 [2.25 4.  ]] | same as np.cov: True
variances on the diagonal: [2.5 4. ] | covariance off-diagonal: 2.25 | total variance (trace): 6.5

STEP 3  eigenvalues and eigenvectors  (C v = lambda v)
characteristic equation: lambda^2 - 6.50 lambda + 4.9375 = 0  ->  lambda = 5.6217, 0.8783
eigenvalues (sorted): [5.6217 0.8783] | eigenvectors as columns:
 [[ 0.5847 -0.8112]
 [ 0.8112  0.5847]]
hand-built PC1 from (C - lambda I)v = 0: [0.5847 0.8112] | orthogonal PCs: v1.v2 = -0.0

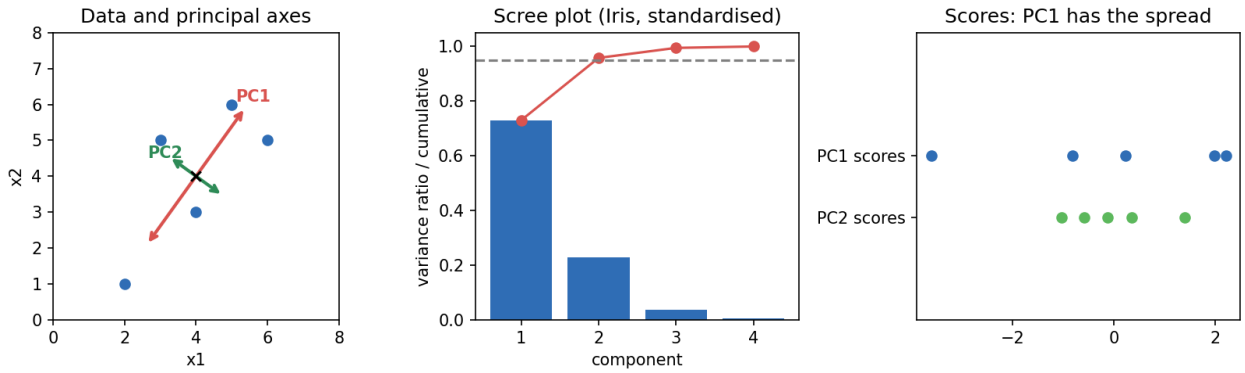
STEP 4  explained variance
explained variance ratio: [0.8649 0.1351] | cumulative: [0.8649 1.    ]
```

Verified output (continued)

```
STEP 5  project  Z = Xc W  (keep k = 1)
scores on PC1: [-3.6031  0.2265 -0.8112  2.2072  1.9807] | variance of scores: 5.6217 = lambda1

STEP 6  reconstruct and measure the loss
reconstruction from 1 component:
 [[1.8932 1.077 ]
 [4.1325 4.1838]
 [3.5257 3.3419]
 [5.2906 5.7906]
 [5.1581 5.6068]]
squared error = 3.5132 | (n-1) * discarded eigenvalue = 3.5132

CHECK  SVD route and scikit-learn
singular values^2/(n-1): [5.6217 0.8783] | components (rows of Vt):
 [[ 0.5847  0.8112]
 [ 0.8112 -0.5847]]
sklearn explained_variance_: [5.6217 0.8783] | ratio: [0.8649 0.1351]
sklearn components_:
 [[ 0.5847  0.8112]
 [ 0.8112 -0.5847]]
(signs of a component may be flipped: -v and v are the same direction)
sklearn transform equals centered data @ components^T: True
```



Left: data with the two principal axes (PC1 follows the long direction). Middle: scree plot and cumulative variance for Iris (standardised) with the 95% line. Right: the five scores on PC1 and PC2.

Code (n2_pca.py): steps 1 to 6 and the checks

```
print("STEP 1  mean and centering")
mu = X.mean(axis=0)
Xc = X - mu
print("mean =", mu, "\ncentered data:\n", Xc, "\ncolumn sums of centered data:", Xc.sum(axis=0))

print("\nSTEP 2  covariance matrix  C = Xc^T Xc / (n-1)")
C = Xc.T @ Xc / (n - 1)
print(C, "| same as np.cov:", np.allclose(C, np.cov(X.T)))
print("variances on the diagonal:", np.diag(C), "| covariance off-diagonal:", C[0, 1], "| total variance (trace):",
    np.trace(C))

print("\nSTEP 3  eigenvalues and eigenvectors  (C v = lambda v)")
tr, det = np.trace(C), np.linalg.det(C)
disc = np.sqrt(tr ** 2 - 4 * det)
lam_hand = np.array([(tr + disc) / 2, (tr - disc) / 2])
print(f"characteristic equation: lambda^2 - {tr:.2f} lambda + {det:.4f} = 0  ->  lambda = {lam_hand[0]:.4f},
    {lam_hand[1]:.4f}")
w, V = np.linalg.eigh(C)
order = np.argsort(w)[::-1]
w, V = w[order], V[:, order]
print("eigenvalues (sorted):", w, "| eigenvectors as columns:\n", V)
v1_hand = np.array([C[0, 1], lam_hand[0] - C[0, 0]])
v1_hand /= np.linalg.norm(v1_hand)
print("hand-built PC1 from (C - lambda I)v = 0:", v1_hand, "| orthogonal PCs: v1.v2 =", round(float(V[:, 0] @ V[:,
    1]), 10))

print("\nSTEP 4  explained variance")
ratio = w / w.sum()
print("explained variance ratio:", ratio, "| cumulative:", np.cumsum(ratio))

print("\nSTEP 5  project  Z = Xc W  (keep k = 1)")
Z = Xc @ V[:, :1]
print("scores on PC1:", Z.ravel(), "| variance of scores:", round(float(Z.var(ddof=1)), 4), "= lambda1")

print("\nSTEP 6  reconstruct and measure the loss")
Xhat = Z @ V[:, :1].T + mu
print("reconstruction from 1 component:\n", Xhat)
sse = ((X - Xhat) ** 2).sum()
print("squared error =", round(float(sse), 4), "| (n-1) * discarded eigenvalue =", round(float((n - 1) * w[1]), 4))

print("\nCHECK  SVD route and scikit-learn")
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
print("singular values^2/(n-1):", S ** 2 / (n - 1), "| components (rows of Vt):\n", Vt)
pca = PCA(n_components=2).fit(X)
print("sklearn explained_variance_:", pca.explained_variance_, "| ratio:", pca.explained_variance_ratio_)
print("sklearn components_:\n", pca.components_, "\n(signs of a component may be flipped: -v and v are the same
    direction)")
print("sklearn transform equals centered data @ components^T:", np.allclose(pca.transform(X), Xc @
    pca.components_.T) or np.allclose(np.abs(pca.transform(X)), np.abs(Xc @ pca.components_.T)))
```

Scaling, variance and other traps

Verified output

```
--- does scaling matter? Iris, raw vs standardised ---
raw (cm)       ratio: [0.9246 0.0531 0.0171 0.0052] | PC1 loadings: [ 0.3614 -0.0845  0.8567  0.3583]
standardised   ratio: [0.7296 0.2285 0.0367 0.0052] | PC1 loadings: [ 0.5211 -0.2693  0.5804  0.5649]
components needed for >= 95% (standardised): 2 | PCA(0.95) picks: 2

--- a unit change alone rewrites PC1 ---
cm: PC1 = [0.1441 0.9896] | mm: PC1 = [1.     0.0085] (height dominates just because its numbers got bigger)

--- high variance is not the same as predictive ---
PC1 loadings: [1.     0.0041] -> PC1 is almost entirely the noisy feature
correlation of PC1 score with y: 0.004 | of PC2 score: 0.792

--- limits: max components, outliers, inverse_transform ---
components limit = min(n_samples, n_features): 4
PC1 direction without outlier: [ 1.     -0.0075] | with one outlier: [0.7956 0.6058]

saved notes_pca.png
```

2.11-Q1 What does PCA do, and what quantity does it maximise? [5 marks]

It finds orthogonal directions that capture the maximum variance in turn: PC1 is the direction of greatest variance, PC2 the greatest remaining variance orthogonal to PC1, and so on. Equivalently it minimises the squared reconstruction error when data are projected on k components.

2.11-Q2 List the steps of PCA and the role of the covariance matrix. [5 marks]

Centre (and usually scale) > covariance matrix > eigenvalues and eigenvectors > sort > select k > project. The covariance matrix records how features vary and co-vary; its eigenvectors are the PC directions, its eigenvalues the variance along each.

2.11-Q3 Compute the covariance matrix for the five points above. [tricky]

Mean (4, 4); centred points (-2,-3), (-1,1), (0,-1), (1,2), (2,1). Var x = (4+1+0+1+4)/4 = 2.5; var y = (9+1+1+4+1)/4 = 4; cov = (6 - 1 + 0 + 2 + 2)/4 = 2.25. So C = [[2.5, 2.25], [2.25, 4]]. Divide by n - 1 = 4 (sample covariance), as `np.cov` does.

2.11-Q4 Find the eigenvalues of C by hand and the percentage of variance on PC1. [tricky]

trace = 6.5, det = 2.5 x 4 - 2.25^2 = 4.9375. lambda^2 - 6.5 lambda + 4.9375 = 0 gives lambda = (6.5 +- sqrt(42.25 - 19.75))/2 = (6.5 +- 4.7434)/2 = 5.6217 and 0.8783. PC1 explains 5.6217 / 6.5 = 86.5% (the sum of eigenvalues equals the total variance, the trace).

2.11-Q5 Find the PC1 direction and the score of the first point. [tricky]

v is proportional to (c12, lambda - c11) = (2.25, 5.6217 - 2.5) = (2.25, 3.1217); its length is 3.8480, so v = (0.5847, 0.8112). First point centred is (-2, -3): score = -2(0.5847) - 3(0.8112) = -3.603, matching the output (-3.6031). PC2 is orthogonal: (-0.8112, 0.5847), eigenvalue 0.8783.

2.11-Q6 How much information is lost keeping only PC1 here? [tricky]

13.5% of the variance (0.8783 / 6.5). The squared reconstruction error summed over the points is 3.5132 = (n - 1) x discarded eigenvalue = 4 x 0.8783 (output, step 6).

2.11-Q7 Why must the data be centred? Why standardise? [tricky]

PCA describes variation *around the mean*; without centring, the first PC mostly points at the mean. Variance depends on units, so a feature in millimetres would dominate one in centimetres. In the output, changing height from cm to mm flips PC1 from (0.14, 0.99) to (1.00, 0.0085). Standardise (z-scores) when features have different units or scales; PCA then works on the correlation matrix.

2.11-Q8 Why does PCA on raw Iris give 92.5% for PC1 but 73.0% after standardising? [tricky]

Petal length has by far the largest variance (in cm), so raw PCA is dominated by it (PC1 loading 0.86). After z-scoring every feature has variance 1 and the structure shared by several features appears, giving [0.730, 0.229, 0.037, 0.005] and a different PC1 (0.52, -0.27, 0.58, 0.56). Two components then hold 95.8%.

2.11-Q9 How many components should you keep? [3 marks]

Common rules: enough to reach a target of cumulative variance (e.g. 95%: `PCA(0.95)` picks 2 for standardised Iris); the elbow of the scree plot; Kaiser rule (eigenvalue > 1 on standardised data); or the number that gives the best cross-validated score of the downstream model.

2.11-Q10 What are loadings and scores? [3 marks]

Loadings (`components_`) are the weights of the original features in each PC: Iris PC1 = 0.52 sepal length - 0.27 sepal width + 0.58 petal length + 0.56 petal width, so PC1 is a 'size' axis. Scores (`transform`) are the coordinates of each observation on the PCs.

2.11-Q11 Is a PC with high variance always useful for prediction? [tricky]

No. PCA ignores the target. In the demo a large-variance noise feature becomes PC1 and has correlation 0.004 with the label, while the low-variance PC2 (the real signal) has 0.79. Check downstream performance; supervised alternatives are LDA or feature selection.

2.11-Q12 The signs of components differ between my hand calculation and scikit-learn. Is that a bug? [tricky]

No. An eigenvector and its negative describe the same axis. Compare variances, ratios and absolute loadings, not signs (the SVD route in the output gave (0.81, -0.58) for PC2 where the hand value is (-0.81, 0.58)).

2.11-Q13 What is the maximum number of principal components? [2 marks]

min(n_samples, n_features) (4 for a 4 x 10 matrix in the output); only n - 1 of them can have non-zero variance after centring.

2.11-Q14 What is the relation between PCA and SVD? [3 marks]

If Xc = U S VT, the rows of VT are the PCs and the eigenvalues are s^2 / (n - 1) (output: 5.6217 and 0.8783 again). scikit-learn uses SVD because it is numerically stabler than forming the covariance matrix.

2.11-Q15 Name limitations of PCA. [3 marks]

Linear only (use kernel PCA, t-SNE or UMAP for curved structure); sensitive to outliers (one point at (30, 30) rotated PC1 from (1, 0) to (0.80, 0.61) in the demo); components are hard to interpret; needs numeric features (encode categoricals first); variance is not the same as importance.

2.11-Q16 Where must PCA be fitted when you also have a test set? [2 marks]

On the training data only, then `transform` the test data: put the scaler and PCA inside a `Pipeline` (see 2.10).

2.11-Q17 PCA versus feature selection versus LDA. [3 marks]

	PCA	Feature selection	LDA
Supervised?	no	usually yes	yes
Output	new features (linear mixes)	subset of original features	new axes that separate classes
Goal	maximise variance	keep informative features	maximise class separation

2.12 K-means clustering in depth (tricky topic)

- Unsupervised clustering into k groups. **Lloyd's algorithm**: (1) choose k initial centroids (k-means++ spreads them out); (2) assign every point to the nearest centroid; (3) move each centroid to the mean of its points; repeat 2 to 3 until assignments stop changing.
- Objective: minimise **inertia** (within-cluster sum of squared distances to the centroid). It never increases, so the algorithm converges, but only to a **local** optimum: run with several starts (`n_init`) and keep the best.
- Choosing k: elbow plot of inertia, silhouette score (-1 to 1, higher is better), or domain knowledge. Inertia always falls as k grows, so it cannot choose k alone.
- Assumes round, similar-sized clusters of similar density; sensitive to feature scale and outliers; needs numeric data. Cost about O(n x k x d) per iteration.

Verified output

```
=== K-MEANS by hand: 6 points, k = 2, one full iteration ===
start centroids: [[1.0, 1.0], [5.0, 7.0]]
distances to c1, c2:
 [[0.    7.211]
 [1.118 6.103]
 [3.606 3.606]
 [7.211 0.   ]
 [4.717 2.5  ]
 [5.315 2.062]]
assignment: [0, 0, 0, 1, 1, 1]
new centroids (means of members): [[1.8333333333333333, 2.3333333333333335], [4.333333333333333, 5.666666666666667]]
point 3 = (3, 4) is equally far from both centroids (3.606 each); argmin keeps the first, so it joins cluster 0
inertia (sum of squared distances to the NEW centroids): 10.667
second assignment: [0, 0, 0, 1, 1, 1] -> unchanged, so converged: True
sklearn with the same start: labels [0, 0, 0, 1, 1, 1] centroids [[1.8333333333333335, 2.3333333333333335],
    [4.333333333333333, 5.666666666666666]] inertia 10.667

=== K-means choices: k-means++, n_init, elbow, silhouette, scaling ===
k=2: inertia=  8867.6  silhouette=0.605
k=3: inertia=  2313.8  silhouette=0.786
k=4: inertia=   365.6  silhouette=0.846
k=5: inertia=   327.1  silhouette=0.735
k=6: inertia=   293.0  silhouette=0.595
one random start per seed gives different inertias (local minima): [2278.5, 365.6, 2280.5, 365.6, 2279.7, 2283.7]
n_init=10 keeps the best one: 365.6
agreement (ARI) with the unit-scaled clustering - raw x1000 feature: 0.774 | after standardising: 1.0
```

2.12-Q1 Run one full K-means iteration for points (1,1), (1.5,2), (3,4), (5,7), (3.5,5), (4.5,5) with starting centroids (1,1) and (5,7). [5 marks]

Assignments by distance: the first three points go to centroid 1, the last three to centroid 2 (point (3,4) is exactly 3.606 from both; ties go to the first centroid). New centroids are the member means: (1.833, 2.333) and (4.333, 5.667). A second assignment pass changes nothing, so the algorithm has converged; inertia = 10.667 (scikit-learn gives the same).

2.12-Q2 Why can two runs of K-means give different answers? [tricky]

The result depends on the starting centroids and the algorithm finds only a local minimum. With one random start per seed the inertia was either 365.6 (the good solution) or between 2278 and 2284 (a poor local minimum); `n_init=10` keeps the best run (365.6). k-means++ initialisation reduces the problem.

2.12-Q3 How do you choose k? What do the printed inertia and silhouette show? [3 marks]

Inertia drops sharply until k = 4 and flattens afterwards (366, then 327, 293): the elbow. Silhouette peaks at k = 4 (0.846). Both point to 4, the number of blobs generated.

2.12-Q4 Why must features be scaled before K-means? [tricky]

It uses Euclidean distance, so a feature with large numbers dominates. Multiplying one coordinate by 1000 changed the clusters (agreement with the unscaled solution fell to 0.774 ARI); standardising restored 1.0.

2.12-Q5 K-means versus hierarchical clustering versus DBSCAN. [3 marks]

	K-means	Hierarchical	DBSCAN
k needed?	yes	cut the dendrogram	no (eps, min_samples)
Shapes	round	depends on linkage	arbitrary, finds noise
Scales to big n	well	poorly	moderately

2.12-Q6 Is K-means classification? How do you evaluate clusters without labels? [3 marks]

No, it is unsupervised: cluster numbers are arbitrary labels. Evaluate with inertia, silhouette, or (if true labels exist) adjusted Rand index or a cross-tab. In the Iris run cluster 1 held exactly the 50 setosa flowers while versicolor and virginica overlapped.

2.13 Deep learning frameworks: Keras, TensorFlow, PyTorch

- A neural network is layers of weighted sums plus activation functions. **Training**: forward pass (predictions) > loss > **backpropagation** (gradients by the chain rule) > optimiser step `w = w - lr x gradient`.
- Vocabulary: **epoch** = one pass over all training data; **batch** = rows per update; **iteration** = one update (150 rows / batch 16 = 10 updates per epoch); **learning rate** = step size.

- Activations: ReLU `max(0, z)` (hidden layers), sigmoid (binary output), softmax (class probabilities that sum to 1), tanh. Losses: MSE (regression), cross-entropy (classification).
- Parameters of a dense layer = inputs x units + units (bias): 4 -> 8 gives 40; 8 -> 3 gives 27; total 67.
- **TensorFlow** (Google): tensors, automatic differentiation (`GradientTape`), graph compilation, deployment tools. **Keras**: high-level API (`Sequential`, `compile`, `fit`, `predict`); Keras 3 runs on TensorFlow, PyTorch or JAX. **PyTorch** (Meta): define-by-run tensors and `autograd`, explicit training loop, popular in research.
- Overfitting remedies: more data, dropout, weight decay (L2), early stopping, smaller network.

Verified output (TensorFlow 2.21, Keras 3.12, PyTorch 2.14)

```
=== one gradient-descent step by hand: y = w*x + b, MSE loss ===
pred=[0.0, 0.0, 0.0] loss=mean((pred-y)^2)=18.6667
dL/dw = mean(2(pred-y)x) = -18.6667 | dL/db = mean(2(pred-y)) = -8.0000
update: w = w - lr*dL/dw = 1.8667 | b = b - lr*dL/db = 0.8000
PyTorch autograd : dL/dw = -18.6667, dL/db = -8.0000
TensorFlow tape  : dL/dw = -18.6667, dL/db = -8.0000
trap: PyTorch accumulates gradients - calling backward() twice without zero_grad():
   grad after two backward calls = 6.0 (should be 3 each time -> 6 means accumulation)

=== activations ===
sigmoid(0) = 0.5 | relu([-2, 3]) = [0, 3] | tanh(0) = 0.0
softmax([1,2,3]) = [0.09, 0.2447, 0.6652] sums to 1.0 -> class probabilities

=== Keras (Sequential API): 4 inputs -> Dense(8, relu) -> Dense(3, softmax) ===
params layer 1 = 4*8 + 8 = 40 | layer 2 = 8*3 + 3 = 27 | model.count_params() = 67
loss 0.897 -> 0.039 | accuracy 0.60 -> 0.98
predict output shape: (2, 3) (one probability per class)
one epoch = a pass over all 150 rows; with batch_size=16 that is 10 weight updates per epoch

=== PyTorch: the explicit training loop ===
loss 1.112 -> 0.090 | accuracy 0.97 | parameters 67

=== TensorFlow tensors and shapes ===
tf.matmul: [[7.0, 10.0], [15.0, 22.0]] | shape (2, 2) | dtype float32 | numpy interop: ndarray
```

Keras and PyTorch code (n2_dl.py)

```python
keras.utils.set_random_seed(0)
Xi, yi = load_iris(return_X_y=True)
Xi = (Xi - Xi.mean(0)) / Xi.std(0)
model = keras.Sequential([keras.layers.Input(shape=(4,)), keras.layers.Dense(8, activation="relu"),
    keras.layers.Dense(3, activation="softmax")])
print("params layer 1 = 4*8 + 8 =", 4 * 8 + 8, "| layer 2 = 8*3 + 3 =", 8 * 3 + 3, "| model.count_params() =",
    model.count_params())
model.compile(optimizer=keras.optimizers.Adam(0.05), loss="sparse_categorical_crossentropy", metrics=["accuracy"])
hist = model.fit(Xi, yi, epochs=30, batch_size=16, verbose=0)
print(f"loss {hist.history['loss'][0]:.3f} -> {hist.history['loss'][-1]:.3f} | accuracy
    {hist.history['accuracy'][0]:.2f} -> {hist.history['accuracy'][-1]:.2f}")
print("predict output shape:", model.predict(Xi[:2], verbose=0).shape, "(one probability per class)")
print("one epoch = a pass over all 150 rows; with batch_size=16 that is", int(np.ceil(150 / 16)), "weight updates
    per epoch")

print("\n=== PyTorch: the explicit training loop ===")
torch.manual_seed(0)
net = torch.nn.Sequential(torch.nn.Linear(4, 8), torch.nn.ReLU(), torch.nn.Linear(8, 3))
opt = torch.optim.Adam(net.parameters(), lr=0.05)
lossf = torch.nn.CrossEntropyLoss()
Xt = torch.tensor(Xi, dtype=torch.float32)
yt = torch.tensor(yi, dtype=torch.long)       # CrossEntropyLoss needs int64 class labels; int32 raises a
    RuntimeError
first = None
for epoch in range(30):
    opt.zero_grad()                # 1 clear old gradients
    out = net(Xt)                  # 2 forward pass
    loss = lossf(out, yt)          # 3 loss
    loss.backward()                # 4 backpropagation
    opt.step()                     # 5 weight update
    first = first if first is not None else loss.item()
print(f"loss {first:.3f} -> {loss.item():.3f} | accuracy {(net(Xt).argmax(1) == yt).float().mean().item():.2f} |
    parameters {sum(p.numel() for p in net.parameters())}")
```

2.13-Q1 Do one gradient-descent step for y = w x + b with x = [1,2,3], y = [2,4,6], w = b = 0, learning rate 0.1 (MSE loss). [tricky]

Predictions are 0, so errors are -2, -4, -6 and loss = (4 + 16 + 36)/3 = 18.667. dL/dw = mean(2 x error x x) = (2/3)(-2 - 8 - 18) = -18.667; dL/db = (2/3)(-2 - 4 - 6) = -8. Update: w = 0 - 0.1(-18.667) = 1.867; b = 0 - 0.1(-8) = 0.8. PyTorch autograd and TensorFlow's tape return the same gradients (output).

2.13-Q2 PyTorch loss stays wrong / gradients look doubled. What is the usual cause? [tricky]

Gradients accumulate in `.grad` across `backward()` calls (6.0 instead of 3.0 in the output). Call `optimizer.zero_grad()` at the start of every iteration. The loop is: zero_grad, forward, loss, backward, step.

2.13-Q3 `CrossEntropyLoss` raises `expected target dtype to be Long or Byte, but got Int`. Why? [tricky]

Class labels must be 64-bit integers. NumPy/scikit-learn labels can be int32 (on Windows); convert with `torch.tensor(y, dtype=torch.long)` (this error occurred while preparing these notes).

2.13-Q4 Keras: how many parameters does `Dense(8)` after a 4-feature input have, and what is the output shape of a 3-class softmax head for 2 samples? [3 marks]

4 x 8 + 8 = 40 parameters. The output shape is (2, 3): one probability per class per sample.

2.13-Q5 Compare Keras, TensorFlow and PyTorch. [3 marks]

	Keras	TensorFlow	PyTorch
Level	high-level API	full framework	full framework
Style	`compile` + `fit`	tensors, tapes, graphs	explicit loop, autograd
Strength	fast prototyping	deployment ecosystem	research flexibility

2.13-Q6 Explain epoch, batch and iteration; why use mini-batches? [3 marks]

Mini-batches give frequent updates at moderate cost and fit in memory, with some noise that helps escape poor minima. 150 rows with batch 16 makes ceil(150/16) = 10 iterations per epoch.

2.13-Q7 Why use softmax with cross-entropy for multi-class output? [2 marks]

Softmax turns scores into probabilities that sum to 1 (softmax([1,2,3]) = [0.09, 0.245, 0.665]); cross-entropy penalises low probability on the true class.

2.14 XGBoost

- **Gradient-boosted decision trees**: trees are added one at a time, each fitted to the errors (gradients of the loss) of the ensemble so far. Contrast **bagging** (random forest: independent trees averaged).
- Key hyperparameters: `n_estimators`, `learning_rate` (shrinkage), `max_depth`, `subsample`, `colsample_bytree`, regularisation `reg_lambda`, `reg_alpha`. Provides `feature_importances_`, handles missing values, strong on tabular data.
- **Early stopping** needs a separate validation set: using the test set to decide when to stop leaks information.

Verified output (XGBoost 3.2)

```
=== XGBoost: boosting adds trees that fix the previous errors ===
n_estimators=1    test accuracy=0.924
n_estimators=10   test accuracy=0.959
n_estimators=100  test accuracy=0.953
top features: ['worst perimeter', 'worst radius', 'worst concave points']
early stopping: best tree = 51 of 500 allowed | untouched test accuracy: 0.936 (never use the test set for early
    stopping)
```

2.14-Q1 Boosting versus bagging. [3 marks]

Boosting builds trees sequentially, each correcting the previous ones (reduces bias, can overfit with too many trees). Bagging builds independent trees on bootstrap samples and averages them (reduces variance).

2.14-Q2 Why can adding more trees hurt, and how does early stopping help? [3 marks]

Training error keeps falling but validation error eventually rises (overfitting; test accuracy went 0.959 at 10 trees to 0.953 at 100). Early stopping stops when the validation loss has not improved for N rounds (best tree 51 of 500 here).

2.14-Q3 Why is a small learning rate usually paired with more trees? [2 marks]

Each tree contributes a smaller correction, so the model approaches the optimum more smoothly and generalises better, at the cost of more trees and time.

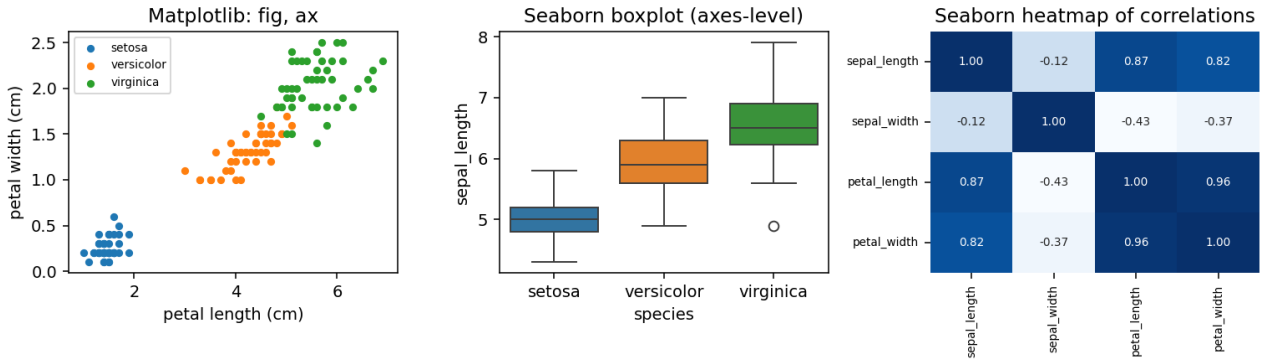
2.15 Data visualisation: Matplotlib, Seaborn, Bokeh, Plotly, Folium

Library	Style	Key objects / calls	Best for
Matplotlib	static, low-level	`fig, ax = plt.subplots()`, `ax.plot/scatter/bar/hist`	full control, publication figures
Seaborn	static, statistical, built on Matplotlib	`histplot, boxplot, heatmap, pairplot, relplot`	quick exploration of DataFrames
Bokeh	interactive (BokehJS), servers	`figure()`, `ColumnDataSource`, `HoverTool`, `output_file`, `save`	linked interactive plots, dashboards
Plotly	interactive (Plotly.js), Dash apps	`plotly.express`: `px.bar/scatter/line`; `graph_objects`	fast interactive charts, 3-D, maps
Folium	interactive maps (Leaflet)	`folium.Map`, `Marker`, `Choropleth`, `.save('map.html')`	geographic data

- Matplotlib has two interfaces: the pyplot state machine (`plt.plot` draws on the current axes) and the explicit object-oriented one (`fig, ax = plt.subplots(); ax.plot(...)`). Use the object-oriented form for anything with several panels.
- Seaborn **axes-level** functions accept `ax=`; **figure-level** ones (`pairplot`, `relplot`, `catplot`) create their own figure.
- Choosing a chart: comparison across categories > bar; trend over time > line; relationship of two numbers > scatter; distribution > histogram or box plot; correlations > heatmap; geography > map. Label axes and give a title.

Verified output

```
pyplot state-machine call plt.plot(...) draws on the 'current axes'; the object-oriented form ax.plot(...) names the
    axes explicitly.
plt.subplots returns: Figure and ndarray of shape (3,)
Seaborn axes-level functions (boxplot, heatmap, scatterplot) accept ax=...; figure-level ones (pairplot, relplot,
    catplot) create their own figure.
pairplot returns: PairGrid (a figure-level object)
saved notes_viz.png
```



Matplotlib scatter with the object-oriented API, Seaborn box plot (axes-level) and heatmap of correlations (Iris).

Tested code for the lab charts (Plotly, Bokeh, Folium)

```
# ---- Lab 6: Plotly interactive bar chart ----------------------------------
cities = pd.DataFrame({"city": ["Mumbai", "Delhi", "Bengaluru", "Chennai", "Hyderabad"],
                       "population_m": [12.4, 11.0, 8.4, 7.1, 6.8]})        # illustrative values
fig_bar = px.bar(cities, x="city", y="population_m", title="Population of cities (millions)",
                 hover_data={"population_m": ":.1f"})
fig_bar.write_html("plotly_bar.html", include_plotlyjs="cdn")

# ---- Lab 7: Plotly scatter, house size vs price ---------------------------
rng = np.random.default_rng(1)
size = rng.integers(500, 3000, 40)
house = pd.DataFrame({"size_sqft": size, "price_lakh": size * 0.03 + rng.normal(0, 8, 40)})
fig_sc = px.scatter(house, x="size_sqft", y="price_lakh", trendline="ols",   # 'ols' needs statsmodels
                    title="House size vs price")
fig_sc.write_html("plotly_scatter.html", include_plotlyjs="cdn")

# ---- Lab 8: Bokeh line plot, daily temperature ----------------------------
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
temps = [31, 33, 32, 35, 34, 30, 29]
src = ColumnDataSource(dict(day=days, temp=temps))
p1 = figure(x_range=days, title="Daily temperature", height=300, width=500,
            tools="pan,wheel_zoom,reset,save")
p1.line("day", "temp", source=src, line_width=2)
p1.scatter("day", "temp", source=src, size=8)
p1.add_tools(HoverTool(tooltips=[("day", "@day"), ("temp", "@temp °C")]))
output_file("bokeh_line.html")
save(p1)

# ---- Lab 9: Bokeh bar chart, product sales --------------------------------
products = ["Laptop", "Phone", "Tablet", "Watch"]
sales = [120, 300, 90, 150]
p2 = figure(x_range=products, title="Sales by product", height=300, width=500)
p2.vbar(x=products, top=sales, width=0.6)
output_file("bokeh_bar.html")
save(p2)

# ---- Lab 10: Bokeh scatter, Iris petal length vs width --------------------
iris = load_iris(as_frame=True).frame
p3 = figure(title="Iris: petal length vs width", height=300, width=500)
p3.scatter(iris["petal length (cm)"], iris["petal width (cm)"], size=6, alpha=0.6)
output_file("bokeh_scatter.html")
save(p3)

# ---- Folium: map with a marker --------------------------------------------
m = folium.Map(location=[12.9692, 79.1559], zoom_start=12)                # Vellore
folium.Marker([12.9692, 79.1559], popup="VIT Vellore").add_to(m)
m.save("folium_map.html")
```

Output (HTML files written)

```
plotly_bar.html      written,     8,190 characters
plotly_scatter.html  written,     9,758 characters
bokeh_line.html      written,     6,570 characters
bokeh_bar.html       written,     6,396 characters
bokeh_scatter.html   written,     7,401 characters
folium_map.html      written,     4,073 characters

Plotly bar trace type  : bar with 5 bars
Plotly scatter traces  : ['markers', 'lines']
Iris points plotted    : 150
```

2.15-Q1 Compare Matplotlib and Seaborn. [3 marks]

Matplotlib is the foundation with full control over figures and axes. Seaborn sits on top, understands DataFrames, and gives statistical plots and good defaults in one call; you still customise with Matplotlib.

2.15-Q2 What is the difference between `plt.plot(...)` and `ax.plot(...)`? [2 marks]

`plt.plot` implicitly uses the 'current' axes (global state), which becomes confusing with several subplots. `ax.plot` names the axes object explicitly (`fig, ax = plt.subplots()`).

2.15-Q3 Create an interactive bar chart of city populations in Plotly and a line plot of daily temperatures in Bokeh (lab experiments 6 and 8). [5 marks]

See the tested code above: `px.bar(cities, x='city', y='population_m')` followed by `write_html`, and a Bokeh `figure(x_range=days)` with `line`, `scatter` and a `HoverTool`. Open the HTML files to hover and zoom.

2.15-Q4 Which library would you choose for (a) a map with markers, (b) a seaborn-style correlation heatmap, (c) an interactive web dashboard? [2 marks]

(a) Folium (or Plotly maps), (b) Seaborn `heatmap`, (c) Plotly with Dash or Bokeh server.

2.15-Q5 What does `ColumnDataSource` do in Bokeh? [2 marks]

It holds the columns shared by glyphs and tools, so several plots or a hover tool refer to the same data by column name and update together when the data change.

Module 3: Scala data structures and object-oriented programming

Syllabus (12 hours): expanded function format; variables and strings; user input; numbers; variable types; operators; booleans; arrays, lists, tuples, sets, hash set, maps; functional combinators; objects and classes; anonymous object, singleton; companion object, case classes; constructors; method overloading; `this`; inheritance; field overriding; `final`; abstract class; trait and mixins; access modifiers; Scala array and the REPL.

Where each syllabus topic is covered

Syllabus topic	Section
Expanded function format	3.1
Variables and strings; getting user input	3.2
Numbers, variable types, operators, booleans	3.3
Arrays, lists, tuples, sets, hash set, maps	3.4
Functional combinators (map, flatMap, fold ...)	3.5
Option and pattern matching	3.6
Objects and classes, constructors, method overloading, `this`	3.7
Anonymous object, singleton, companion object	3.8
Case classes	3.9
Inheritance, field overriding, final, abstract class	3.10
Trait, trait mixins	3.11
Access modifiers	3.12
Scala array, REPL	3.13

3.1 Functions: expanded format, short forms, higher-order functions

- Expanded format: `def name(p1: T1, p2: T2): R = { body }`. The **last expression is the result**; `return` is legal but unidiomatic. Parameter types are mandatory; the result type may be inferred (annotate public methods anyway).
- Short form: `def add(a: Int, b: Int) = a + b`. Function literal (anonymous function): `(a: Int, b: Int) => a + b`; placeholder form `_ + _`. A function is a value: `val f: Int => Int = _ * 2`.
- Higher-order: takes or returns functions (`map`, `filter`). Default and named arguments, varargs (`xs: Int*`, pass a list with `: _*`), currying (`def add(a: Int)(b: Int)`), partial application (`add(5) _`).
- **Call-by-value** (default): the argument is evaluated once before the call. **Call-by-name** (`x: => Int`): evaluated at every use inside the function.
- A closure captures the *variable*, not a copy of its value.

Verified output (first lines: function forms)

```
functions : 5 5 5 18
val/var    : 10 1, types int
  (computing lazy val once)
lazy       : 42 42
s-string   : Hello Rohan, next year 2027
```

Verified output

```
--- val, var, def, lazy val, call-by-name ---
after definitions: calls=1
v twice: 1 1 | d twice: 2 3 | l twice: 4 4 | calls=4
  byValue body
byValue(next()) = 2 (argument evaluated once, before the body); calls=1
  byName body
byName(next())  = 3 (argument evaluated at each use); calls=2

--- functions: default/named args, varargs, currying, closures ---
Hello, Asha! | Hello, Ravi? | Hi, Meena!
varargs: 0 6 9
curried: add(2)(3) = 5 | add5(10) = 15
closure sees the variable, not a copy: times(3) = 30
andThen composes functions left to right: (5 + 1) * 2 = 12
Unit value: () | Any holds anything: List(1, a, 2.0, true) | Nothing: throw has type Nothing: true
```

3.1-Q1 What do `v twice`, `d twice` and `l twice` print for `val v = next()`, `def d = next()`, `lazy val l = next()`? [tricky]

`v twice: 1 1 | d twice: 2 3 | l twice: 4 4`. The `val` ran once at definition; the `def` runs on every use (2 then 3); the `lazy val` ran once at its first use (4) and is then cached. At the end `calls = 4`.

3.1-Q2 Explain call-by-value and call-by-name using `byValue(next())` and `byName(next())`. [tricky]

With call-by-value `next()` ran once (calls = 1) before the body started and the result 1 + 1 = 2. With call-by-name (`x: => Int`) the expression is re-evaluated at each use of `x`, so it ran twice (1 + 2 = 3, calls = 2). Use by-name for lazy defaults, logging and custom control structures.

3.1-Q3 Write `add` in expanded form, short form and as a function value, and show a higher-order function. [3 marks]

Tested in the question bank (M3-C5): all four give 5 or 18

```
def addExpanded(a: Int, b: Int): Int = {
  a + b
}
def add(a: Int, b: Int) = a + b
val addLit: (Int, Int) => Int = (a, b) => a + b
def twice(f: Int => Int, x: Int) = f(f(x))
twice(_ * 3, 2)        // 18
```

3.1-Q4 A closure `val times = (x: Int) => x * factor` is defined with `var factor = 2`, then `factor = 10`. What is `times(3)`? [tricky]

30. The closure captures the variable `factor`, not the value 2 (output: `times(3) = 30`).

3.1-Q5 What is currying and what is partial application? [3 marks]

Currying splits a multi-argument function into a chain of single-argument lists: `def add(a: Int)(b: Int)`. Supplying only the first list gives a new function: `val add5 = add(5) _`, so `add5(10) = 15` (output).

3.2 Variables, strings and user input

- `val` = immutable reference (cannot be reassigned); `var` = mutable; `lazy val` = evaluated at first use. Prefer `val`. Types are inferred: `val n = 10` is an `Int`.
- Strings are Java strings (immutable). Interpolation: `s"Hello $name, ${1 + 2}"`, `f"$gpa%.2f"` (printf style, type checked), `raw"no \\n escape"`. Triple quotes `"""..."""` give multi-line text; `.stripMargin` removes the `|` margin.
- Useful methods: `length, toUpperCase, reverse, split, mkString, substring, contains, replace, trim, toInt, toDouble, format`.
- Input: `scala.io.StdIn.readLine()`, `readInt()`, `readDouble()`; `args` in `main` hold command-line arguments.

Verified output

```
val/var    : 10 1, types int
  (computing lazy val once)
lazy       : 42 42
s-string   : Hello Rohan, next year 2027
f-string   : GPA = 8.46, padded = 00042
raw-string : no escape \n here
methods    : SCALA alacs List(a, b, c)
input      : age=25, language=Scala
```

3.2-Q1 Difference between `val`, `var` and `lazy val`. [3 marks]

`val` can be assigned once; `var` can be reassigned; `lazy val` is computed only when first used and then cached. For a `val` holding a mutable object, the object can still change, only the reference is fixed.

3.2-Q2 What do `s`, `f` and `raw` interpolators do? [2 marks]

`s` substitutes `$name` and `${expr}`; `f` adds printf formats and checks types at compile time (`f"$gpa%.2f"` gives 8.46 for 8.4567); `raw` leaves backslash sequences unprocessed.

3.2-Q3 How do you read an integer and a line from the keyboard? [2 marks]

The tested program feeds these from a StringReader via Console.withIn

```
val age  = scala.io.StdIn.readInt()
val lang = scala.io.StdIn.readLine()
```

3.2-Q4 What is the difference between `"5".toInt` and `'5'.toInt`? [tricky]

`"5".toInt` parses the text and gives 5. `'5'.toInt` converts the *character* to its code, 53. Use `'5'.asDigit` or `'5' - '0'` for the digit (output below). `"abc".toIntOption` returns `None` instead of throwing.

Verified output

```
--- Char vs String conversion trap ---
"5".toInt = 5 | '5'.toInt = 53 (the character code!) | '5'.asDigit = 5 | '5' - '0' = 5
scala.util.Try("abc".toInt).isFailure = true | "abc".toIntOption = None
7 / 2 = 3 | 7 / 2.0 = 3.5 | 7 % 3 = 1 | -7 % 3 = -1 | Int.MaxValue + 1 = -2147483648 | 0.1 + 0.2 =
    0.30000000000000004
```

3.3 Numbers, variable types, operators and booleans

Type	Size	Note
Byte, Short, Int, Long	8, 16, 32, 64 bit	integer literals are `Int`; `10L` is `Long`; Int overflow wraps silently
Float, Double	32, 64 bit	`3.14` is `Double`; `0.1 + 0.2 != 0.3`
Char, String, Boolean	16 bit, text, true/false	`'a'` Char, `"a"` String
Unit, Null, Nothing, Any	-	`Unit` is `()`; `Any` is the root; `Nothing` is the subtype of everything (e.g. `throw`)

- Operators are **methods**: `1 + 2` is `1.+(2)`; `xs map f` is `xs.map(f)`. Precedence is decided by the first character of the operator. Operators ending in `:` are right-associative (`1 :: Nil`).
- Integer division truncates: `7 / 2 = 3`, `7 % 3 = 1`, `-7 % 3 = -1`. Bitwise `& | ^ << >>`.
- Booleans: `&&` and `||` short-circuit; `&` and `|` always evaluate both sides. `==` is value equality (calls `equals`); `eq` is reference equality.
- Conversions are explicit: `3.9.toInt = 3`, `5.toDouble`; assigning a `Double` to an `Int` does not compile.

Verified output

```
numbers    : 3 1 3.5 7 3
types      : 2147483647 9223372036854775807 5.0 3 97
overflow   : -2147483648
bitwise    : 2 7 5 8
short-circuit: false
or          : true
equality   : true true false
```

Verified output

```
--- equality: == vs eq, arrays ---
List(1,2) == List(1,2): true | Array(1,2) == Array(1,2): false (arrays compare by reference!)
Array(1,2).sameElements(Array(1,2)): true | Array(1,2).toList == Array(1,2).toList: true
strings: s1 == s2 is true, s1 eq s2 is false
Set(1,2) == Set(2,1): true | Map(1->2) == Map(1->2): true | (1,2) == (1,2): true

--- Char vs String conversion trap ---
"5".toInt = 5 | '5'.toInt = 53 (the character code!) | '5'.asDigit = 5 | '5' - '0' = 5
scala.util.Try("abc".toInt).isFailure = true | "abc".toIntOption = None
7 / 2 = 3 | 7 / 2.0 = 3.5 | 7 % 3 = 1 | -7 % 3 = -1 | Int.MaxValue + 1 = -2147483648 | 0.1 + 0.2 =
    0.30000000000000004
```

3.3-Q1 What are `7 / 2`, `7 / 2.0`, `-7 % 3` and `Int.MaxValue + 1`? [tricky]

3, 3.5, -1 and -2147483648 (the sign of `%` follows the dividend, and Int overflow wraps without an error). Use `Long` for large sums: Spark's `reduce(_ + _)` over 1..1,000,000 as `Int` gave 1784293664 instead of 500000500000.

3.3-Q2 Difference between `==` and `eq`; predict `new String("hi") == new String("hi")` and `... eq ...`. [tricky]

`true` and `false`. `==` compares contents; `eq` compares object identity. `Array(1,2) == Array(1,2)` is also `false` (arrays use reference equality); compare with `sameElements` or `.toList`.

3.3-Q3 What does `false && noisy(true)` do? [2 marks]

Nothing on the right-hand side: `&&` short-circuits, so `noisy` is never called. Likewise `true || ...`. The non-short-circuit operators `&` and `|` evaluate both sides.

3.3-Q4 Why is `1 + 2` described as a method call? [2 marks]

Every operator is a method of the left operand: `1.+(2)`. This is why you can define `+` on your own class and why `a max b` and `xs map f` work as infix notation.

3.4 Arrays, lists, tuples, sets, hash sets and maps

Type	Mutable?	Order / unique	Key facts
`Array[T]`	elements yes, size fixed	ordered, duplicates ok	JVM array, `a(i)` O(1); equality is by reference
`ArrayBuffer`, `ListBuffer`	yes	ordered	growable (`scala.collection.mutable`)
`List[T]`	no	ordered, duplicates ok	linked list; `::` prepend O(1); `a(i)` and `:+` O(n); `Nil` is empty
`Vector`	no	ordered	fast random access and updates (effectively O(1))
`Set[T]` / `HashSet`	no (mutable version exists)	unique, no guaranteed order	hash based; `&`, `|`, `--` (`intersect`, `union`, `diff`)
`Map[K, V]` / `HashMap`	no (mutable version exists)	unique keys	`get` returns `Option`; `m(k)` throws if missing; `+ (k -> v)` returns a new map
`(A, B, C)`	no	fixed size, mixed types	`t._1`, destructure `val (a, b) = t`

- Immutable collections never change: every "modification" returns a new collection, and `var l = List(1)` only rebinds the variable.
- A duplicate key in a Map overwrites the old value; hash collisions (different keys, same bucket) are resolved internally. Lab 15's "chaining" is a user-level `Map[K, List[V]]`.

Verified output (Labs 13 to 16)

```
sum = 147, joined = apple,banana,cherry,mango,orange
squares : List(4, 9, 25, 49, 121)
cons    : List(0, 2, 3, 5, 7, 11)
set and : List(d, l, o, r, w)
tuple   : 42, scala, true
grades  : List((Alice,95), (Bob,78))
merge   : List((Alice,92), (Bob,90), (Emma,88))
chained : List((Alice,List(85, 95)), (Bob,List(78)))
filter  : List(2, 4, 6, 8, 10)
map     : List(3, 6, 9, 12, 15, 18, 21, 24, 27, 30)
flatMap : List((1,1), (2,4), (3,9), (4,16))
foldLeft: 3628800
reduce  : 55
flatMap on strings: List(h, i, y, o)
```

Verified output

```
--- lists: :: is right-associative, immutability ---
1 :: 2 :: Nil = List(1, 2) | List(1,2) ::: List(3) = List(1, 2, 3) | List(1,2) :+ 3 = List(1, 2, 3) | 0 +: List(1) =
     List(0, 1)
sorted returns a new list: List(1, 2, 3), original still List(3, 1, 2) | head/tail: 3/List(1, 2) | last: 2
ListBuffer is mutable: ListBuffer(0, 1, 2, 3) | toList: List(0, 1, 2, 3)

--- more collection operations ---
partition: (List(3, 6, 9),List(1, 2, 4, 5, 7, 8, 10)) | groupBy(_ % 3): List((0,List(3, 6, 9)), (1,List(1, 4, 7,
    10)), (2,List(2, 5, 8)))
zip: List((1,a), (2,b)) | zipWithIndex: List((a,0), (b,1)) | sliding: List(List(1, 2), List(2, 3), List(3, 4))
take/drop/takeWhile: List(1, 2)List(9, 10)List(1, 2, 3) | distinct: List(1, 2) | sortWith: List(3, 2, 1)
1 to 5 = Range 1 to 5 | 1 until 5 = Range 1 until 5 | by 2: List(1, 5, 9)
mapValues-free word count: List((be,2), (not,1), (or,1), (to,2))
Set(1,2,2,3) = Set(1, 2, 3) | & : Set(2, 3) | | : Set(1, 2, 5) | -- : Set(2, 3)
Map + new pair, then + same key again (overwrites): Map(a -> 9, b -> 2)
Array elements are mutable, indexed with (): List(10, 2, 3) | ArrayBuffer can grow: ArrayBuffer(1, 2, 3)
```

3.4-Q1 Compare Array and List. [5 marks]

	Array	List
Mutability	elements can be changed	immutable
Size	fixed	any, but each change makes a new list
Access	`a(i)` O(1)	`l(i)` O(n)
Prepend	no	`x :: l` O(1)
Equality	reference (`Array(1,2) == Array(1,2)` false)	structural (true)

3.4-Q2 What is `1 :: 2 :: Nil`? Why is `::` written on the left of the list? [tricky]

`List(1, 2)`. Operators ending in a colon are right-associative and are methods of the *right* operand, so it parses as `1 :: (2 :: Nil)`, i.e. `Nil.::(2).::(1)`.

3.4-Q3 Create the squares of the first five primes, the unique characters of "hello world", and the intersection of two sets. [3 marks]

The first and third lines are run in the output above (m3_q1)

```
List(2, 3, 5, 7, 11).map(p => p * p)                // List(4, 9, 25, 49, 121)
"hello world".filter(_ != ' ').toSet           // Set of unique characters
("hello world".toSet & "world".toSet)                 // common characters
```

3.4-Q4 A `Map` holds duplicate-key writes. Show overwrite, merge-with-max and chaining. [5 marks]

`grades.put("Alice", 95)` replaces 85. Merge: for each key in `a.keySet ++ b.keySet` take `math.max(a.getOrElse(k, 0), b.getOrElse(k, 0))`. Chaining: `foldLeft(Map.empty[String, List[Int]])` appending to `acc.getOrElse(k, Nil)`. Output above: merge `(Alice,92), (Bob,90), (Emma,88)`, chained `(Alice, List(85, 95))`.

3.4-Q5 `m("z")` versus `m.get("z")` on a Map without "z". [2 marks]

`m("z")` throws `NoSuchElementException`; `m.get("z")` returns `None`; `m.getOrElse("z", 0)` returns a default.

3.4-Q6 `val l = ListBuffer(1, 2); l += 3; l.prepend(0)`: is that allowed with `val`? [tricky]

Yes. The reference `l` is fixed but the buffer is mutable (output: `ListBuffer(0, 1, 2, 3)`). The same trick with an immutable `List` would not compile.

3.5 Functional combinators

- `map` transforms each element; `filter` keeps matches; `flatMap` maps then flattens one level; `foldLeft(z)(f)` / `foldRight(z)(f)` accumulate with a start value; `reduce` folds without a start value (fails on empty); `scanLeft` keeps the intermediate results.
- Also: `groupBy`, `partition`, `zip`, `zipWithIndex`, `take/drop/takeWhile`, `sortBy/sortWith`, `distinct`, `sliding`, `exists/forall/count/find`.
- `for (x <- xs if cond) yield f(x)` is sugar for `filter`/`map`/`flatMap`.
- **Fold direction matters** unless the operation is associative *and* commutative.

Verified output

```
--- folds: direction matters for non-commutative operations ---
foldLeft(0)(_ - _)  = -10   ((((0-1)-2)-3)-4)
foldRight(0)(_ - _) = -2    (1-(2-(3-(4-0))))
reverse via foldLeft  : List(4, 3, 2, 1)
copy via foldRight    : List(1, 2, 3, 4)
reduce(_ - _) = -8 | scanLeft(0)(_ + _) = List(0, 1, 3, 6, 10)
reduce on empty list: UnsupportedOperationException | fold(0) on empty: 0
```

3.5-Q1 Compute `List(1,2,3,4).foldLeft(0)(_ - _)` and `foldRight(0)(_ - _)`. [tricky]

foldLeft: ((((0-1)-2)-3)-4) = -10. foldRight: 1-(2-(3-(4-0))) = -2. Subtraction is not associative, so the direction changes the answer (output).

3.5-Q2 Reverse a list with `foldLeft`; copy it with `foldRight`. [3 marks]

`foldLeft(List.empty[Int])((acc, x) => x :: acc)` gives `List(4,3,2,1)`; `foldRight(List.empty[Int])((x, acc) => x :: acc)` gives `List(1,2,3,4)` (output).

3.5-Q3 Difference between `map` and `flatMap` with `List("hi", "yo")` and `_.toList`. [3 marks]

`map(_.toList)` gives `List(List(h,i), List(y,o))`; `flatMap(_.toList)` flattens to `List(h, i, y, o)`. `flatMap` also lets you chain `Option`s and generators in a for-comprehension.

3.5-Q4 Trace `reduce`, `fold` and `scanLeft` on an empty list. [tricky]

`List.empty[Int].reduce(_ + _)` throws `UnsupportedOperationException`; `fold(0)(_ + _)` returns 0; use `reduceOption` or `fold` for possibly-empty data.

3.5-Q5 Create a list of the integers 1 to 10, filter evens, multiply by 3, build (n, n*n) tuples with flatMap, and the product with foldLeft. [5 marks]

`xs.filter(_ % 2 == 0)`, `xs.map(_ * 3)`, `xs.flatMap(n => List((n, n * n)))`, `xs.foldLeft(1L)(_ * _)` = 3628800 (use `Long`; 13! already overflows `Int`). Output in 3.4.

3.5-Q6 What does `for { i <- 1 to 3; j <- 1 to 2 if i != j } yield (i, j)` produce? [predict the output]

`List((1,2), (2,1), (3,1), (3,2))`: nested generators with a guard (output).

3.6 Option and pattern matching

- `Option[A]` is `Some(a)` or `None`; it replaces `null`. `map`, `flatMap`, `filter`, `getOrElse`, `fold`; a for-comprehension over Options gives `None` as soon as one is `None`.
- `match` tests patterns top to bottom: constants, variables, types (`case n: Int`), tuples, lists (`head :: tail`, `Nil`), case classes, guards (`if`), alternatives (`|`), binders (`x @ pattern`), wildcard _. The first match wins.
- On a `sealed` trait the compiler warns when a case is missing (exhaustiveness).

Verified output

```
--- Option instead of null ---
m.get("a") = Some(1) | m.get("z") = None | m("a") = 1 | m("z") throws NoSuchElementException
chain: Some(4).map(_ * 2).filter(_ > 5) = Some(8) | Some(1).flatMap(x => None: Option[Int]) = None |
    None.getOrElse(0) = 0
for-comprehension over Options: Some(6) | with a None inside: None
for over lists: List((1,2), (2,1), (3,1), (3,2))

--- pattern matching ---
  zero
  negative int -5
  int 7
  string of length 5
  pair 1,2
  empty list
  list head=1 tail=List(2, 3)
  some(x)
  something else
binder @ and varargs pattern: whole=List(9, 8, 7) first=9
tuple destructuring: q=3 r=2
```

3.6-Q1 Why prefer `Option` to `null`? [3 marks]

The possible absence is in the type, so the compiler makes you handle it; combinators (`map`, `getOrElse`) remove `if (x != null)` checks and `NullPointerException`s.

3.6-Q2 Predict `describe` for `0`, `-5`, `"scala"`, `(1,2)`, `Nil`, `List(1,2,3)`. [predict the output]

`zero`, `negative int -5`, `string of length 5`, `pair 1,2`, `empty list`, `list head=1 tail=List(2, 3)` (output). Order of cases matters: `case 0` must come before `case n: Int`.

3.6-Q3 What does `val whole @ List(first, _*) = List(9, 8, 7)` bind? [tricky]

`whole = List(9, 8, 7)` and `first = 9`; `@` names the whole matched value and `_*` ignores the rest.

3.6-Q4 `Some(2)` and `Option.empty[Int]` in a for-comprehension: result? [2 marks]

`None`. With `Some(2)` and `Some(3)` the result is `Some(6)`; a single `None` makes the whole expression `None`.

3.7 Classes, objects, constructors, this and method overloading

- `class Point(val x: Int, var y: Int, tag: String)`: the class header is the **primary constructor**. `val` makes a read-only field, `var` a read/write field, a plain parameter is just a constructor parameter (private to the class if used inside).
- **Auxiliary constructors** are `def this(...)` and their first statement must call another constructor: `def this(x: Int) = this(x, 0, "aux")`.
- `this` is the current object; return `this` (type `this.type`) to chain calls. Use `this.y = y` when a parameter hides a field.
- **Overloading**: same name, different parameter lists, chosen at compile time. **Overriding**: a subclass replaces an inherited member (`override`), chosen at run time.
- Instances are created with `new`, or without `new` if a companion `apply` exists (case classes).

Verified output

```
--- initialisation order ---
  Base constructor sees name = null | name.length -> NullPointerException
  after construction: d.name = derived
  SafeBase constructor sees name = derived

--- constructors ---
  Point(1,5,t) | aux constructor: Point(7,0,aux) | chaining: Point(1,1,t)
  p.x = 1 (val is a read-only field); p.tag does not compile: tag is just a constructor parameter
```

Verified output (question bank M3-C2)

```
  note: salary
Account(Asha, 750.00)
A dog makes the sound 'woof' | kind = animal
Circle area = 7.07
Rect area = 6.00
protected, public, private
Welcome to PDS Demo
Rohan, Meena, created = 2
anonymous: makes the sound 'meow'
```

3.7-Q1 Primary versus auxiliary constructor; what is the rule for the auxiliary one? [3 marks]

The primary constructor is the class header and defines the fields. An auxiliary constructor (`def this`) must begin with a call to a previously defined constructor. This chains all constructors back to the primary one.

3.7-Q2 What is the difference between `class P(x: Int)`, `class P(val x: Int)` and `class P(var x: Int)`? [tricky]

Plain: no accessor (`p.x` does not compile). `val`: a public read-only field. `var`: a public field with a setter.

3.7-Q3 What is printed when `Derived` overrides a `val` that `Base`'s constructor uses? [tricky]

`Base constructor sees name = null` and `name.length` throws `NullPointerException`. The superclass constructor runs before the subclass field is initialised, so the overriding `val` is still `null` (0 for numbers). Fixes: make the member a `def`, a `lazy val`, or avoid using it in the constructor (output shows `SafeBase` seeing "derived").

3.7-Q4 Show method overloading and say how it differs from overriding. [3 marks]

Account in m3_q2_oop.scala

```
def deposit(amount: Double): Unit = { balance += amount }
def deposit(amount: Double, note: String): Unit = { balance += amount; println(note) }
```

3.7-Q5 What does returning `this.type` allow? [2 marks]

Method chaining: `p.moveTo(0, 0).moveTo(1, 1)` (output: `Point(1,1,t)`). Each call returns the same object.

3.8 Singleton, anonymous and companion objects

- `object Config { ... }` is a **singleton**: one instance, created lazily on first use, no `new`. `object Main extends App` (or `def main`) is the program entry point.
- A **companion object** has the same name as a class and sits in the same file. Each can access the other's private members. It holds `apply` factories (so `Student("A", 1)` works without `new`) and static-like helpers.

- An **anonymous class** is a one-off subclass created inline: `new Animal { override val sound = "meow" }`.
- `unapply` (in a companion/extractor object) powers pattern matching; case classes generate both `apply` and `unapply`.

Verified output

```
--- extractors and companions ---
  List(even, half is 5, odd 7)
  Secret(21) code = 42 (companion may call the private constructor; `new Secret(1)` outside does not compile)
```

3.8-Q1 How does a companion object remove the need for `new`? [5 marks]

The compiler rewrites `Student("Rohan", 101)` to `Student.apply("Rohan", 101)`, defined in the companion. The companion can call a private constructor, validate input, count instances (`created = 2` in the question-bank output) or return a subtype.

3.8-Q2 What is the difference between a singleton object and a class? [3 marks]

A class is a blueprint of which many instances can exist; an `object` is a single, ready-made instance, used for utilities, constants and `main`. There is no `static` keyword in Scala; companion objects play that role.

3.8-Q3 What is `Secret.peek(Secret(21))` in the sample and why can it be called? [tricky]

42: the companion `apply` doubles the code and `peek` reads the private constructor's field. A companion may access the class's private members; outside code cannot call `new Secret(1)`.

3.8-Q4 What is an extractor? [2 marks]

An object with `unapply` that tells `match` how to deconstruct a value. Output: `Even.unapply` made `10` match `case Even(h)` with half 5 while 7 fell through to `odd 7`.

3.9 Case classes

- `case class Person(name: String, age: Int, city: String)`: constructor parameters become public `val` fields; the compiler adds `apply` (no `new`), `unapply` (pattern matching), structural `equals` and `hashCode`, readable `toString`, and `copy`.
- Meant for immutable data records. Combine with a `sealed trait` and `match` for algebraic data types; `case object` for singletons/enumerations.
- `copy` is **shallow**; an `Array` field is compared by reference, so two otherwise equal case classes holding arrays are not `==`.

Verified output (Lab 17 operations, then pattern matching)

```
toString : Person(Asha,22,Chennai)
equals   : true  (structural)   eq: false  (reference)
hashCode : true
copy     : Person(Asha,22,Pune)
unapply  : name=Asha age=22
older than 30  : List(Ravi, Meena, Sneha)
grouped by city: List((Chennai,List(Asha, Meena)), (Delhi,List(Ravi, Sneha)), (Pune,List(Kabir)))
as strings     : List(Asha is 22 years old and lives in Chennai, Ravi is 34 years old and lives in Delhi)
sorted by age  : List(Asha, Kabir, Sneha, Ravi, Meena)
  Large transaction T1 logged
  Standard transaction T2 approved
  Alert: Transaction T3 failed
  Unknown transaction state
```

Verified output

```
--- sealed + case classes ---
  areas: List(3.14, 6.00, 0.00)
  case object equality: true | toString: Dot

--- case class: equality, copy, arrays inside ---
  c1 == c2: true | c1 eq c2: false | hashCode equal: true
  Team(A, Array(..)) == same data: false (the Array field is compared by reference)
  copy is shallow, shares the array: true
```

3.9-Q1 List four capabilities that distinguish a case class from an ordinary class. [5 marks]

(1) public immutable fields from constructor parameters, (2) companion `apply` so no `new`, (3) structural `equals`/`hashCode`/`toString`, (4) `copy` with changed fields, and `unapply` for pattern matching.

3.9-Q2 `Team("A", Array("x","y")) == Team("A", Array("x","y"))`: true or false? [tricky]

False. The two arrays are different objects and arrays use reference equality. Use a `List` or `Vector` field for structural equality.

3.9-Q3 What does `p1.copy(city = "Pune")` do, and is it a deep copy? [3 marks]

It makes a new object with `city` replaced and the other fields reused; nested mutable objects are shared (output: the copied `Team` shares the same array).

3.9-Q4 How does a `sealed trait` help `match`? [3 marks]

All subclasses must be in the same file, so the compiler knows every case. A `match` that forgets `Dot` produces a warning. The `area` function covers `Circle`, `Rect` and `Dot`.

3.9-Q5 Write the Lab 17 filter, group, format and sort operations. [5 marks]

`people.filter(_.age > 30)`, `people.groupBy(_.city)`, `people.map(p => s"${p.name} is ${p.age} years old and lives in ${p.city}")`, `people.sortBy(_.age)`. Output in the question bank (M3-C3) and above.

3.10 Inheritance, field overriding, final and abstract classes

- `class Dog extends Animal`. A subclass inherits members; `override` is **required** to replace a concrete member; `super.method()` calls the parent version.
- Fields can be overridden too: `override val sound = "woof"`. A `val` can override a parameterless `def`, but a `def` cannot override a `val`.
- `final` on a member forbids overriding; `final class` forbids extending; `sealed` restricts subclasses to the same file.
- **Abstract class**: may have abstract members (no body) and constructor parameters; cannot be instantiated. A concrete subclass implements them (`override` optional).
- Dynamic dispatch: the method run is chosen by the object's real class, not the variable's declared type.

Verified output

```
--- overriding and super ---
  Rex says woof | Bit says woof (squeaky)
  static type Animal, dynamic dispatch picks Puppy.sound: woof (squeaky) | isInstanceOf[Dog]: true
```

3.10-Q1 `val a: Animal = new Puppy("Bit"); a.sound`: which version runs? [3 marks]

`Puppy.sound`, giving `woof (squeaky)`. The declared type decides what you may call; the runtime class decides which implementation runs.

3.10-Q2 What happens if you override a concrete method without `override`, or override a `final` one? [2 marks]

Both are compile-time errors ("needs `override` modifier", "cannot override final member").

3.10-Q3 Abstract class versus concrete class; can an abstract class have constructor parameters? [3 marks]

Yes. `abstract class Animal(val name: String) { def sound: String }`. It cannot be instantiated with `new` (except as an anonymous subclass); subclasses supply the abstract members.

3.10-Q4 Explain field overriding with a short example. [3 marks]

m3_q2_oop.scala

```scala
class Animal { val sound: String = "..." }
class Dog extends Animal { override val sound: String = "woof" }
```

3.11 Traits, mixins and linearization (tricky topic)

- A **trait** is like an interface that may also hold concrete methods and fields. A class can mix in many traits: `class Worker extends Logger with Auditor`; a single instance can too: `new Worker("K") with Notifier`.
- Scala 2 traits cannot take constructor parameters (Scala 3 allows them). Abstract classes can, and a class extends only one class.
- **Linearization** solves the diamond problem: the parent chain is flattened to one line, so `super` always means exactly one thing. For `class D extends B with C` (both extend `A`): L(D) = D, L(C) then L(B) with duplicates removed leaving the later occurrence = **D, C, B, A**. `super.who` in D runs C first, then B, then A.
- **The order of `with` matters**: the right-most trait is consulted first.

Verified output (question bank M3-C3/C4)

```
  [LOG] Asha started
  [AUDIT] Asha finished
linearization of D extends B with C: D > C > B > A
  [LOG] Kabir started
  [AUDIT] Kabir finished
  [NOTIFY] shift over
```

Verified output

```
--- trait mixin order changes behaviour ---
  Loud with Polite : HELLO ASHA, please   (Polite runs first, Loud wraps its result)
  Polite with Loud : HELLO ASHA, PLEASE   (Loud runs first, ', please' is added after upper-casing)
```

3.11-Q1 What does `(new D).who` print for `trait A {def who="A"}`, `trait B extends A {override def who="B > "+super.who}`, `trait C extends A {... "C > "...}`, `class D extends B with C {... "D > "...}`? [predict the output]

`D > C > B > A`. L(D) = D, C, B, A (output). Each `super.who` moves one step to the right along that line.

3.11-Q2 Show that the order of mixins changes the result: `Greeter with Loud with Polite` versus `Greeter with Polite with Loud`. [tricky]

The first gives `HELLO ASHA, please` (Polite is outermost: Loud upper-cases first, then Polite appends); the second gives `HELLO ASHA, PLEASE` (Loud is outermost and upper-cases everything Polite produced). Output above.

3.11-Q3 Trait versus abstract class. [3 marks]

	Trait	Abstract class
Inheritance	many per class (mixin)	one per class
Constructor parameters	Scala 2: no; Scala 3: yes	yes
Java interop	compiles to an interface	abstract class
Use	reusable capabilities (logging)	a base type with state and a constructor

3.11-Q4 Compare linearization in Scala with Python's MRO. [tricky]

Both flatten multiple inheritance into one order so `super` is unambiguous. Python (C3) gives `D > B > C > A` for `class D(B, C)` with the leftmost parent first; Scala mixes right to left, so `extends B with C` gives `D, C, B, A`. In both, the shared base `A` appears once, last.

3.11-Q5 Can a trait have a concrete method and a field? [2 marks]

Yes: `trait Logger { def log(m: String): Unit = println(m) }`, plus abstract members that the class must define. `Worker` in the output uses `log` and `audit` from two traits.

3.12 Access modifiers

Modifier	Visible in
(none)	public: everywhere (the default)
`protected`	the class and its subclasses (not the whole package, unlike Java)
`private`	the class itself and its companion object
`private[this]`	only the same instance
`private[pkg]`	the named enclosing package or class

Verified output (Base and Child classes: `protected, public, private` is printed by `c.peek`, `c.open`, `c.reveal`)

```
protected, public, private
```

3.12-Q1 What is the default access level, and how does `protected` differ from Java? [2 marks]

Public. Scala's `protected` allows subclasses only; Java also allows the same package.

3.12-Q2 Can the companion object read a `private` field of its class? [2 marks]

Yes, and vice versa; this is why a companion `apply` can call a `private` constructor.

3.12-Q3 A subclass tries `c.secret` where `secret` is `private` in `Base`. Result? [2 marks]

Compile error: value secret in class Base cannot be accessed. Through the public method `reveal` the value is reachable (output `private`).

3.13 Scala arrays and the REPL

- Arrays are fixed-size JVM arrays: `val a = Array(2, 5, 9)`, index with **parentheses** `a(0)`, update `a(0) = 10` (allowed even for a `val`). Iterate with `for (x <- a)` or `a.foreach(println)`; `a.sum`, `a.mkString(",")`, `a.map`, `a.filter`. `Array.fill(3)(0)`, `Array.ofDim[Int](2, 3)`.
- Equality is by reference; compare with `sameElements`. Use `ArrayBuffer` when the size must change.
- **REPL** (Read-Eval-Print Loop): start with `scala` (or `scala-cli repl`). Each expression is evaluated and printed with a generated name (`res0: Int = 5`). Commands: `:help`, `:paste` (multi-line, finish with Ctrl-D), `:type expr`, `:load file`, `:reset`, `:quit`. (REPL sessions were not recorded in this document; the array operations below were run as a compiled program.)

Verified output

```
sum = 147, joined = apple,banana,cherry,mango,orange
```

Verified output

```
--- equality: == vs eq, arrays ---
List(1,2) == List(1,2): true | Array(1,2) == Array(1,2): false (arrays compare by reference!)
Array(1,2).sameElements(Array(1,2)): true | Array(1,2).toList == Array(1,2).toList: true
strings: s1 == s2 is true, s1 eq s2 is false
Set(1,2) == Set(2,1): true | Map(1->2) == Map(1->2): true | (1,2) == (1,2): true
```

3.13-Q1 Create an array 2, 5, 9, 14, 20; write a function returning its sum; join a fruit array with commas; print each element. [5 marks]

`val nums = Array(2, 5, 9, 14, 20)`, `def sum(a: Array[Int]) = a.sum`, `fruits.mkString(",")`, `for (n <- nums) println(n)`. Output (Lab 13): `sum = 147` after the program changed `nums(0)` to 99, `joined = apple,banana,cherry,mango,orange`.

3.13-Q2 Why does `Array(1,2) == Array(1,2)` give false, and what is the fix? [tricky]

Arrays use Java reference equality. Use `a.sameElements(b)` or compare `a.toList == b.toList` (both true in the output).

3.13-Q3 What is the REPL used for, and how do you enter a multi-line definition? [2 marks]

Quick experiments with expressions, collections and library calls without a project. Type `:paste`, paste the code, press Ctrl-D.

Module 4: Scala libraries and Spark basics

Syllabus (8 hours): Scala libraries: Breeze, Saddle; exception handling; Apache Spark architecture; Spark and big data; Apache Spark components.

Where each syllabus topic is covered

Syllabus topic	Section
Breeze	4.1
Saddle (and Spire from the labs)	4.2
Exception handling	4.3
Spark big data	4.4
Apache Spark architecture	4.5
RDDs, transformations, actions (core of Spark)	4.6, 4.7
DataFrames, Datasets, Catalyst	4.8
Apache Spark components	4.9

4.1 Breeze (numerical computing)

- Breeze (`org.scalanlp::breeze`) is Scala's numerical library: `DenseVector`, `DenseMatrix`, linear algebra (`inv`, `det`, `eigSym`, `svd`, solve with `\`), statistics (`breeze.stats.mean/variance`) and optimisation.
- Operators: `+ -` element-wise; `*:*` **element-wise product**; `*` is the **matrix product** for matrices (and scalar scaling for vectors); `dot` inner product; `.t` transpose; `A \ b` solves A x = b.
- Indexing is 0-based with parentheses: `v(0 to 1)`, `m(::, 0)` (a column), `m(1, ::)` (a row). Broadcasting uses `*`: `m(*, ::) - rowVector`.
- `variance` is the **sample** variance (divides by n - 1), like `np.var(ddof=1)`.
- Dependency (sbt): `"org.scalanlp" %% "breeze" % "2.1.0"` (tested version).

Verified output

```
--- Breeze: vectors, matrices, slicing, broadcasting ---
  v + w = DenseVector(5.0, 7.0, 9.0) | v *:* w = DenseVector(4.0, 10.0, 18.0) | v dot w = 32.0 | v * 2.0 =
      DenseVector(2.0, 4.0, 6.0) | sum(v) = 6.0
trying to return java 16 instance
return java 11 instance
  m * m (matrix product) = 7.0   10.0   ; 15.0  22.0   | m *:* m (element-wise) = 1.0  4.0    ; 9.0  16.0
  m.t = 1.0  3.0   ; 2.0  4.0   | det = -2.0 | inv = -1.9999999999999996  0.9999999999999998    ; 1.4999999999999998
         -0.4999999999999999
  slicing: v(0 to 1) = DenseVector(1.0, 2.0) | m(::, 0) = DenseVector(1.0, 3.0) | m(1, ::) = DenseVector(3.0, 4.0) |
       m(0, 1) = 2.0
  broadcasting: subtract column means = -1.0  -10.0   ; 0.0   0.0     ; 1.0   10.0
  indexing is 0-based and uses (), like Scala arrays; v(5) throws IndexOutOfBoundsException

--- PCA in Breeze: eigen-decomposition of the covariance matrix (same 5 points as the Python notes) ---
  covariance = 2.5   2.25   ; 2.25  4.0
  eigenvalues (ascending) = DenseVector(0.8782917548737156, 5.6217082451262845) | explained ratio of the largest =
      0.8649
  eigenvectors (columns) = -0.8112421851755608  0.584710284663765    ; 0.584710284663765    0.8112421851755608
```

Lab 19 revenue calculation (question bank M4-C2)

```
Apple   2.50 x 150 = 375.00
Banana  1.20 x 300 = 360.00
Cherry  4.00 x 80 = 320.00
Mango   3.50 x 110 = 385.00
Orange  2.00 x 200 = 400.00
total revenue = 1840.00, mean price = 2.640, variance = 1.2730
A*x = b  ->  x = DenseVector(0.8, 1.4)
A * A^T  = 5.0  5.0
5.0  10.0
dot      = 1840.0
Double 0.1 + 0.2 = 0.30000000000000004   | Rational 1/10 + 2/10 = 3/10  (exactly 3/10: true)
```

4.1-Q1 Compute the total revenue per fruit with Breeze (price times quantity). [3 marks]

Lab 19, run in the question bank: Apple 375.00 ... Orange 400.00, total 1840.00

```
val revenue = price *:* qty        // element-wise product
sum(revenue)
```

4.1-Q2 Difference between `*:*` and `*` in Breeze. [tricky]

`*:*` multiplies element by element (matrix `m *:* m` gives 1, 4, 9, 16 in the output); `*` on two matrices is the matrix product (7, 10, 15, 22). `dot` on vectors gives the scalar inner product.

4.1-Q3 Solve 2x + y = 3, x + 3y = 5 with Breeze. [3 marks]

`DenseMatrix((2.0, 1.0), (1.0, 3.0)) \ DenseVector(3.0, 5.0)` gives x = 0.8, y = 1.4 (Lab 19 run).

4.1-Q4 How do you do PCA in Breeze? [tricky]

Centre the columns, form the covariance `(Xc.t * Xc) / (n - 1.0)`, and call `eigSym`. The output reproduces the Python result: eigenvalues 0.8783 and 5.6217 (ascending order in Breeze), largest-variance ratio 0.8649. Breeze returns eigenvalues in ascending order, NumPy's `eigh` too, so reverse them before reading PC1.

4.2 Saddle (data frames) and Spire (exact numerics)

- Saddle gives indexed, array-backed data structures: `Vec` (1-D), `Index` (labels), `Series` (Vec + Index), `Frame` (columns of aligned Series), with automatic alignment on labels and NA handling (NaN for Double). It is Scala's small pandas.
- Workflow of Lab 18: build a Frame, filter rows with a boolean mask, take the column, compute `mean`; charts with Vegas are declarative (data + encodings + mark) and need a notebook/browser.
- The original `org.scala-saddle` artifacts on Maven Central stop at Scala 2.11; the tested program uses the maintained fork `io.github.pityka::saddle-core:4.0.0-M14` on Scala 2.13 (needs `import org.saddle.ops.BinOps._` for `Vec * Vec`).
- **Spire** provides generic and exact numeric types: `Rational`, `Algebraic`, `Complex`, and type classes. Use it when floating-point error is unacceptable.

Verified output (Saddle)

```
apple prices     : 2.4, 2.65, 2.5, 3.1
average apple    : 2.6625
Series at 2023-10-02:
[2 x 1]
2023-10-02 -> 2.6500
2023-10-02 -> 1.8000

frame shape      : 6 rows x 2 cols
revenue column   : 288.0, 291.5, 350.0, 300.0, 324.0, 403.0
mean with NA     : 3.0  (NA skipped), count = 2
```

Verified output (last line: Spire)

```
Double 0.1 + 0.2 = 0.30000000000000004   | Rational 1/10 + 2/10 = 3/10  (exactly 3/10: true)
```

4.2-Q1 Filter the apple rows and compute the average apple price in Saddle. [5 marks]

Build a boolean `Vec` with `fruit.map(_ == "Apple")`, take its positions with `find(identity)`, then `price.take(positions).mean`. Output: prices 2.4, 2.65, 2.5, 3.1; average 2.6625.

4.2-Q2 How does Saddle treat missing values? [2 marks]

As NA (NaN for doubles). Statistics skip them: the mean of `Vec(2.0, NaN, 4.0)` is 3.0 and the count is 2 (output).

4.2-Q3 Why use Spire's `Rational` instead of `Double`? [3 marks]

`0.1 + 0.2` in `Double` is `0.30000000000000004`; `Rational(1,10) + Rational(2,10)` equals exactly `3/10` (output). Rationals are exact but slower.

4.2-Q4 Compare Breeze, Saddle and Spire. [3 marks]

	Breeze	Saddle	Spire
Purpose	linear algebra, statistics	labelled data tables, NA handling	exact, generic numbers
Main types	`DenseVector`, `DenseMatrix`	`Vec`, `Series`, `Frame`	`Rational`, `Algebraic`
Python analogue	NumPy/SciPy	pandas	`fractions`, `decimal`

4.3 Exception handling in Scala

- `try { ... } catch { case e: IOException => ... } finally { ... }`. `try` is an **expression** with a value; `catch` takes pattern-matching cases; the first matching case wins; **no checked exceptions**.
- Custom exception: `class MyEx(msg: String) extends Exception(msg)`; `throw new MyEx("...")` has type `Nothing`.
- Functional style: `Option` (absence), `Try` (`Success`/`Failure`, wraps code that may throw; `map`, `flatMap`, `recover`, `recoverWith`, `getOrElse`, `toOption`), `Either` (`Left` = error, `Right` = value; the error can carry information).
- `scala.util.control.NonFatal` matches ordinary exceptions but not fatal ones (`OutOfMemoryError`, `StackOverflowError`); never swallow fatal errors. `Try` does not catch them.
- `scala.util.Using` closes resources automatically (like Java try-with-resources or Python `with`).

Verified output

```
--- try/catch/finally as an expression ---
  finally ran
  finally ran
f(2) = 5 | f(0) = -1
  finally before the value is returned
from try

--- catch order and NonFatal ---
  IllegalArgument: bad arg | IllegalArgument: For input string: "x" | NonFatal: IOException    (case 1:
      NumberFormatException is a subclass of IllegalArgumentException, so the FIRST handler wins)
  NonFatal(IOException) = true | NonFatal(StackOverflowError) = false | NonFatal(OutOfMemoryError) = false  (never
      swallow fatal errors; Try does not catch them)

--- Try: map/flatMap/recover/recoverWith/toOption/fold ---
  '4' -> Success(0.25) | getOrElse(-1.0) = 0.25 | recover = Success(0.25)
  '0' -> Failure(java.lang.ArithmeticException: zero) | getOrElse(-1.0) = -1.0 | recover = Success(Infinity)
  'x' -> Failure(java.lang.NumberFormatException: For input string: "x") | getOrElse(-1.0) = -1.0 | recover =
      Success(NaN)
  for-comprehension stops at the first failure: Failure(java.lang.NumberFormatException: For input string: "oops")
  fold: got 5
  a Failure is only a value; calling .get on it re-throws the stored exception: NumberFormatException

--- Either: errors with information ---
  List(Right(21), Left(negative age), Left('abc' is not a number))
  fold: error: negative age | partitionMap: (List(negative age, 'abc' is not a number),List(21, 40))
  sequence-like: all valid? true | first error: Some('x' is not a number)

--- Using: automatic resource closing ---
  closing file1
  Success(data from file1)
  closing file2
  with a failure inside, the resource still closes and the result is: Failure(java.lang.RuntimeException: boom)
  closing B
  closing A
  two resources close in reverse order: Success(data from A+data from B)
```

Question bank M4-C1 program output

```
  finally runs for divide(10, 2)
divide(10, 2) = 5
  caught ArithmeticException
  finally runs for divide(10, 0)
divide(10, 0) = 0
custom: balance 100.0 < 250.0
Option  : Some(98400) | None | n/a
Try     : '42' -> Success(42)
Try     : '4x2' -> Failure(NumberFormatException)
recover : -1
chain   : Some(40)
Either  : Right(4.0) | Left(cannot take sqrt of -4.0)
compose : Right(2.0)
```

4.3-Q1 What is printed first for `f(2)`: the `finally` message or the returned value, and what does `f(0)` return? [predict the output]

The `finally` message prints first, then the caller prints `f(2) = 5`; `f(0)` catches `ArithmeticException` and returns -1 (output). `finally` runs before the value is handed back.

4.3-Q2 `NumberFormatException` is a subclass of `IllegalArgumentException`. In which order should the cases be written? [tricky]

Most specific first. If `case e: IllegalArgumentException` comes before `case e: NumberFormatException` the latter is unreachable (the output shows `"x"` handled by the first handler).

4.3-Q3 Differentiate `Option`, `Try` and `Either`. [5 marks]

	Cases	Use
`Option`	`Some` / `None`	absence only
`Try`	`Success` / `Failure(ex)`	code that may throw
`Either`	`Right(v)` / `Left(err)`	typed, informative errors

4.3-Q4 What does a for-comprehension over `Try` do when the second step fails? [3 marks]

It stops and returns that `Failure`; later steps do not run (output: `Failure(NumberFormatException: For input string: "oops")`).

4.3-Q5 Does `Try(parse("x").get)` throw? [tricky]

`parse("x")` is only a `Failure` value; calling `.get` on a `Failure` re-throws the stored exception, and the outer `Try` captures it (a `NumberFormatException`). Avoid `.get`; use `getOrElse`, `fold`, `recover`.

4.3-Q6 Why is catching `Throwable` or `Exception` blindly a bad idea? [3 marks]

It hides programming errors and (for Throwable) fatal JVM errors. Catch specific exceptions or use `NonFatal`; `NonFatal(StackOverflowError)` is false (output).

4.4 Big data and Apache Spark: why Spark

- Big data: characterised by volume, velocity, variety (and veracity, value). One machine cannot store or process it fast enough, so work is split over a cluster.
- **Hadoop MapReduce** writes intermediate results to disk between map and reduce steps. **Spark** plans a DAG of operations and keeps intermediate data in memory where possible; it also unifies batch, SQL, streaming, machine learning and graphs.
- Spark is written in Scala and runs on the JVM; APIs exist for Scala, Java, Python (PySpark), R and SQL. Main abstractions: RDD (low level), DataFrame/Dataset (structured, optimised).
- Modes: local (one JVM, used for the tests in these notes), standalone, YARN, Kubernetes. The current Spark release line is 4.x; the labs and textbooks use 3.x, and the concepts here are the same.

4.4-Q1 Why is Spark often faster than Hadoop MapReduce? [3 marks]

It avoids writing intermediate data to disk by caching in memory, optimises the whole job as a DAG, and reuses datasets in iterative algorithms (machine learning) instead of re-reading from disk each pass.

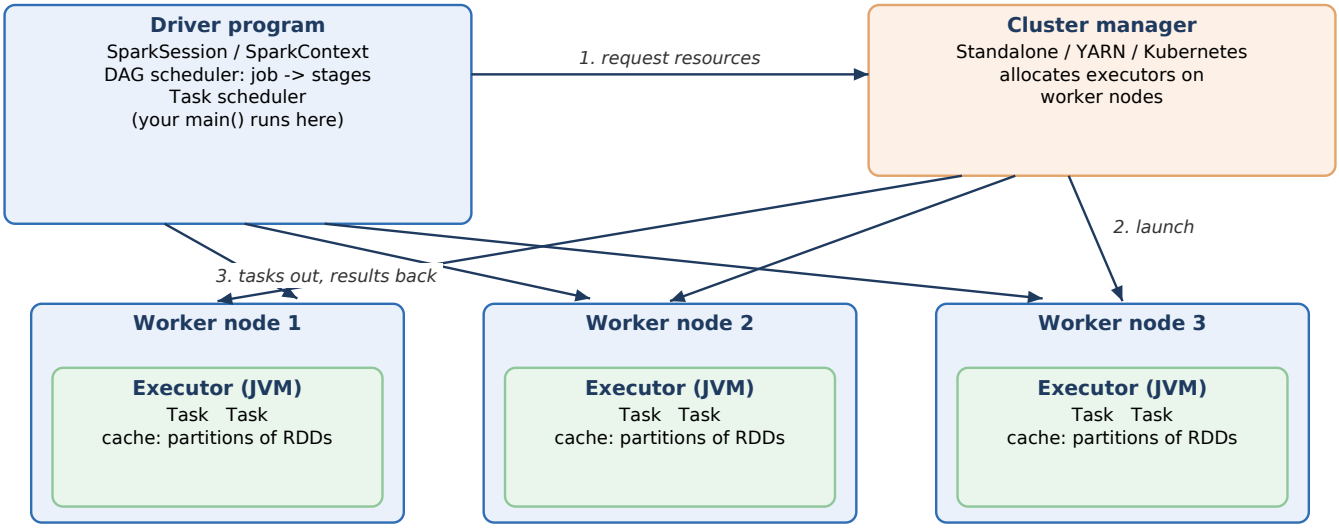
4.4-Q2 What are the V's of big data? [2 marks]

Volume (size), Velocity (arrival speed), Variety (structured, semi-structured, unstructured); often Veracity (quality) and Value.

4.4-Q3 Why is Scala a natural fit for Spark? [2 marks]

Spark itself is written in Scala, the Scala API is the most complete and has no cross-language serialisation overhead; Dataset typed operations and Catalyst use Scala features such as case classes and pattern matching.

4.5 Spark architecture



Driver, cluster manager and executors.

- **Driver**: runs your `main`, holds the `SparkSession`/`SparkContext`, converts actions into jobs, stages and tasks (DAG scheduler), schedules tasks, collects results. It must stay alive for the whole application.
- **Cluster manager**: allocates resources (Standalone, YARN, Kubernetes). **Worker nodes** run **executors** (JVM processes) that execute tasks and cache partitions.
- **Application > job > stage > task**: each action creates a job; a job is cut into stages at shuffle boundaries; a stage has one task per partition.
- **Partition** = slice of a dataset, the unit of parallelism. A task processes one partition.

Verified output (local[4])

```
--- partitions: the unit of parallelism ---
glom shows the contents of each partition: List(List(1, 2, 3), List(4, 5, 6), List(7, 8, 9), List(10, 11, 12))
defaultParallelism (cores in local[4]) = 4 | spark.sql.shuffle.partitions = 200
repartition(2) -> 2 (full shuffle, can increase) | coalesce(2) -> 2 (no shuffle, only decreases)
coalesce(8) cannot increase: 4
```

4.5-Q1 Draw and explain Spark's architecture. [10 marks]

The diagram above: the driver talks to the cluster manager to obtain executors, then sends tasks directly to the executors and receives results. Executors hold cached data. Explain each box in one sentence and mention the DAG scheduler.

4.5-Q2 What is the difference between a job, a stage and a task? [3 marks]

Job: triggered by one action. Stage: a set of operations that run without a shuffle. Task: one stage on one partition; with 4 partitions a stage runs 4 tasks (the output shows `parallelize(1 to 12, 4)` split into four partitions of three numbers).

4.5-Q3 `repartition(2)` versus `coalesce(2)`; can `coalesce(8)` increase partitions? [tricky]

`repartition` does a full shuffle and can increase or decrease the count; `coalesce` merges partitions without a full shuffle and can only decrease. `coalesce(8)` on 4 partitions stays 4 (output).

4.5-Q4 What does `spark.sql.shuffle.partitions = 200` mean? [3 marks]

After a shuffle in the DataFrame API (join, groupBy) the result has 200 partitions by default, even for tiny data; tune it, or let Adaptive Query Execution merge small partitions.

4.5-Q5 Client mode versus cluster mode. [3 marks]

In client mode the driver runs on the machine that submitted the application (convenient for interactive work); in cluster mode the driver runs inside the cluster (survives the client disconnecting, used in production).

4.6 RDDs, transformations, actions and lazy evaluation (tricky topic)

- **RDD** = Resilient Distributed Dataset: partitioned across the cluster, **immutable**, typed, recoverable. Created by `sc.parallelize(...)`, `sc.textFile(...)` or by transforming another RDD.

- **Transformations** (lazy; return a new RDD): `map`, `filter`, `flatMap`, `distinct`, `union`, `reduceByKey`, `groupByKey`, `join`, `sortBy`. **Actions** (eager; return a value or write data): `count`, `collect`, `take`, `first`, `reduce`, `foreach`, `saveAsTextFile`.
- **Lazy evaluation**: transformations only extend the **lineage** (the DAG); an action triggers computation. Each new action recomputes the lineage unless the RDD is `cache()`d/`persist()`ed.
- **Fault tolerance**: a lost partition is recomputed from its lineage, so no data replication is needed. `toDebugString` shows the lineage.
- **Narrow** dependencies (map, filter, flatMap): each output partition depends on one input partition, pipelined in one stage. **Wide** (reduceByKey, groupByKey, join, distinct, repartition): data is shuffled across the network, which starts a new stage and is the main cost.

Verified output

```
--- narrow vs wide: look for the shuffle in the lineage ---
map/filter chain (narrow):
(2) MapPartitionsRDD[10] at filter at n4_spark_traps.scala:27 []
 |  MapPartitionsRDD[9] at map at n4_spark_traps.scala:27 []
 |  ParallelCollectionRDD[8] at parallelize at n4_spark_traps.scala:26 []
reduceByKey (wide):
(2) ShuffledRDD[11] at reduceByKey at n4_spark_traps.scala:28 []
 +-(2) ParallelCollectionRDD[8] at parallelize at n4_spark_traps.scala:26 []
reduceByKey = List((a,9), (b,6)) | groupByKey.mapValues(sum) = List((a,9), (b,6))  (same answer; reduceByKey
    combines inside each partition first, so it shuffles less)

--- laziness: nothing runs until an action ---
after map: accumulator = 0
count() action: 100 | accumulator now = 100
second action recomputes the lineage: accumulator = 200 (200 = 2 x 100)
with cache(): two actions but only 100 map calls: accumulator = 100 | storage level: Memory Deserialized 1x
    Replicated
```

4.6-Q1 An accumulator is incremented inside `map`. What does it read after `map`, after `count()`, and after a second `count()`? [tricky]

0, 100, 200. `map` did nothing (lazy); the first action ran the function for 100 elements; the second action recomputed the whole lineage. With `cache()` two actions gave only 100 calls (output).

4.6-Q2 Why does `reduceByKey` shuffle less than `groupByKey` for the same answer? [tricky]

`reduceByKey` first combines values inside each partition (map-side combine) so only partial sums cross the network; `groupByKey` ships every value. Both returned `(a,9), (b,6)` in the output.

4.6-Q3 Define RDD and its four properties; explain how lineage gives fault tolerance. [5 marks]

Resilient, Distributed, Dataset, Immutable. Each RDD records its parent and the function applied; if an executor fails, only the lost partitions are rebuilt by re-running those steps from the source data, with no replication cost during normal running.

4.6-Q4 Name three transformations and three actions and say why laziness helps. [5 marks]

Transformations: `map`, `filter`, `reduceByKey`. Actions: `count`, `collect`, `saveAsTextFile`. Because nothing runs before an action, Spark sees the whole plan, pipelines narrow steps into one pass, avoids storing intermediate results and can skip unused work.

4.6-Q5 Why can `collect()` crash the driver? [3 marks]

It copies the entire dataset into the driver's memory. Use `take(n)`, `takeOrdered`, aggregations, or write results to storage. The output uses `take(3)` on a 1,000,000-element RDD.

4.6-Q6 What does `glom()` show, and how many partitions does a shuffle produce? [3 marks]

`glom()` turns each partition into an array so you can see how data is split: `List(1,2,3), List(4,5,6), ...`. For RDDs a shuffle such as `reduceByKey` keeps the parent partition count unless you pass `numPartitions` (3 before and after in the output). For DataFrames it is `spark.sql.shuffle.partitions` (200).

4.6-Q7 What does persisting an RDD do and what is its default storage level? [3 marks]

`cache()` keeps the computed partitions in memory (`MEMORY_ONLY`, shown as 'Memory Deserialized 1x Replicated') so later actions skip recomputation; use `persist(StorageLevel.MEMORY_AND_DISK)` for data that may not fit.

4.7 Shared variables and closures (tricky topic)

- Code inside `foreach`/`map` runs on executors in separate JVMs. The closure (including captured variables) is **serialised and copied** to each task, so modifying a driver variable from there does not update the driver's copy.
- **Accumulators**: write-only-from-tasks counters that the driver can read (`sc.longAccumulator`). Reliable only when updated inside actions; updates inside transformations may repeat if a stage re-runs.
- **Broadcast variables**: a read-only value sent once to each executor (lookup tables) instead of shipping it with every task.

Verified output

```
--- closures and shared variables ---
driver variable updated inside foreach: counter = 0 (executors modified private copies; use an accumulator)
accumulator gives the right total: 55
broadcast variable (read-only copy per executor): List((Alpha,9), (Beta,6))
```

4.7-Q1 `var counter = 0; rdd.foreach(x => counter += x)`: what is `counter` on the driver afterwards? [tricky]

0 (output). Each task incremented its own deserialised copy. Use an accumulator: `sc.longAccumulator("sum")` gave 55 for 1 to 10. Spark's guide warns that behaviour of such code is undefined; in local mode it may even appear to work in some set-ups, so never rely on it.

4.7-Q2 When do you use a broadcast variable? [3 marks]

For a medium-sized read-only table used by many tasks (country codes, a lookup map). It is shipped once per executor rather than once per task. The output maps keys to names with `lookup.value(k)`.

4.7-Q3 Why can accumulator values be wrong when used inside transformations? [tricky]

A task may be re-executed (failure, speculation, or recomputation without cache), applying the update again. In the output the second `count()` doubled the value to 200.

4.8 Spark SQL, DataFrames, Datasets and Catalyst

- A **DataFrame** is a distributed table with a schema; a **Dataset[T]** adds compile-time types (case classes). Both are optimised by **Catalyst** and executed with Tungsten (compact binary memory and generated code). The Spark quick-start recommends them over raw RDDs.
- DataFrame API and SQL (`createOrReplaceTempView` + `spark.sql`) compile to the **same plan**. Actions: `show`, `collect`, `count`, `write`.
- **Catalyst phases**: analysis (resolve names and types) > logical optimisation (predicate pushdown, column pruning, constant folding, combine filters) > physical planning (choose strategies, e.g. broadcast join) > code generation.
- `explain(true)` prints the parsed, analysed, optimised and physical plans. Adaptive Query Execution (`AdaptiveSparkPlan`) can re-optimise at run time.

Verified output

```
--- DataFrame API vs SQL: same plan ---
API   : List(N/15.0/1, S/50.0/2, W/25.0/1)
SQL   : List(N/15.0/1, S/50.0/2, W/25.0/1)
same optimised plan: true
schema: region:string, product:string, amount:double
Dataset[Sale] is typed: List(book, book) | a typo like _.amout fails at compile time, while $"amout" fails only at
    run time
join: List(N/North, S/South, W/null) | W has no match -> null label
broadcast join hint is applied by Catalyst for small tables: true
```

Catalyst plans (question bank M4-C5)

```
result:
+----+-----------+
|dept|avg(salary)|
+----+-----------+
|  AI|   100000.0|
|  DB|    70000.0|
+----+-----------+

optimised logical plan and physical plan (excerpt):
  Sort [dept#4 ASC NULLS FIRST], true
  +- Aggregate [dept#4], [dept#4, avg(salary#5) AS avg(salary)#17]
     +- LocalRelation [dept#4, salary#5]
  AdaptiveSparkPlan isFinalPlan=false
  +- Sort [dept#4 ASC NULLS FIRST], true, 0
     +- Exchange rangepartitioning(dept#4 ASC NULLS FIRST, 200), ENSURE_REQUIREMENTS, [plan_id=107]
        +- HashAggregate(keys=[dept#4], functions=[avg(salary#5)], output=[dept#4, avg(salary)#17])
           +- Exchange hashpartitioning(dept#4, 200), ENSURE_REQUIREMENTS, [plan_id=104]
              +- HashAggregate(keys=[dept#4], functions=[partial_avg(salary#5)], output=[dept#4, sum#28, count#29L])
                 +- LocalTableScan [dept#4, salary#5]

analysed plan (as written):
  Project [(id#39L * cast(5 as bigint)) AS x#45L]
  +- Filter (id#39L < cast(50 as bigint))
     +- Filter (id#39L > cast(10 as bigint))
        +- Range (0, 100, step=1, splits=Some(2))
optimised plan (after Catalyst):
  Project [(id#39L * 5) AS x#45L]
  +- Filter ((id#39L > 10) AND (id#39L < 50))
     +- Range (0, 100, step=1, splits=Some(2))
rows kept: 39
```

4.8-Q1 Explain Catalyst's four phases with an example. [10 marks]

In the second query the two `filter` calls were merged into `(id > 10) AND (id < 50)` and `cast(5 as bigint)` was folded to the literal `5` (logical optimisation, output). Analysis resolved `id`, physical planning chose `Range`/`Project`/`Filter` operators, and whole-stage code generation compiled them to bytecode.

4.8-Q2 DataFrame versus Dataset versus RDD. [5 marks]

	RDD	DataFrame	Dataset[T]
Schema	no	yes (untyped Rows)	yes (typed)
Optimiser	none	Catalyst	Catalyst
Type safety	compile-time	run time	compile-time
Use	low-level control	SQL-style analytics	typed pipelines in Scala/Java

4.8-Q3 Is the DataFrame result the same via the API and via SQL? Why? [3 marks]

Yes: both returned `N/15.0/1, S/50.0/2, W/25.0/1` and the optimised plans were identical (output `same optimised plan: true`), because both are parsed into the same logical plan before Catalyst runs.

4.8-Q4 Why does a typo in `ds.filter(_.amout > 20)` fail earlier than `df.filter($"amout" > 20)`? [tricky]

The typed Dataset lambda is checked by the Scala compiler; the DataFrame column name is a string resolved at run time during analysis (`AnalysisException`).

4.8-Q5 What does a broadcast join do and when is it chosen? [3 marks]

The small table is sent to every executor so the large table need not be shuffled. Catalyst picks it automatically when one side is below `spark.sql.autoBroadcastJoinThreshold` or when you add `broadcast(df)` (the plan contained `BroadcastHashJoin`).

4.9 Spark components: Core, SQL, Streaming, MLlib, GraphX

Component	Purpose	Notes
Spark Core	RDD API, scheduling, memory management, fault recovery	foundation of everything else
Spark SQL	structured data: DataFrames, Datasets, SQL, Catalyst	read/write Parquet, JSON, CSV, JDBC
Structured Streaming	live data processed as an unbounded table with the DataFrame API	micro-batches; exactly-once with suitable sources/sinks; older RDD-based DStreams are legacy
MLlib	scalable machine learning: pipelines, regression, classification, clustering, recommendation	`DataFrame`-based API is the main one
GraphX	graph computation: PageRank, connected components	RDD-based; Pregel API

4.9-Q1 List the Spark ecosystem components and one use of each. [5 marks]

Spark Core (task scheduling and RDDs), Spark SQL (analytics on tables), Structured Streaming (real-time data), MLlib (distributed machine learning), GraphX (graph analytics).

4.9-Q2 How does Structured Streaming treat a stream? [3 marks]

As an unbounded table to which new rows keep being appended; your query (the same as for batch) is run incrementally on each micro-batch and updates the result.

4.9-Q3 What is an MLlib Pipeline? [2 marks]

A sequence of stages (transformers such as feature encoders and estimators such as logistic regression) fitted together, the Spark analogue of scikit-learn's `Pipeline`.

Module 5: Programming language selection criteria

Syllabus (6 hours): size of the deployment (data, resource and load); security; skill set; targeted platform; elasticity of a language; time to production; performance; support and community; purpose; programmer experience; ease of development and maintenance; efficiency; availability of an IDE; error checking and diagnosis.

Where each syllabus topic is covered

Syllabus topic	Section
All fourteen criteria	5.1
Comparing languages (Python, Scala, Java, R)	5.2
Applying the criteria to a case study	5.3

5.1 The language-selection criteria

The syllabus names these criteria. Your FAT notes group them into 13 (IDE and diagnostics together). For each one, know the **question to ask** and an example of how it can tip the decision.

Criterion	Question to ask	Typical lean
Size of the deployment (data, resource, load)	how much data, how many users or events per second, what hardware?	large and concurrent: Scala/Java on a cluster; small and exploratory: Python
Security	authentication, encryption, input validation, memory safety, dependency vetting?	mature enterprise stacks on the JVM; any language needs secure design and patched dependencies
Skill set	what does the team already know; who can we hire?	Python has the largest data-science pool; Scala engineers are fewer and specialised
The targeted platform	where will it run: JVM cluster, cloud, notebook, browser, device?	Spark/JVM: Scala, Java (or PySpark); embedded devices: C/C++
Elasticity of a language	can it serve a script, a service and a distributed pipeline, and adapt as needs change?	Scala and Python stretch widely; R is mostly analysis
Time to production	how fast can we deliver a working version?	Python for prototypes
Performance	CPU-bound speed, latency, throughput?	JIT-compiled JVM code is faster than CPython; Python is fast only through C libraries (NumPy)
Support and community	libraries, documentation, answers, long-term maintenance?	PyPI and Maven are huge; check that the key libraries are actively maintained
Purpose	ML, ETL, streaming, web, statistics?	ML: Python; Spark jobs: Scala; statistics: R
Programmer experience	readability, REPL/notebooks, learning curve?	Python is gentle; Scala is expressive but steeper
Ease of development and maintenance	refactoring safety, testing, typing?	static typing (Scala, Java) helps large code bases; Python needs tests and type hints
Efficiency	memory, CPU and energy cost per result?	JVM tuning vs Python per-object overhead
Availability of an IDE	debugger, profiler, completion, refactoring tools?	IntelliJ IDEA (Scala/Java), PyCharm/VS Code (Python), RStudio (R)
Error checking and diagnosis	are errors found at compile time or run time; how good are stack traces and debuggers?	static languages catch type errors before running; dynamic ones need tests and linters

5.1-Q1 Explain 'size of the deployment: data, resource and load'. [3 marks]

Data: gigabytes in pandas versus terabytes on Spark. Resource: memory, CPU and budget (a JVM service needs heap sizing). Load: concurrent users or events per second; high concurrency favours the JVM and its concurrency libraries.

5.1-Q2 Why can a language that is fast for development be slow in production, and what can be done? [3 marks]

CPython interprets bytecode and has per-object overhead; heavy loops are slow unless the work is vectorised (NumPy) or moved to compiled code. Remedies: vectorise, use PySpark DataFrames (they run in the JVM), rewrite hot paths in Scala/Java/C, profile first.

5.1-Q3 Explain the 'elasticity of a language'. [2 marks]

How well the same language covers small scripts through to large systems, and adapts to changing requirements. Scala (concise scripts to large typed systems) and Python (with packaging and type hints) score well.

5.1-Q4 Static versus dynamic typing for maintenance. [3 marks]

	Static (Scala, Java)	Dynamic (Python, R)
Type errors	compile time	run time
Refactoring large code	compiler guides you	needs good tests
Writing speed	more annotations (inference reduces them)	faster to write

5.2 Python, Scala, Java and R compared

	Python	Scala	Java	R
Typing	dynamic (hints optional)	static, inferred	static, verbose	dynamic
Runs on	CPython interpreter	JVM	JVM	R interpreter
Strength	ML ecosystem, readability, notebooks	Spark, streaming, functional + OO	enterprise services, huge ecosystem	statistics, graphics, reports
Weakness	slow in pure loops; GIL limits threads	steeper learning, slower compile, fewer people	verbose	weaker for production services
Typical use	analysis, ML, prototypes, glue	big-data pipelines, high-throughput jobs	back-end systems, integration	statistical modelling

- **PySpark versus Scala Spark**: DataFrame operations run in the JVM engine either way, so speed is similar; Python UDFs and RDD lambdas run in Python worker processes and pay serialisation cost. Scala gives typed Datasets and first access to new APIs.
- The 'best' language depends on the project: tie every recommendation to criteria and facts of the scenario.

5.2-Q1 Compare Python and Scala across the selection criteria. [10 marks]

Use the 14-row table in 5.1 plus: Python wins on time to production, skills and ML ecosystem; Scala wins on performance, load, platform fit with Spark, and maintainability of large code. Close with a recommendation for a stated scenario.

5.2-Q2 When would you choose R? [2 marks]

For statistical analysis, specialist CRAN packages, and publication-quality graphics or reports (R Markdown, Shiny), usually by statisticians and for small to medium data; less often for production services.

5.2-Q3 Is PySpark slower than Scala Spark? [tricky]

For DataFrame/SQL code it is about the same because the plan runs in the JVM. It is slower when Python functions run per row (UDFs, `rdd.map(lambda ...)`), because rows are serialised between the JVM and Python workers.

5.3 Weighted decision matrix (worked example)

- Method: list the criteria that matter; give each a weight (weights sum to 1); score each language 1 to 5 on each criterion; multiply, add, compare; sanity-check the winner against the scenario.
- The scores are judgements; the method makes the reasoning explicit and lets stakeholders change a weight and see the effect.

Code (tested)

```
"""M5  Weighted decision matrix for language selection (two case studies).
Scores are 1-5 judgement values chosen by the analyst - the point is the method.
"""
CRITERIA = ["Deployment size / load", "Performance", "Time to production", "Skill set available",
            "Ecosystem / community", "Maintainability", "Platform fit (Spark/JVM)"]
SCORES = {                      # criterion order as above
    "Python": [3, 2, 5, 5, 5, 3, 3],
    "Scala":  [5, 5, 3, 2, 4, 5, 5],
    "Java":   [5, 5, 3, 4, 5, 4, 4],
    "R":      [2, 2, 4, 2, 3, 2, 1],
}
CASES = {
    "Case A: real-time fraud detection, 5 TB/day, Spark cluster, 8 experienced JVM engineers":
        [0.25, 0.20, 0.05, 0.10, 0.05, 0.15, 0.20],
    "Case B: 6-week ML prototype for a startup, 3 data scientists, small CSV/Parquet data":
        [0.05, 0.05, 0.30, 0.25, 0.20, 0.05, 0.10],
}
for case, weights in CASES.items():
    assert abs(sum(weights) - 1.0) < 1e-9
    print(case)
    print("  weights:", dict(zip(CRITERIA, weights)))
    ranking = sorted(((sum(w * s for w, s in zip(weights, sc)), lang) for lang, sc in SCORES.items()),
                     reverse=True)
    for total, lang in ranking:
        print(f"    {lang:<7} weighted score = {total:.2f}")
    print("  -> choose", ranking[0][1], "\n")
```

Verified output

```
Case A: real-time fraud detection, 5 TB/day, Spark cluster, 8 experienced JVM engineers
  weights: {'Deployment size / load': 0.25, 'Performance': 0.2, 'Time to production': 0.05, 'Skill set available':
      0.1, 'Ecosystem / community': 0.05, 'Maintainability': 0.15, 'Platform fit (Spark/JVM)': 0.2}
    Scala   weighted score = 4.55
    Java    weighted score = 4.45
    Python  weighted score = 3.20
    R       weighted score = 1.95
  -> choose Scala

Case B: 6-week ML prototype for a startup, 3 data scientists, small CSV/Parquet data
  weights: {'Deployment size / load': 0.05, 'Performance': 0.05, 'Time to production': 0.3, 'Skill set available':
      0.25, 'Ecosystem / community': 0.2, 'Maintainability': 0.05, 'Platform fit (Spark/JVM)': 0.1}
    Python  weighted score = 4.45
    Java    weighted score = 4.00
    Scala   weighted score = 3.45
    R       weighted score = 2.70
  -> choose Python
```

5.3-Q1 Case A: fraud detection at 5 TB/day on Spark with eight JVM engineers. Case B: a three-person start-up ML prototype in six weeks. Which language for each? [10 marks]

Case A: Scala (4.55, Java 4.45, Python 3.20) because throughput, load, Spark fit and maintainability dominate. Case B: Python (4.45, Java 4.00, Scala 3.45) because time to production, skills and ecosystem dominate. If Case B later reaches terabytes, change the weights and decide again.

5.3-Q2 Compute Scala's weighted score for Case A: scores 5, 5, 3, 2, 4, 5, 5 with weights 0.25, 0.20, 0.05, 0.10, 0.05, 0.15, 0.20. [tricky]

1.25 + 1.00 + 0.15 + 0.20 + 0.20 + 0.75 + 1.00 = 4.55.

5.3-Q3 What are the weaknesses of a weighted decision matrix? [3 marks]

Subjective scores and weights, false precision, criteria that overlap and double count, and neglect of one deal-breaker (for example a mandatory platform). Use it to structure the discussion, not to replace it.

Module 6: Contemporary issues

Syllabus (2 hours): no topics are listed in the syllabus.

Where each syllabus topic is covered

Syllabus topic	Section
Privacy, security, ethics	6.1
Bias and responsible AI	6.2
Generative AI and coding assistants	6.3
Drift, monitoring, reproducibility	6.4

6.1 Privacy, security and ethics in data work

Watch out. The syllabus gives Module 6 no topic list. These sections cover themes that usually come under 'contemporary issues'. Confirm with your faculty which topics were taught.

- **Personal data** identifies a person directly or with other data. Laws to know by name: the EU General Data Protection Regulation (GDPR) and India's Digital Personal Data Protection Act, 2023.
- Principles: lawful basis and consent, purpose limitation, **data minimisation** (collect only what is needed), storage limitation (delete when no longer needed), accuracy, security, accountability.
- **Anonymised** data cannot be linked back to a person; **pseudonymised** data can, using separately kept extra information, so it is usually still personal data.
- Security basics: least-privilege access, encryption at rest and in transit, audit logs, secrets in a vault or environment variables (never in code), patched dependencies.

6.1-Q1 List practices for handling personal data in a pipeline. [5 marks]

Get a lawful basis and consent; minimise the fields; pseudonymise or anonymise before analysis; control access; encrypt; log access; set retention limits; document lineage so you can show where data came from and who used it.

6.1-Q2 Anonymisation versus pseudonymisation. [tricky]

Anonymisation is irreversible; pseudonymisation replaces identifiers with tokens but keeps a mapping, so the data can be re-identified by whoever holds it and is still regulated as personal data.

6.1-Q3 Why should API keys not be in source code? [2 marks]

Source code is shared and committed to version control; a leaked key can be abused. Keep it in an environment variable or secrets manager and rotate it if exposed.

6.2 Bias, fairness and responsible AI

- Bias enters through unrepresentative data, historical prejudice in labels, proxy features (a postcode standing in for a protected attribute) and evaluation only on the overall average.
- Mitigation: audit and rebalance data, review features, evaluate metrics **per group**, set fairness checks before release, keep a human in the loop, document the model (purpose, limits) and allow appeals.

6.2-Q1 Where does bias in a machine-learning system come from and how can it be reduced? [5 marks]

Source	Mitigation
unrepresentative or historically biased training data	audit, rebalance, collect missing groups
proxy features	review and constrain features
biased labels	clear guidelines, several annotators
only the overall metric	report metrics per group

6.2-Q2 A model is 95% accurate overall but wrong for 40% of one minority group. Is it acceptable? [3 marks]

Not without investigation. The overall average hides a large per-group error. Report per-group error rates (compare with the metrics in 2.10), find the cause (data coverage, features) and fix or restrict use.

6.3 Generative AI and AI coding assistants

- Large language models generate plausible text and code; they can **hallucinate** (fluent but wrong or invented answers), reproduce outdated or insecure patterns, and raise privacy and licensing questions when private data or code is pasted in.
- Good practice: treat output as a draft, run and test it, check statistics and API usage against documentation, never paste secrets or personal data, cite assistance where required, and be able to explain every line.

6.3-Q1 Benefits and risks of AI coding assistants for data-science programming? [5 marks]

Benefits: faster boilerplate, explanations, test ideas. Risks: wrong or hallucinated code, subtle statistical errors (for example data leakage as in 2.10), insecure code, over-reliance that weakens understanding, and data privacy. Mitigate by testing, reading documentation and keeping private data out.

6.3-Q2 Why can an assistant's pipeline code that 'works' still be wrong? [tricky]

It may run without errors while leaking test data into training, scaling before the split, or mis-specifying a join. Run the checks in these notes (pipelines, `validate=` in merges) and compare results by hand on small data.

6.4 Model and data drift, monitoring, reproducibility

- **Data drift**: the distribution of incoming data changes. **Concept drift**: the relationship between inputs and target changes. Either lowers accuracy after deployment.
- Monitor input statistics and model metrics, alert on thresholds, retrain on a schedule or trigger, version data, code and models, and fix random seeds so results can be reproduced.

6.4-Q1 What is drift and how do you handle it? [3 marks]

A change after deployment in the data or in the input-target relationship. Handle it with monitoring, alerts and retraining on recent data, plus versioned data and models so you can roll back.

6.4-Q2 Why do these notes set `random_state`/`default_rng(seed)` in every demo? [2 marks]

Reproducibility: with a fixed seed the same numbers appear on each run, so results can be checked, compared and debugged.

Cheat sheet and last-minute revision

Costs in Python

Structure	Index / key	Search / membership	Add / remove
list	O(1)	O(n)	end O(1), front/middle O(n)
tuple	O(1)	O(n)	immutable
dict	O(1) avg	O(1) avg (by key)	O(1) avg
set	-	O(1) avg	O(1) avg
deque	ends O(1)	O(n)	both ends O(1)
heapq	peek O(1)	O(n)	push/pop O(log n)

Python and Scala side by side

Idea	Python	Scala
constant / variable	`x = 5` (no constants)	`val x = 5` / `var x = 5`
function	`def f(a, b): return a + b`	`def f(a: Int, b: Int) = a + b`
lambda	`lambda x: x * 2`	`(x: Int) => x * 2` or `_ * 2`
list index	`a[0]`	`a(0)`
map / filter	`[f(x) for x in xs if p(x)]`	`xs.filter(p).map(f)`
f-string	`f'{x:.2f}'`	`f"$x%.2f"`
class	`class A(B): def __init__(self, x): ...`	`class A(val x: Int) extends B`
no value	`None`	`None` (inside `Option`)
exceptions	`try / except / finally`	`try / catch { case ... } / finally`, `Try`
dictionary	`{'a': 1}`	`Map("a" -> 1)`
equality	`==` value, `is` identity	`==` value, `eq` identity

Formulas

Topic	Formula
Z-score, min-max	z = (x - mean) / std; x' = (x - min) / (max - min)
Covariance (sample)	C = XcT Xc / (n - 1)
2 x 2 eigenvalues	lambda^2 - trace lambda + det = 0
PCA explained ratio, reconstruction error	lambda_i / sum(lambda); SSE = (n - 1) x sum of discarded lambda
K-means inertia	sum over clusters of sum ||x - centroid||^2
Precision, recall, F1	TP/(TP+FP); TP/(TP+FN); 2PR/(P+R)
Gradient descent	w = w - lr x dL/dw; MSE gradient dL/dw = mean(2 (pred - y) x)
Dense layer parameters	inputs x units + units
TF-IDF	tf x idf, idf = ln((1+N)/(1+df)) + 1 in scikit-learn
Weighted score	sum of weight x score

One-line facts to revise

Python

- `list.append` and `pop()` are O(1); `insert`, `pop(0)`, `in` are O(n). Use `deque.popleft()` for queues.
- Dict and set lookups are O(1) on average; keys and members must be hashable (tuples yes, lists no).
- A tuple is immutable, but it may hold a mutable list; `t[0] += [x]` raises `TypeError` after mutating the list.
- `list.sort()` returns `None`; `sorted()` returns a new list. `[[0]*2]*2` repeats one inner list.
- Default argument values are created once; a mutable default is shared between calls.
- `finally` always runs; `else` runs when no exception occurred; put specific `except` clauses first.
- `self.__x` is stored as `_Class__x`; MRO for `class D(B, C)` is D, B, C, A.
- `heapq` is a min-heap; push negated priorities for a max-heap; add a counter to break ties.
- GET is safe and idempotent; POST creates; retry 429 (after Retry-After) and 5xx, not 404; always set `timeout=`.

- `open(..., 'w')` truncates, `'a'` appends, `'x'` fails if the file exists; JSON turns tuples into lists and int keys into strings.
- Lemmatization needs the part of speech (`better` -> `good` only with `pos='a'`); stemming can give non-words.

pandas, NumPy and machine learning

- `loc` slices by label and includes the end; `iloc` by position and excludes the end.
- Combine masks with `&`, `|` and parentheses; assign through one `df.loc[mask, col] = v`.
- Series arithmetic aligns on index labels; missing labels give NaN.
- A many-to-many merge multiplies rows; `validate=` and `indicator=True` help. `agg` reduces groups, `transform` keeps the length.
- NumPy: basic slices are views, fancy and boolean indexing copy; `*` is element-wise, `@` is the matrix product; broadcasting right-aligns shapes (equal or 1).
- PCA: centre (and usually scale), covariance, eigen-decomposition, sort, project. Eigenvalue = variance on that PC; ratios sum to 1; signs are arbitrary.
- K-means finds a local optimum (use `n_init`); scale first; choose k with elbow or silhouette.
- Fit scalers, selectors and PCA on training data only (use a Pipeline); otherwise results are over-optimistic.
- Accuracy misleads on imbalanced data: use precision, recall, F1; recall = TP/(TP+FN).
- PyTorch: `zero_grad` each step, int64 class labels; gradient step is w = w - lr x gradient. Early stopping needs a validation set, not the test set.

Scala and Spark

- `val` fixed reference, `def` evaluated at every use, `lazy val` once at first use; by-name `=> T` re-evaluates at each use.
- `foldLeft` and `foldRight` differ for non-associative operations (subtraction): -10 versus -2 for 1 to 4.
- `::` is right-associative and prepends in O(1); `List(i)` is O(n); `Array(1,2) == Array(1,2)` is false (use `sameElements`); `'5'.toInt` is 53.
- Case class = fields + `apply` + `unapply` + `equals`/`hashCode`/`toString` + `copy` (shallow).
- Companion object: same name and file, reads private members, `apply` removes `new`.
- Trait linearization is right to left: `D extends B with C` gives D, C, B, A; the order of `with` changes behaviour.
- An overridden `val` is `null` (or 0) inside the superclass constructor; use a `def` or `lazy val`.
- `Try` captures non-fatal exceptions; `NonFatal` excludes fatal ones; `.get` on a `Failure` re-throws; `Either` carries a typed error.
- Spark: transformations are lazy, actions run jobs; wide transformations shuffle and start a new stage; each action recomputes the lineage unless cached.
- `reduceByKey` combines before the shuffle; `groupByKey` ships every value. `collect()` copies everything to the driver.
- Driver variables are not updated from tasks; use accumulators (counters) and broadcast variables (read-only lookups).
- Catalyst: analysis, logical optimisation, physical planning, code generation. DataFrame shuffles use 200 partitions by default.

Index of tricky questions

Every question tagged tricky or predict-the-output, with its number. Use it as a final revision list.

Module 1: Python data structures, OOP, APIs and NLP

No.	Question
1.1-Q1	What does print(0 or 'x', '' and 5, 3 and 4) print?
1.1-Q2	A grading function uses if m >= 50: 'C' elif m >= 70: 'B' elif m >= 85: 'A'. What is wrong?
1.1-Q4	Why does 0.1 + 0.2 == 0.3 give False, and how do you compare floats?
1.2-Q2	After a = [3, 1, 2]; b = a; c = a[:]; b.append(9), what are a and c?
1.2-Q3	What does grid = [[0] * 2] * 2; grid[0][0] = 1 give? How do you build a real 2 x 2 grid?
1.2-Q4	What is the output of bad(1), bad(2) for def bad(x, acc=[]): acc.append(x); return acc?
1.2-Q5	What does print(a.sort()) print and why?
1.3-Q2	t = ([1], 2). Does t[0].append(3) work? What does t[0] += [4] do?
1.4-Q1	Why can a tuple but not a list be a dictionary key?
1.4-Q2	What happens if you add keys to a dictionary while looping over it?
1.5-Q1	What are type({}) and type(set())?
1.6-Q1	Predict data and num after grow(data), rebind(data), bump(num) for the three functions in the output.
1.6-Q2	nested = [[1, 2], [3]]; sh = nested.copy(); nested[0].append(99). What is sh?
1.9-Q1	data = [5, 3, 8, 1, 9, 2]; heapq.heapify(data). Is data now sorted?
1.9-Q2	Two tasks have the same priority and the second field is a custom object. What goes wrong with heappush((1, obj))?
1.10-Q1	What does 'xxhixx'.strip('x') return? What does 'abcabc'.strip('ba') return?
1.10-Q2	Difference between find and index; split() and split(' ').
1.11-Q1	What is printed by parse(None) for the function in the output and in what order do the messages appear?
1.11-Q2	What does g() return: try: return 'from try' finally: return 'from finally'?
1.12-Q1	A class has items = [] at class level and add appends to self.items. What goes wrong with two objects?
1.12-Q3	For class D(B, C) where both extend A, in what order are who() methods called, and why?
1.12-Q6	Why define __hash__ when you define __eq__?
1.13-Q2	Why does the printed URL contain q=a+b%26c?
1.14-Q1	What does opening an existing file with mode 'w' do?
1.14-Q3	Why can f.seek(6) land in the wrong place on Windows in text mode?
1.14-Q4	What changes when a Python object goes through json.dumps and json.loads?
1.15-Q1	Stemming versus lemmatization: compare the outputs for studies, studying, geese, running, better, was.
1.15-Q2	Why can removing stopwords hurt sentiment analysis?

Module 2: Python libraries for data science

No.	Question
2.1-Q1	What is the result of pd.Series([10,20,30], index=list('abc')) + pd.Series([1,2,3], index=list('bcd'))?
2.2-Q1	For the default index, how many rows do df.loc[1:3] and df.iloc[1:3] return?
2.2-Q2	Why does df[df.age > 25 and df.city == 'Delhi'] fail, and what is the fix?
2.2-Q3	d2[d2.age > 30]['score'] = 0 leaves the data unchanged and warns. Why? How do you do it correctly?
2.4-Q4	What does pd.cut(score, [0, 70, 85, 100], labels=['C','B','A']) do with a mark of exactly 70?
2.5-Q1	Why does filtering with df[df.name.str.contains('a')] sometimes raise an error?
2.6-Q2	Difference between agg and transform.
2.6-Q3	Two tables each have two rows with k = 1. How many rows does merge(a, b, on='k') return? How do you guard against this?
2.7-Q3	A page shows data in the browser but soup.find_all finds nothing. Why?
2.8-Q1	State the broadcasting rules and decide whether (3,4)+(4,), (3,4)+(3,1), (3,4)+(3,) work.
2.8-Q2	v = a[0, :2]; v[0] = 99. Does a change? What about c = a[[0, 1]]; c[0, 0] = -1?
2.8-Q4	Why does np.array([250], dtype=np.uint8) + 10 equal 4?
2.10-Q2	A model predicts 'no fraud' for everyone on a data set with 5% fraud. What are its accuracy and recall?
2.10-Q3	What is data leakage? Give an example the output demonstrates.
2.11-Q3	Compute the covariance matrix for the five points above.
2.11-Q4	Find the eigenvalues of C by hand and the percentage of variance on PC1.
2.11-Q5	Find the PC1 direction and the score of the first point.
2.11-Q6	How much information is lost keeping only PC1 here?
2.11-Q7	Why must the data be centred? Why standardise?
2.11-Q8	Why does PCA on raw Iris give 92.5% for PC1 but 73.0% after standardising?
2.11-Q11	Is a PC with high variance always useful for prediction?
2.11-Q12	The signs of components differ between my hand calculation and scikit-learn. Is that a bug?
2.12-Q2	Why can two runs of K-means give different answers?
2.12-Q4	Why must features be scaled before K-means?
2.13-Q1	Do one gradient-descent step for y = w x + b with x = [1,2,3], y = [2,4,6], w = b = 0, learning rate 0.1 (MSE loss).
2.13-Q2	PyTorch loss stays wrong / gradients look doubled. What is the usual cause?
2.13-Q3	CrossEntropyLoss raises expected target dtype to be Long or Byte, but got Int. Why?

Module 3: Scala data structures and object-oriented programming

No.	Question
3.1-Q1	What do v twice, d twice and l twice print for val v = next(), def d = next(), lazy val l = next()?
3.1-Q2	Explain call-by-value and call-by-name using byValue(next()) and byName(next()).
3.1-Q4	A closure val times = (x: Int) => x * factor is defined with var factor = 2, then factor = 10. What is times(3)?
3.2-Q4	What is the difference between "5".toInt and '5'.toInt?
3.3-Q1	What are 7 / 2, 7 / 2.0, -7 % 3 and Int.MaxValue + 1?
3.3-Q2	Difference between == and eq; predict new String("hi") == new String("hi") and ... eq ....
3.4-Q2	What is 1 :: 2 :: Nil? Why is :: written on the left of the list?
3.4-Q6	val l = ListBuffer(1, 2); l += 3; l.prepend(0): is that allowed with val?
3.5-Q1	Compute List(1,2,3,4).foldLeft(0)(_ - _) and foldRight(0)(_ - _).
3.5-Q4	Trace reduce, fold and scanLeft on an empty list.
3.5-Q6	What does for { i <- 1 to 3; j <- 1 to 2 if i != j } yield (i, j) produce?
3.6-Q2	Predict describe for 0, -5, "scala", (1,2), Nil, List(1,2,3).
3.6-Q3	What does val whole @ List(first, _*) = List(9, 8, 7) bind?
3.7-Q2	What is the difference between class P(x: Int), class P(val x: Int) and class P(var x: Int)?
3.7-Q3	What is printed when Derived overrides a val that Base's constructor uses?
3.8-Q3	What is Secret.peek(Secret(21)) in the sample and why can it be called?
3.9-Q2	Team("A", Array("x","y")) == Team("A", Array("x","y")): true or false?
3.11-Q1	What does (new D).who print for trait A {def who="A"}, trait B extends A {override def who="B > "+super.who}, trait C extends A {... "C > "...}, cl...
3.11-Q2	Show that the order of mixins changes the result: Greeter with Loud with Polite versus Greeter with Polite with Loud.
3.11-Q4	Compare linearization in Scala with Python's MRO.
3.13-Q2	Why does Array(1,2) == Array(1,2) give false, and what is the fix?

Module 4: Scala libraries and Spark basics

No.	Question
4.1-Q2	Difference between *:* and * in Breeze.
4.1-Q4	How do you do PCA in Breeze?
4.3-Q1	What is printed first for f(2): the finally message or the returned value, and what does f(0) return?
4.3-Q2	NumberFormatException is a subclass of IllegalArgumentException. In which order should the cases be written?
4.3-Q5	Does Try(parse("x").get) throw?
4.5-Q3	repartition(2) versus coalesce(2); can coalesce(8) increase partitions?
4.6-Q1	An accumulator is incremented inside map. What does it read after map, after count(), and after a second count()?
4.6-Q2	Why does reduceByKey shuffle less than groupByKey for the same answer?
4.7-Q1	var counter = 0; rdd.foreach(x => counter += x): what is counter on the driver afterwards?
4.7-Q3	Why can accumulator values be wrong when used inside transformations?
4.8-Q4	Why does a typo in ds.filter(_.amout > 20) fail earlier than df.filter($"amout" > 20)?

Module 5: Programming language selection criteria

No.	Question
5.2-Q3	Is PySpark slower than Scala Spark?
5.3-Q2	Compute Scala's weighted score for Case A: scores 5, 5, 3, 2, 4, 5, 5 with weights 0.25, 0.20, 0.05, 0.10, 0.05, 0.15, 0.20.

Module 6: Contemporary issues

No.	Question
6.1-Q2	Anonymisation versus pseudonymisation.
6.3-Q2	Why can an assistant's pipeline code that 'works' still be wrong?

Appendix: how the demonstrations were produced

Programs (Question_Bank/code)	Runtime	Covers
n1_basics, n1_structures, n1_strings_exceptions, n1_oop, n1_queues	Python 3.10	sections 1.1 to 1.12
n1_files_rest	Python 3.10, requests; a local HTTP server on 127.0.0.1	1.13, 1.14 (real HTTP status codes, no internet service)
n1_nlp	NLTK 3.10.4, scikit-learn	1.15
n1_chatterbot	ChatterBot 1.2.15 with spaCy en_core_web_sm	1.16
n2_pandas_a, n2_pandas_b	pandas 2.3.3	2.1 to 2.6
n2_numpy_scipy	NumPy 1.26.4, SciPy 1.15.3	2.8, 2.9
n2_pca, n2_ml	scikit-learn 1.7.2	2.10 to 2.12
n2_dl	TensorFlow 2.21, Keras 3.12, PyTorch 2.14 (CPU), XGBoost 3.2	2.13, 2.14
n2_viz and the question-bank programs m2_q4, m2_q5	Matplotlib, Seaborn, Plotly, Bokeh, Folium, Beautiful Soup, Scrapy	2.7, 2.15
n3_scala_traps, n3_oop_traps, m3_q1 to m3_q4	Scala 2.13.12, Java 17, scala-cli	3.1 to 3.13
n4_try_breeze, m4_q1 to m4_q3	Breeze 2.1.0, Saddle 4.0.0-M14, Spire 0.18.0	4.1 to 4.3
n4_spark_traps, m4_q4	Spark 3.5.1, local mode	4.4 to 4.9
m5_decision_matrix	Python 3.10	5.3

Not verified by running

- Interactive charts (Plotly, Bokeh, Folium) were generated as HTML files but not opened in a browser.
- The Scala REPL was not recorded; the same expressions were run as compiled programs.
- Spark ran in local mode on one machine, not on a real cluster; cluster-manager behaviour and client/cluster deploy modes are described, not demonstrated.
- Section 6 is general; the syllabus gives Module 6 no topics.
- Timings (such as set versus list membership, deque versus list) were measured on one machine and will differ on yours.

Run it yourself

```
python run_and_capture.py n2_pca.py            # output goes to outputs/n2_pca.out.txt
python run_and_capture.py n4_spark_traps.scala # needs scala-cli and Java 17; downloads Spark on first run
python build_notes.py                          # rebuilds this PDF
```